



A comprehensive survey on pretrained foundation models: a history from BERT to ChatGPT

Ce Zhou¹ · Qian Li² · Chen Li² · Jun Yu³ · Yixin Liu³ · Guangjing Wang¹ · Kai Zhang³ · Cheng Ji² · Qiben Yan¹ · Lifang He³ · Hao Peng² · Jianxin Li² · Jia Wu⁴ · Ziwei Liu⁵ · Pengtao Xie⁶ · Caiming Xiong⁷ · Jian Pei⁸ · Philip S. Yu⁹ · Lichao Sun³

Received: 6 October 2024 / Accepted: 19 October 2024 / Published online: 24 November 2024
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2024

Abstract

Pretrained Foundation Models (PFMs) are regarded as the foundation for various downstream tasks across different data modalities. A PFM (e.g., BERT, ChatGPT, GPT-4) is trained on large-scale data, providing a solid parameter initialization for a wide range of downstream applications. In contrast to earlier methods that use convolution and recurrent modules for feature extraction, BERT learns bidirectional encoder representations from Transformers, trained on large datasets as contextual language models. Similarly, the Generative Pretrained Transformer (GPT) method employs Transformers as feature extractors and is trained on large datasets using an autoregressive paradigm. Recently, ChatGPT has demonstrated significant success in large language models, utilizing autoregressive language models with zero-shot or few-shot prompting. The remarkable success of PFMs has driven significant breakthroughs in AI, leading to numerous studies proposing various methods, datasets, and evaluation metrics, which increases the demand for an updated survey. This study provides a comprehensive review of recent research advancements, challenges, and opportunities for PFMs in text, image, graph, and other data modalities. It covers the basic components and existing pretraining methods used in natural language processing, computer vision, and graph learning, while also exploring advanced PFMs for different data modalities and unified PFMs that address data quality and quantity. Additionally, the review discusses key aspects such as model efficiency, security, and privacy, and provides insights into future research directions and challenges in PFMs. Overall, this survey aims to shed light on the research of the PFMs on scalability, security, logical reasoning ability, cross-domain learning ability, and user-friendly interactive ability for artificial general intelligence.

Keywords Pretrained foundation models · Natural language processing · Computer vision · Graph learning · ChatGPT · BERT · GPT-4

1 Introduction

Pretrained Foundation Models (PFMs) are regarded as essential and significant components of Artificial Intelligence (AI) in the era of big data. The foundation model is first named in [1], which means a broader class of models and their

Ce Zhou, Qian Li, Chen Li, Jun Yu and Yixin Liu contributed equally to this work.

✉ Ce Zhou
zhouce@msu.edu

✉ Qian Li
liqian@act.buaa.edu.cn

¹ Michigan State University, East Lansing, MI, USA

² Beihang University, Beijing, China

³ Lehigh University, Bethlehem, PA, USA

⁴ Macquarie University, Sydney, Australia

⁵ Nanyang Technological University, Singapore, Singapore

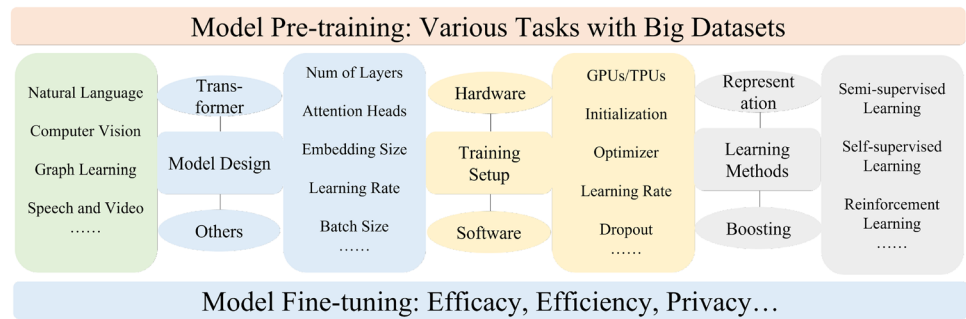
⁶ University of California San Diego, San Diego, CA, USA

⁷ Salesforce AI Research, Palo Alto, CA, USA

⁸ Duke University, Durham, NC, USA

⁹ University of Illinois at Chicago, Chicago, IL, USA

Fig. 2 The general conceptual architecture of PFMs: data, model, and system



First, minor fine-tuning is required to enhance the model performance on downstream tasks. Second, the PFMs have already been vetted on the quality aspect. Instead of building a model from scratch to solve a similar problem, we can apply PFMs to task-related datasets. The great promise of PFMs has inspired a wealth of related work to focus on the model efficiency [30], security [31–34] and compression [35, 36].

1.2 Contribution and organization

There are several survey studies [1, 5–8, 37] that have reviewed the pretrained models for some specific areas such as text generation [6], visual transformer [7], objection detection [8].

Bommasani et al. [1] summarize the opportunities and risks of the foundation model. However, existing works did not achieve a comprehensive review of PFMs in different areas (e.g., CV, NLP, GL, Speech, Video) and different aspects such as pretraining tasks, efficiency, efficacy, and privacy. In this survey, we specifically track the evolution of PFMs in the NLP domain, as well as how pretraining is transferred to and adopted by CV and GL. Compared with other surveys, there is no comprehensive introduction and analysis of existing PFMs from all three fields. Unlike reviews of previous pretrained models, we summarize existing models ranging from traditional models to PFMs with the most state-of-the-art methods by the time of writing² in the three domains. Traditional models emphasize static feature learning. Dynamic PFMs give an introduction to structures, which is the mainstream research. We further present some other research for PFMs, including other advanced and unified PFMs, model efficiency and compression, security, and privacy. Finally, we summarize future research challenges and open problems in different domains. We also comprehensively present the related evaluation metrics and datasets in Appendix E and F. In summary, the main contributions are as follows:

- We present a solid and up-to-date review of the development of PFM in NLP, CV, and GL. Over the review, we discuss and provide insights about the generalized PFM design and pretraining methodology among the three major application domains.
- We summarize the development of PFMs in other multimedia areas such as speech and video. Besides, we discuss advanced topics about PFMs, including unified PFMs, model efficiency and compression, and security and privacy.
- Through the review of PFMs in various modalities for different tasks, we discuss the main challenges and opportunities for future research of very large models in the big data era, which guides a new generation of collaborative and interactive intelligence based on PFMs.

The rest of the survey is organized as follows. Section 2 introduces the basic components. Sections 3, 4 and 5 summarize the existing PFMs in NLP, CV and GL, respectively. Sections 6, 7 introduce other advanced research for PFMs, including advanced and unified PFMs, model efficiency and compression, as well as security and privacy, respectively. Furthermore, we summarize the main challenges for PFMs in Sect. 8 before concluding the survey in Sect. 9.

2 Basic components

The general conceptual architecture of PFMs is shown in Fig. 2. The PFMs are huge neural network models, which are all about neural information processing. The specific designs of PFMs vary according to the data modality and task requirements in different areas. Transformer is a mainstream model architecture design for PFMs in many areas such as NLP and CV. Training large models needs to have various datasets for model pretraining. After training the PFMs, the model should be fine-tuned to satisfy downstream requirements such as efficacy, efficiency, and privacy. In this section, we introduce the basic model architectures, concepts, and settings of PFMs in NLP, CV, and GL domains.

² March 2023.

For the introduction of a more detailed component, please refer to Appendix A.

2.1 Transformer for PFMs

The Transformer [38] is an innovative architecture that facilitates the transfer of weighted representation knowledge between various neural units. It relies solely on attention mechanisms and doesn't use recurrent or convolutional architectures. The attention mechanism is a crucial component of the Transformer as it assigns weights to all the encoded input representations and learns the most important part of the input data. The output of the attention is obtained by taking the weighted sum of the values, and the weights are calculated using the compatibility function of the query with the corresponding key [38]. Numerous attention mechanisms [39] have been developed in large models. For instance, in natural language processing, self-attention is created to connect various positions in a single sequence to generate a representation of the same sequence. Transformer leverages a mask matrix to provide an attention mechanism based on self-attention, in which the mask matrix specifies which words can “see” each other.

The transformer is an important structure for PFMs in NLP, CV, and GL areas. For NLP, the Transformer can help solve the long-range dependency issues when processing sequential input data. For example, the GPT-3 [20] is a generative model based on the transformer. For CV, the Vision Transformer (ViT) [40] is proposed to represent an image to a series of image patches, which is similar to a series of word embeddings. For GL, the Graph Transformer Networks (GTN) [41] are employed to learn new graph structures and powerful node representations without domain knowledge. Transformers enable groundbreaking scalability for PFMs, thanks to their architecture, which supports greater parallelization. The ViT-22B model [42], for instance, has about 22B parameters, and the largest language models can have upwards of 100B parameters (e.g., GPT-3 has 175B and PaLM [43] has 540B parameters).

2.2 Learning mechanisms for PFMs

Deep learning models in CV have been shown a large margin to outperform traditional learning models in most tasks, including the common classification, recognition, detection, and segmentation tasks and the specific matching, tracking, and sequence prediction. These learning methods are not only available in CV, but also in NLP and GL.

Supervised learning

Suppose we are given a training dataset X containing $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ to represent the original data in training dataset,

where $\mathbf{x}_i$ denotes the i th training sample, and y_i denotes the corresponding label. The complete network is to learn a function $f(\mathbf{x}; \theta)$ by minimizing the objective function as follows.

$$\arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(\mathbf{x}_i; \theta), y_i) + \lambda \Omega(\theta), \quad (1)$$

where $\mathcal{L}$ and Ω represent the predefined loss function and a regularization term, respectively. The function f has a nested form like

$$\begin{aligned} \mathbf{h}_1(\mathbf{x}_i) &= g(\mathbf{x}_i^\top \boldsymbol{\omega}_1 + b_1), \\ \mathbf{h}_{l+1}(\mathbf{x}_i) &= g(\mathbf{h}_l(\mathbf{x}_i)^\top \boldsymbol{\omega}_l + b_l), \quad l = 1, 2, \dots, N \end{aligned} \quad (2)$$

where l is the index of layer in deep learning model and N is the number of layers, which means that $\theta = \{\boldsymbol{\omega}_l, b_l, l = 1, 2, \dots, N\}$.

Semi-supervised learning

Assume we are given another unlabelled dataset $Z = \{\mathbf{z}_i\}_{i=1}^m$ in addition to the previous dataset with human labels. If we want to utilize both datasets to learn an ideal network, the learning process can be formulated as

$$\begin{aligned} \arg \min_{\theta} \quad & \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(\mathbf{x}_i; \theta), y_i) \\ & + \frac{1}{m} \sum_{i=1}^m \mathcal{L}'(f'(\mathbf{z}_i; \theta'), R(\mathbf{z}_i, X)) + \lambda \Omega(\theta), \end{aligned} \quad (3)$$

where R is a relation function defining the targets for unlabelled data, and then these pseudo-labels are integrated into the end-to-end training process. f' is an encoder to learn a new representation for the original data in the dataset Z . Specifically, if there is no label to any data in the training process, we can learn from the properties inside the data itself via the internal distance or the designed pretext tasks, which are known as unsupervised learning and self-supervised learning (SSL), respectively. The latter is our main focus discussed in detail in Sect. 4.3.

Weakly-supervised learning

The weakly-supervised method is the balance between fully-supervised learning and SSL according to the dependence on human labels. The SSL designs special pretext tasks to serve as the supervised learning, but the fully supervised learning utilizes existing labels attached to the data. However, both of them can learn good visual features and perform well on specific downstream tasks. Suppose there are inaccurate K labels for the dataset, and any label can be attached to a data sample. Thus, we denote the true label of image $\mathbf{x}_i$ as $\mathbf{y}_i \in \{0, 1\}^K, i = 1, 2, \dots, n$, and any entry of $\mathbf{y}_i$

could be 0 or 1. Here we need to minimize the total nK loss terms, which are formulated as follows.

$$\arg \min_{\theta} \frac{1}{nK} \sum_{i=1}^n \sum_{k=1}^K \mathcal{L}(f(\mathbf{x}_i; \theta), y_i^k) + \lambda \Omega(\theta), \quad (4)$$

where $[y_i^1, y_i^2, \dots, y_i^K] = \mathbf{y}_i$, and $\mathcal{L}$ could be a loss function suitable for binomial classification problem. For any entry in $\mathbf{y}_i$, computing the loss function of the one-versus-all binomial classification is needed.

Self-supervised learning

SSL utilizes the information in the data itself to learn essential feature representations for different tasks. By applying the self-defined pseudo labels, it can avoid the cost of manually labeling large datasets for PFMs. In NLP, the language models can be trained by predicting masked characters, words, or sentences. Variational autoencoder (VAE) and generative adversarial network (GAN) are two types of generative SSL methods, which are to reconstruct the data itself. Besides, contrastive learning, as a type of discriminative SSL method, is widely applied in CV, NLP, and GL. The main idea of contrastive learning is to learn the prior knowledge distribution of the data itself with the aid of various methods such as data augmentation. In this way, contrastive learning can learn a model that makes similar instances closer in the projected space, and dissimilar instances farther apart in the projected space. Here we show a simple version of contrastive loss:

$$\mathcal{L}_c(\mathbf{x}_i, \mathbf{x}_j, \theta) = m \|f_{\theta}(\mathbf{x}_i) - f_{\theta}(\mathbf{x}_j)\|_2^2 + (1 - m) \max(0, \epsilon - \|f_{\theta}(\mathbf{x}_i) - f_{\theta}(\mathbf{x}_j)\|_2)^2 \quad (5)$$

where m is 1 if two samples have the same label, otherwise 0, and ϵ is the upper bound distance.

Reinforcement learning

RL is another type of learning paradigm that models the learning process as a sequential interaction between an agent and an environment, where an RL agent seeks to learn an optimal policy for sequential decision-making problems. Specifically, at each time interaction step t , the agent receives a state s_t in a state space $\mathcal{S}$, and selects an action a_t from an action space $\mathcal{A}$, following a policy $\pi_{\theta}(a_t | s_t) : \mathcal{A} \rightarrow \mathcal{S}$ parameterized by θ . Then the agent receives a scalar immediate reward $r_t = r(s_t, a_t)$ and the next state s_{t+1} according to the environment dynamics, where $r(s, a)$ is the reward function. For each episode, this process continues until the agent reaches a terminal state. After an episode is finished, the RL agent will restart to begin a new episode. The return for each state is discounted, accumulated reward with the discount factor $\gamma \in (0, 1]$, $R_t = R(s_t, a_t) = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$. The

agent aims to maximize the expectation of such long-term return from each state,

$$\max_{\theta} \mathbb{E}_{s_t} [R_t | s_t, a_t = \pi_{\theta}(s_t)]. \quad (6)$$

2.3 Pretraining tasks for PFMs

Pretraining is an initialization framework, which generally needs to be used in conjunction with fine-tuning downstream tasks. In the scheme of pretraining and finetuning, the parameters of the model are trained on pre-set tasks to capture specific attributes, structure, and community information. The pretrained features can assist downstream tasks, provide sufficient information, and speed up the convergence of the model.

2.3.1 Pretraining tasks for NLP

The pretraining tasks can be divided into five categories according to the learning methods: Mask Language Modeling (MLM), Denoising AutoEncoder (DAE), Replaced Token Detection (RTD), Next Sentence Prediction (NSP), Sentence Order Prediction (SOP). RTD, NSP, and SOP are contrastive learning methods, which assume that the observed samples are more semantically similar than the random samples.

Mask language modeling (MLM). MLM erases some words randomly in the input sequence and then predicts these erased words during pretraining. Typical examples include BERT [13] and SpanBERT [44].

Denoising autoencoder (DAE). DAE is used to add noise to the original corpus and reconstruct the original input using the corpus containing noise. BART [45] is a representative example.

Replaced token detection (RTD). RTD is a discriminant task that determines whether the LM has replaced the current token. This task is introduced in ELECTRA [46]. By training the model to distinguish whether a token has been replaced or not, the model can acquire language knowledge.

Next sentence prediction (NSP). In order to make the model understand the correlation between the two sentences and capture sentence-level representations, an NSP task is introduced. The PFM inputs two sentences from different documents and checks whether the order of the sentences is correct. A typical example is BERT.

Sentence order prediction (SOP). Different from NSP, SOP uses two contiguous fragments from a document as positive samples and the exchange order of the two fragments as negative samples. The PFMs can better model the correlation between sentences, such as ALBERT [47].

2.3.2 Pretraining tasks for CV

There are many pretraining tasks created for CV to learn the feature space, which is based on SSL. It utilizes pretext tasks that contain human-designed labels, like jigsaw puzzles or the comparison of various patches from images. This enables the generalization of learned representations to a range of downstream tasks.

Specific pretext task. A pretext task also referred to as a predefined task, is created for the encoder networks to perform during the pretraining phase. The network is trained by predicting the answer to a special pretext task. Based on particular features of the data, pseudo labels are generated for the fictitious task. Then, using guided learning techniques, the encoder networks are trained to solve the pretext task. For example, inpainting aims to pretrain models by predicting the missed center part.

Frame order learning task. Learning frame order from videos involves frame processing through time steps, which can serve as the pretraining task for CV. This issue usually relates to completing pretextual exercises that can aid in the acquisition of visual temporal representations.

Data generation task. The representational capabilities within the generative adversarial networks (GANs) can also be used in the pretraining tasks. Projecting data back into the latent space, as demonstrated by BiGANs [48], is helpful for auxiliary supervised discrimination tasks by acting as feature representations.

Data reconstruction task. Since the images can be divided into patches inspired by the natural language, some pretraining tasks for NLP can also be used in CV, e.g., the autoencoder-based masked prediction. The original image is first divided into a few patches and discrete visual tokens are used to encode each patch. The visual tokens from the masked patches are outputted in the second stage to match the corresponding visual tokens from the fixed tokenizer.

Miscellaneous. To train the PFMs in CV, additional pretraining tasks are suggested. For instance, based on contrastive learning, encoder networks are used for pretraining on various data augmentation. The parameters are trained by maximizing the distance between negative pairs (e.g., pairs with different labels) and minimizing the distance between positive pairs (e.g., pairs with the same labels). To pretrain the parameters of the backbone network, the DeepClustering [49] method divides the representations into various clusters and labels these clusters as supervised signals.

2.3.3 Pretraining tasks for GL

The pre-set tasks in GL are similar to other pretext tasks. However, they can be supervised or unsupervised depending on the design. According to the pretraining purpose and

potential motivation in GL, such tasks can be divided into the following categories:

Graph information completion. This task refers to firstly masking part of the information in the input graph, and then recovering the masked information based on the analysis of the remaining information distribution. Similar tasks also exist in CV and NLP, and their goals are to fill in hidden pixels or words, respectively.

Graph property prediction. Different from directly modeling the information of the input graph, this task aims to provide a variety of self-supervised signals by mining the potential properties of the input graph. Specifically, on the one hand, it considers node attributes, local substructure, and connectivity information to provide predictive regression tasks; on the other hand, it assigns pseudo-labels to nodes through information such as clusters, structure density, and attribute similarity to provide classification tasks.

Graph consistency analysis. The goal of this task is to maximize the consistency between samples with similar semantic information in the graph embedding and minimize the agreement between samples with unrelated semantic information. In the actual scenario, it can be divided into consistency analysis of context/self/cross-scale according to different model training strategies.

Miscellaneous. Compared with using only one pretext task, some methods have designed some integration mechanisms to incorporate the advantages of multiple pretext tasks into a unified framework. Besides, some graph data in specific fields have unique self-supervised signals with practical significance that can be used for pretraining under targeted design.

In summary, the transformer is an important component of the large model architecture, which helps learn the important features and mine intrinsic structure in data. Different learning mechanisms can be used for training PFMs according to the datasets and specific tasks. Especially, SSL is a promising mechanism for learning knowledge embeddings from the data considering the large scale of unlabeled data in various areas. RL provides a new way to fine-tune the PFMs for downstream tasks by optimizing a policy (model) against the reward model. Besides, we also explore the theory-based explanations on PFM in Appendix C. How to design effective and efficient tasks for PFMs to master the knowledge behind the data is an important research topic.

3 PFMs for natural language processing

NLP is a research field that integrates linguistics and computer science. Its main research tasks include part-of-speech tagging, named entity recognition, semantic role labeling, machine translation, question answering, sentiment analysis, text summarization, text classification, relationship

extraction, event extraction, etc. The idea of PFM first gained popularity in NLP. Then CV and GL adopt the promising pretraining technology. The PFM trains on a large benchmark dataset and is fine-tuned on the primary task dataset to obtain a model which can solve new similar tasks. It models syntactic and semantic representations of words simultaneously and changes the representation of polysemous words dynamically according to different input contexts. PFM learns a rich knowledge of grammar and semantic reasoning with better results. Numerous PFMs have been proposed in the past few years, as shown in Table 1.

In this section, we first introduce word representation learning models including the autoregressive language model (LM), contextual LM, and permuted LM. Then, we present the neural network architectures for the PFM designing method and the masking designing method. Besides, we summarize boosting methods for enhancing model performance, multi-task learning, and different downstream tasks. Finally, we introduce the instruction-aligning methods, e.g. RLHF and Chain-of-Thoughts, which are applied in PFMs, such as ChatGPT, to provide outputs that more closely match human preferences and are less harmful.

3.1 Word representations methods

Word representations play a significant role in downstream tasks, which is the basis of NLP. Traditional text learning is summarized in Appendix B.1. Many large-scale pretrained models have achieved better performance than humans in question answering, machine reading comprehension, and natural language reasoning, which indicates that the current construction approach of PFMs is practical. The existing pretraining LMs are mainly divided into three branches according to the word representations approach: (1) *autoregressive LM*, (2) *contextual LM*, and (3) *permuted LM*. The word prediction direction and contextual information are the most important factors among these three branches.

Autoregressive language model

The autoregressive LM predicts the next possible word based on the preceding word or the last possible word based on the succeeding word. It is selected as a feature extractor and text representations are extracted from the former words. Thus, it has better performance in NLG tasks such as text summarization and machine translation. For a sequence, $T = [w_1, w_2, \dots, w_N]$, the probability of a given word calculated as follows:

$$p(w_1, w_2, \dots, w_N) = \prod_{i=1}^N p(w_i | w_1, w_2, \dots, w_{i-1}), \quad (7)$$

where $i > 1$ and N is the length of the input sequence.

The GPT [50] adopts a two-stage method of self-supervised pretraining and supervised fine-tuning and uses stacked Transformer [38] as its decoder. As a follow-up, the OpenAI team continues to expand GPT, proposes the GPT-2 [51] and increases the number of stacked Transformer layers to 48 layers. The total number of parameters reached 1.5 billion. GPT-2 also introduces multi-task learning [52]. The GPT-2 has a considerable model capacity and can be adjusted for different task models rather than fine-tuning them. However, GPT-2 also uses an autoregressive LM. Therefore, it improves the performance of the model without increasing the cost dramatically. Due to the lack of contextual modeling ability with a one-way Transformer, the main performance improvement of GPT-2 comes from the combined effect of multi-task pretraining, super-large datasets, and super-large models. Task-based datasets for fine-tuning are still needed for specific downstream tasks. Increasing the training scale of the LM can lead to a significant enhancement in task-independent performance. Hence, GPT-3 [20] was developed, which features a model size of 175 billion parameters and is trained with 45 Terabytes of data. This enables it to exhibit good performance without the need for fine-tuning for specific downstream tasks.

Contextual language model

The autoregressive LM only uses the information above or below and cannot use the information above and below at the same time. ELMO [53] only uses bi-directional Long Short-Term Memory (LSTM), which is a concatenation of two unidirectional LSTMs backward and forward. The contextual LM predictions are based on contextual words. It uses a Transformer encoder, and the upper and lower layers of the model are all directly connected to each other due to the self-attention mechanism. For a sequence of words T , the probability of a given word is calculated as follows

$$p(w_1, w_2, \dots, w_N) = \prod_{i=1}^N p(w_i | w_1, w_2, \dots, w_N). \quad (8)$$

BERT [13] uses a stacked multi-layer bi-directional Transformer as the basic structure, and WordPiece [54] as a word segmentation method. The model input consists of three parts: word embedding, segment embedding, and position embedding. It uses a bi-directional Transformer as a feature extractor, which offsets the defect of ELMO and GPT. However, the shortcomings of BERT are also not to be ignored. The bidirectional Transformer structure does not eliminate the constraints of the self-encoding model. Its vast number of model parameters are very unfriendly to devices with low computing resources and are challenging to deploy and apply. Furthermore, the hidden language modeling in pretraining will lead to inconsistencies with the input of the model in the fine-tuning stage. Most PFMs need more

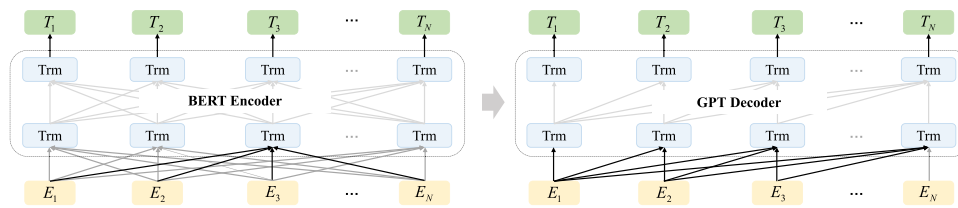


Fig. 3 The architectures of BART [45]: generalizing BERT (due to the bidirectional encoder), GPT (with the left-to-right decoder). An autoregressive decoder is used to determine the likelihood of the

original document after the corrupted document (on the left) has been encoded using a bidirectional model

training tasks and a larger corpus. Aiming at the problem of insufficient training, Liu et al. [55] propose the RoBERTa. It uses a larger batch size and unlabeled data. Furthermore, it trains the model for a longer time, removes the NSP task, and adds long sequence training. In processing text input, different from BERT, Byte Pair Encoding (BPE) [56] is adopted for word segmentation. BPE uses a different mask mode for each input sequence, even if the input sequence is the same.

Permuted language model

The modeling method with a contextual LM can be regarded as the autoencoding model. However, due to the inconsistency in the training stage and fine-tuning stage, the performance of the autoencoding model is poor in the Natural Language Generation (NLG) task. Permuted LM aims to combine the advantages of the autoregressive LM and the autoencoder LM. It improves the defects of the two models to a great extent and can be used as a basic idea for the construction of future pretraining target tasks. For a given input sequence $T = [w_1, w_2, \dots, w_N]$, the formal representation of the target function of the permuted LM is as follows

$$\max_{\theta} \mathbb{E}_{z \sim Z_N} \left[\sum_{t=1}^N \log p_{\theta}(x_{z_{T=t}} | x_{z_{T < t}}) \right], \quad (9)$$

where θ is the shared parameter in all permutations, Z_N represents the set of all possible permutations of the input sequence T , and $z_{T=t}$ and $z_{T < t}$ represents the t -th element and the $[1, 2, \dots, t-1]$ elements of a permutation $z \in Z_N$.

MLM represented by BERT can implement bi-directional coding well. However, MLM uses the mask marking during pretraining but not during fine-tuning, which resulted in inconsistent data during pretraining and fine-tuning. To achieve bi-directional coding and avoid the problems of MLM, the permuted LM is proposed. permuted LM is based on the autoregressive LM, which avoids the influence of inconsistent data. However, unlike traditional autoregressive models, permuted LM no longer models sequences in order. It gives all possible permutations of sequences to maximize the expected logarithmic likelihood of the sequence. In this

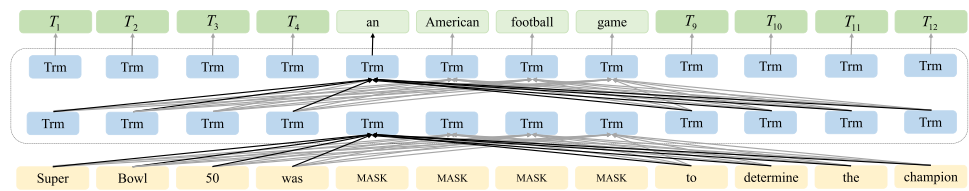
way, any position can take advantage of contextual information from all positions, making permuted LM implement bidirectional encoding. The most common permuted LM models are XLNET [14] and MPNet [57]. XLNET is a PFM based on a permuted language modeling approach, which incorporates two crucial techniques from Transformer-XL: relative positional encoding and the segment recurrence mechanism. In contrast, MPNet combines Masked Language Modeling (MLM) and permuted language modeling to predict token dependencies, using auxiliary position information as input to enable the model to view a complete sentence and reduce position differences. These two models represent significant advancements in the field of PFMs.

3.2 Model architecture designing methods

ELMO adopts a multi-layer RNN structure. Each layer is a bi-directional LSTM structure composed of a forward and backward LM. The maximum likelihood of these two directions is taken as the objective function. Compared with the word vector method, ELMO introduces contextual information and improves the polysemy problem, but ELMO's overall ability to extract linguistic features is weak.

The application research of PFMs has two main directions. One is PFMs with fine-tuning (e.g., BERT), and the other one is PFMs with zero/few-shot prompts (e.g., GPT). **BERT** uses a bi-directional encoder in Transformer to predict which words are masked and determine whether two sentences are contextual. However, the document is encoded bidirectionally and missing tokens are predicted independently, which reduces the generation ability [45]. **GPT** uses an autoregressive decoder as a feature extractor to predict the next word based on the first few words and solve downstream tasks using fine-tuning, so it is more suitable for text-generation tasks. However, GPT only uses the former words for prediction, which cannot learn bidirectional interaction information.

Different from these models, **BART** [45] is a noise-reducing autoencoder built by seq2seq model adopting the encoder-decoder structure, as shown in Fig. 3 from [45]. Pretraining mainly includes using noise to destroy text and

Fig. 4 The architecture of SpanBERT [44]

using the seq2seq model to rebuild the original text. The encoding layer adopts a bi-directional Transformer. It adopts five modes of adding noise: (1) single word mask; (2) word deletion; (3) span mask; (4) sentence rearrangement; (5) document rearrangement. In the encoder part, the sequence has been masked before inputting it into the encoder. Then, the decoder restores the original sequence according to the encoding representation output by the encoder and the sequence that has not been masked. The addition of a series of noise patterns makes the performance of BART in sequence generation and natural language reasoning tasks significantly improved.

3.3 Masking designing methods

The attention mechanism first aggregates essential words into sentence vectors, and vital sentence vectors into text vectors, which allows the model to pay different attention to different inputs [58]. For BERT, as a bidirectional encoding LM, any two words in an input sentence can see each other. However, it hinders the ability of the BERT model to learn NLG tasks.

Joshi et al. [44] propose SpanBERT based on RoBERTa, which adopts the idea of dynamic masking and single segment pretraining, as shown in Fig. 4 from [44]. The span mask and the Span Boundary Objective (SBO) are also proposed to mask words of a certain length. The target task of the span-boundary is to restore all the masked span (tokens) by the observed tokens at both ends. The training stage uses the dynamic mask strategy proposed in the RoBERTa, instead of the mask during the data preprocessing. Unlike BERT, SpanBERT randomly covers up a continuous text and adds the SBO training target. It predicts the span using the token closest to the span boundary and eliminates the NSP pretraining task.

The BERT and GPT can only separate the training encoder and decoder without joint training in the NLG task. Song et al. [59] propose the masked seq2seq pretraining model MASS. In the training stage, the input sequence of the encoder is randomly masked as a continuous segment of length k . The masked segment will be recovered through the MASS decoder. UniLM [60] completes the learning of the NLG model by designing a different mask for two sentences in the input data. For the first sentence, UniLM uses the same structure as the Transformer encoder making

each word notice its preceding and following words. For the second sentence, each word can only notice all the words in the first sentence and the preceding words in the current sentence. Thus, the first and second sentences of the model input form the classic seq2seq pattern.

3.4 Boosting methods

Boosting on model performance

Most of the popular pretraining models need lots of pretraining data, which imposes huge requirements on the hardware, making it challenging to retrain, and only fine-tuning can be done to the model. To solve these problems, some models appear. For example, ERNIE Tiny released by Baidu is a miniaturized ERNIE [61], that reduces the number of layers and increases the prediction speed by 4.3 times with a slight decrease in accuracy. Lan et al. propose the ALBERT [47] to reduce memory consumption and training speed. However, it is undeniable that no matter what kind of compression is done for these large-scale models, the performance of the models in these tasks will deteriorate sharply. It requires paying attention to the efficient representation of high-level semantic and grammatical information and lossless compression in future works. By using word-embedded parameter factorization and hidden parameter sharing between layers, ALBERT significantly reduces the number of parameters of the model without performance loss. It proposes the training task of SOP, which predicts the order of the two sentences to improve the performance.

Boosting for multi-task learning

ERNIE(Baidu) [61] is mainly composed of two parts, the Transformer encoder and task embedding. In the Transformer encoder, the self-attention mechanism is used to capture the context information of each token and generate context representation embedding. Task embedding is a technique that applies different characteristics to a task. ERNIE 2.0 [62] introduces multi-task learning to realize the pretraining of lexical, grammar, and semantics. ERNIE 2.0 uses seven different pretraining tasks, covering three aspects: word level, sentence level, and semantic level. It uses continual learning, making the knowledge in the previous training task retained and enabling the model to acquire long-distance memory. It uses a Transformer encoder and

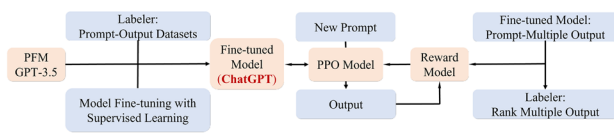


Fig. 5 Boosting GPT-3.5 to ChatGPT using Reinforcement Learning from Human Feedback

introduces task embedding, enabling the model to distinguish different tasks in the continual learning process. UniLM [60] uses three pretraining tasks: unidirectional LM, bidirectional LM, and encoder-decoder LM. It can simultaneously complete three kinds of target tasks in the pretraining stage through the self-attention layer mask mechanism. In the training stage, UniLM adopts the small-segment mask strategy proposed by SpanBERT, and the loss function is composed of the loss functions of the above three pretraining tasks. To maintain the contribution consistency on all loss functions, the three pretraining tasks are trained simultaneously. Modeling and parameter sharing of multiple tasks make LMs achieve good generalization ability in Natural Language Understanding (NLU) and NLG tasks.

Boosting for different downstream tasks

The pretraining models tend to be large-sized, so how to match different downstream tasks is equally important. Some pretraining models that are trained on specialized corpora have appeared [63–65]. Cui et al. [63] propose the BERT-whole word masking model (BERT-WWM). They directly use BERT in Chinese to be masked randomly according to the original MLM training, resulting in the loss of semantic information. Since there is no explicit language boundary in Chinese, it is easy to lose significant meaning. ZEN [64] is a text encoder based on BERT, which adopts N-gram to enhance performance and effectively integrates considerable granular text information with fast convergence speed and good performance. Tsai et al. [65] propose an oriented multilingual sequence labeling model for sequence labeling tasks. The knowledge distillation method is adopted to achieve better performance in the two tasks: part of speech labeling and morphological attribute prediction for multiple low-resource languages. The inference time is shortened by 27 times.

Examples: ChatGPT and Bard

As shown in Fig. 5, ChatGPT is fine-tuned based on the PFM GPT-3.5 using RLHF. ChatGPT uses a different data collection setup compared to InstructGPT. First, a large dataset with prompts and the desired output behaviors is collected. The dataset is used to fine-tune GPT-3.5 with supervised learning. Second, given the fine-tuned model and a prompt, the model will generate several model outputs. A labeler gives the desired score and ranks the output

to compose a comparison dataset, which is used to train the reward model. Finally, the fine-tuned model (ChatGPT) is optimized against the reward model using the Proximal Policy Optimization (PPO)[66] RL algorithm.

Another experimental conversational PFM, the Bard,³ is developed by Google. Bard is based on the LM for Dialogue Applications (LaMDA). LaMDA [67] is built upon the Transformer, which is pretrained on 1.56T words of dialog data and web text. Safety and factual grounding are two main challenges for conversational AI. LaMDA applies the approaches that fine-tuning with high-quality annotated data and external knowledge sources to improve model performance.

3.5 Instruction-aligning methods

Instruction-aligning methods aim to let the LM follow human intents and generate meaningful outputs. The general approach is fine-tuning the pretrained LM with high-quality corpus in a supervised manner. To further improve the usefulness and harmlessness of LMs, some works introduce RL into the fine-tuning procedure so that LMs could revise their responses according to human or AI feedback. Both supervised and RL approaches can leverage chain-of-thought [24] style reasoning to improve the human-judged performance and transparency of AI decision-making.

Supervised fine-tuning (SFT)

SFT is a well-established technique to unlock knowledge and apply it to specific real-world, even unseen tasks. The template for SFT is composed of input–output pairs and an instruction [113]. For example, given the instruction “Translate this sentence to Spanish:” and an input “The new office building was built in less than three months.”, we want the LM to generate the target “El nuevo edificio de oficinas se construyó en tres meses.”. The template is commonly humanmade including unnatural instructions [114] and natural instructions [115, 116], or bootstrap based on a seed corpus [117]. Ethical and social risks of harm from LMs are significant concerns in SFT [118]. LaMDA, the largest LM to date, thus relies on crowdworker annotated data for providing a safety assessment of any generated LaMDA response in three conversation categories: natural, sensitive, and adversarial. The list of rules serves further safety fine-tuning and evaluation purposes.

Reinforcement learning from feedback

RL has been applied to enhance various models in NLP tasks such as machine translation [119], summarization [18], dialogue generation [120], image captioning [121], question generation [122], text-games [123], and more

³ <https://blog.google/technology/ai/bard-google-ai-search-updates/>.

Table 1 Summary of PFM in NLP

Year	Conference	Model	Architecture	Embedding	Training method	Code
2013	NeurIPS	Skip-Gram [68]	Word2Vec	Probabilistic	–	https://github.com/.../models
2014	EMNLP	GloVe [69]	Word2Vec	Probabilistic	–	–
2015	NeurIPS	LM-LSTM [70]	LSTM	Probabilistic	LM	https://github.com/.../GloVe
2016	IJCAI	Shared LSTM [71]	LSTM	Probabilistic	LM	https://github.com/.../adversarial_text
2017	TACL	FastText [72]	Word2Vec	Probabilistic	–	https://github.com/.../fastText
2017	NeurIPS	CoVe [73]	LSTM+Seq2Seq	Probabilistic	–	https://github.com/.../cove
2018	NAACL-HLT	ELMO [53]	LSTM	Contextual	LM	https://allennlp.org/elmo
2018	NAACL-HLT	BERT [13]	Transformer Encoder	Contextual	MLM	https://github.com/.../bert
2018		OpenAI GPT [50]	Transformer Decoder	Autoregressive	LM	https://github.com/.../transformer-lm
2019	ACL	ERNIE(THU)	Transformer Encoder	Contextual	MLM	https://github.com/.../ERNIE
2019	ACL	Transformer-XL [74]	Transformer-XL	Contextual	–	https://github.com/.../transformer-xl
2019	ICLR	InfoWord [75]	Transformer Encoder	Contextual	MLM	–
2019	ICLR	StructBERT [76]	Transformer Encoder	Contextual	MLM	–
2019	ICLR	ALBERT [47]	Transformer Encoder	Contextual	MLM	https://github.com/.../ALBERT
2019	ICLR	WKLM [77]	Transformer Encoder	Contextual	MLM	–
2019	ICML	MASS [59]	Transformer	Contextual	MLM(Seq2Seq)	https://github.com/.../MASS
2019	EMNLP-IJCNLP	KnowBERT [78]	Transformer Encoder	Contextual	MLM	https://github.com/.../kb
2019	EMNLP-IJCNLP	Unicoder [79]	Transformer Encoder	Contextual	MLM+TLM	–
2019	EMNLP-IJCNLP	MultiFit [80]	QRNN	Probabilistic	LM	https://github.com/.../multifit
2019	EMNLP-IJCNLP	SciBERT [81]	Transformer Encoder	Contextual	MLM	https://github.com/.../scibert
2019	EMNLP-IJCNLP	BERT-PKD [82]	Transformer Encoder	Contextual	MLM	https://github.com/.../Compression
2019	NeurIPS	Xlnet [14]	Transformer-XL Encoder	Permutation	PLM	https://github.com/.../xlnet
2019	NeurIPS	UNILM [60]	LSTM + Transformer	Contextual	LM + MLM	https://github.com/.../unilm
2019	NeurIPS	XLM [83]	Transformer Encoder	Contextual	MLM+CLM+TLM	https://github.com/.../XLM
2019	OpenAI Blog	GPT-2 [51]	Transformer Decoder	Autoregressive	LM	https://github.com/.../gpt-2
2019	arXiv	RoBERTa [55]	Transformer Encoder	Contextual	MLM	https://github.com/.../fairseq
2019	arXiv	ERNIE(Baidu) [61]	Transformer Encoder	Contextual	MLM+DLM	https://github.com/.../ERNIE
2019	EMC2@NeurIPS	Q8BERT [84]	Transformer Encoder	Contextual	MLM	https://github.com/.../quantized_bert.py
2019	arXiv	DistilBERT [85]	Transformer Encoder	Contextual	MLM	https://github.com/.../distillation
2020	ACL	fastBERT [86]	Transformer Encoder	Contextual	MLM	https://github.com/.../FastBERT

Table 1 (continued)

Year	Conference	Model	Architecture	Embedding	Training method	Code
2020	ACL	SpanBERT [44]	Transformer Encoder	Contextual	MLM	https://github.com/.../SpanBERT
2020	ACL	BART [45]	Transformer	En: Contextual De: Autoregressive	DAE	https://github.com/.../transformers
2020	ACL	CamemBERT [87]	Transformer Encoder	Contextual	MLM(WWM)	https://camembert-model.fr
2020	ACL	XML-R [88]	Transformer Encoder	Contextual	MLM	https://github.com/.../XML
2020	ICLR	Reformer [89]	Reformer	Permutation	–	https://github.com/.../reformer
2020	ICLR	ELECTRA [46]	Transformer Encoder	Contextual	MLM	https://github.com/.../electra
2020	AAAI	Q-BERT [90]	Transformer Encoder	Contextual	MLM	–
2020	AAAI	XNLG [91]	Transformer	Contextual	MLM+DAE	https://github.com/.../xnlg
2020	AAAI	K-BERT [92]	Transformer Encoder	Contextual	MLM	https://github.com/.../K-BERT
2020	AAAI	ERNIE 2.0 [62]	Transformer Encoder	Contextual	MLM	https://github.com/.../ERNIE
2020	NeurIPS	GPT-3 [20]	Transformer Decoder	Autoregressive	LM	https://github.com/.../gpt-3
2020	NeurIPS	MPNet [57]	Transformer Encoder	Permutation	MLM+PLM	https://github.com/.../MPNet
2020	NeurIPS	ConvBERT [93]	Mixed Attention	Contextual	–	https://github.com/.../ConvBert
2020	NeurIPS	MiniLM [94]	Transformer Encoder	Contextual	MLM	https://github.com/.../minilm
2020	TACL	mBART [95]	Transformer	Contextual	DAE	https://github.com/.../mbart
2020	COLING	CoLAKE [96]	Transformer Encoder	Contextual	MLM+KE	https://github.com/.../CoLAKE
2020	LREC	FlauBERT [97]	Transformer Encoder	Contextual	MLM	https://github.com/.../Flaubert
2020	EMNLP	GLM [98]	Transformer Encoder	Contextual	MLM+KG	https://github.com/.../GLM
2020	EMNLP (Findings)	TinyBERT [99]	Transformer	Contextual	MLM	https://github.com/.../TinyBERT
2020	EMNLP (Findings)	RobBERT [100]	Transformer Encoder	Contextual	MLM	https://github.com/.../RobBERT
2020	EMNLP (Findings)	ZEN [64]	Transformer Encoder	Contextual	MLM	https://github.com/.../ZEN
2020	EMNLP (Findings)	BERT-MK [101]	KG-Transformer Encoder	Contextual	MLM	–
2020	RepL4NLP@ACL	Compressing-BERT [35]	Transformer Encoder	Contextual	MLM(Pruning)	https://github.com/.../bert-prune
2020	JMLR	T5 [102]	Transformer	Contextual	MLM(Seq2Seq)	https://github.com/.../transformer
2021	T-ASL	BERT-wwm-Chinese [63]	Transformer Encoder	Contextual	MLM	https://github.com/.../BERT-wwm
2021	EACL	PET [103]	Transformer Encoder	Contextual	MLM	https://github.com/.../pet
2021	TACL	KEPLER [104]	Transformer Encoder	Contextual	MLM+KE	https://github.com/.../KEPLER

Table 1 (continued)

Year	Conference	Model	Architecture	Embedding	Training method	Code
2021	EMNLP	SimCSE [105]	Transformer Encoder	Contextual	MLM+KE	https://github.com/.../SimCSE
2021	ICML	GLaM [106]	Transformer	Autoregressive	LM	–
2021	arXiv	XLME [107]	Transformer	Contextual	MLM	–
2021	arXiv	T0 [108]	Transformer	Contextual	MLM	https://github.com/.../T0
2021	arXiv	Gopher [109]	Transformer	Autoregressive	LM	–
2022	arXiv	MT-NLG [110]	Transformer	Contextual	MLM	–
2022	arXiv	LaMDA [67]	Transformer Decoder	Autoregressive	LM	https://github.com/.../LaMDA
2022	arXiv	Chinchilla [111]	Transformer	Autoregressive	LM	–
2022	arXiv	PaLM [43]	Transformer	Autoregressive	LM	https://github.com/.../PaLM
2022	arXiv	OPT [112]	Transformer Decoder	Autoregressive	LM	https://github.com/.../MetaSeq

The pretraining task includes language model (LM), masked LM (MLM), permuted LM (PLM), denoising autoencoder (DAE), knowledge graphs (KG), and knowledge embedding (KE)

[124–126]. RL is a helpful method for optimizing non-differentiable objectives in language generation tasks by treating them as sequential decision-making problems. However, there is a risk of overfitting to metrics that use neural networks, leading to nonsensical samples that score well on the metrics [127]. RL is also used to align LMs with human preferences [128–130].

InstructGPT proposes to fine-tune large models with PPO against a trained reward model to align LMs with human preference [19], which is the same method applied by ChatGPT named RLHF. Specifically, the reward model is trained with comparison data of human labelers' manual rankings of outputs. For each of them, the reward model or machine labeler calculates a reward, which is used to update the LM using PPO. More details are illustrated in Fig. 5.

One of the recent breakthroughs in PFM technology is GPT-4 [25], which follows a pretraining approach to predict the subsequent token in a document and then undergoes RLHF fine-tuning. As the task complexity increases, GPT-4 outperforms GPT-3.5 in terms of reliability, creativity, and capability to handle more nuanced instructions.

Sparrow [130], developed by DeepMind, also utilizes RLHF that reduces the risk of unsafe and inappropriate answers. Despite some promising results using RLHF by incorporating fluency, progress in this field is impeded by a lack of publicly available benchmarks and implementation resources, resulting in a perception that RL is a difficult approach for NLP. Therefore, an open-source library named RL4LMs [127] is introduced recently, which consists of building blocks for fine-tuning and evaluating RL algorithms on LM-based generation.

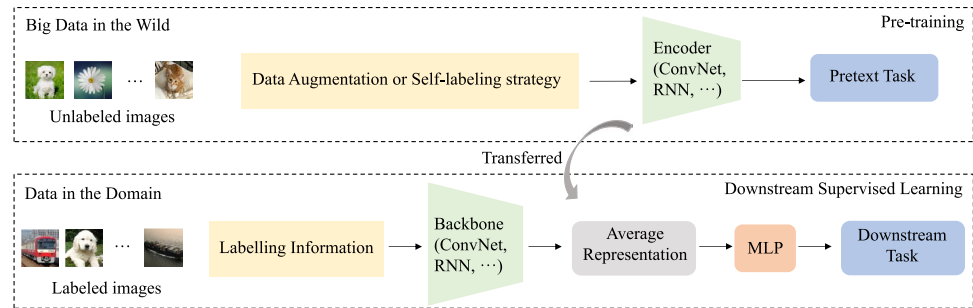
Besides human feedback, one of the latest dialogue agents—Claude favors Constitutional AI [131] where the reward model is learned via RL from AI Feedback (RLAIF). Both the critiques and the AI feedback are steered by a small set of principles drawn from a ‘constitution’, the specification of a short list of principles or instructions, which is the only thing provided by humans in Claude. The AI feedback focuses on controlling the outputs to be less harmful by explaining its objections to dangerous queries.

Chain-of-thoughts

Chain-of-thought (CoT) prompting is a technique for improving the reasoning ability of LLMs by prompting them to generate a series of intermediate steps that lead to the final answer of a multi-step problem. The CoT is a series of intermediate reasoning steps, which can significantly improve the ability of LLMs to perform complex reasoning [24, 132, 133]. Besides, fine-tuning with CoT shows slightly more harmless compared to without CoT [131]. CoT prompting is an emergent property of model scale, meaning it works better with larger and more powerful language models. It is also possible to fine-tune models on CoT reasoning datasets to enhance this capability further and stimulate better interpretability.

In a CoT prompting experiment, a prompt is provided to the model that outlines a multi-step problem. The prompt might pose a question such as “After selling 30 out of his 100 chickens and 10 out of his 20 pigs, how many animals does a farmer have left?” The model then generates a sequence of intermediate reasoning steps, for example, “The farmer has $100 - 30 = 70$ chickens remaining” and “The farmer has $20 - 10 = 10$ pigs remaining,” before generating the final answer, such as “The farmer has $70 + 10 = 80$

Fig. 6 The general pipeline for SSL. The top part represents the pretraining, and the bottom stream obtains transferred parameters from above to learn downstream supervised tasks



animals remaining.” CoT prompting has demonstrated its efficacy in improving the performance of LLMs on various reasoning tasks, such as arithmetic, symbolic reasoning, and common sense. It is a promising technique that can enhance the ability of language models to reason about complicated problems.

3.6 Summary

The neural probabilistic LM uses a neural network to estimate the parameters of the probabilistic LM, which reduces the size of the model parameters while enlarging the number of context windows. With the help of a neural network, the LM does not need to improve the smoothing algorithm to alleviate the performance bottleneck continuously. Since the training target is unsupervised, a corpus with a large amount of data is enough for training. The negative sampling technique in the training process provides a new idea for the follow-up study of the target task in the LM. Furthermore, the neural probabilistic LM promotes the further development of downstream task research because of its good representation capability and training efficiency. After the pretraining LM, especially the BERT model, is proposed, the research in language modeling has entered a new phase. The bidirectional LM, the hidden LM, and the sorted LM adopted by the bidirectional LM have successfully modeled the grammatical and semantic information in natural language at a deeper level. ChatGPT is another milestone work in PFMs using RL. The presentation ability of PFMs is qualitatively better than that of the neural probabilistic LM. It even exceeds that of humans in some tasks [37].

4 PFMs for computer vision

With the popularity of PFM used in NLP, it motivates researchers to start exploring PFM in CV. In the deep learning era of image learning, several types of neural networks have emerged, starting with the widely recognized Convolutional Neural Networks (CNNs) and followed by more advanced models such as Attention-based and

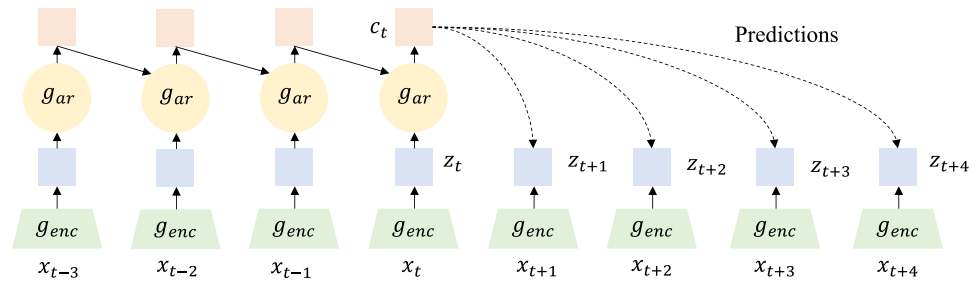
Transformer-based networks. We introduce the milestone networks in CV chronologically in Appendix B.2.

The term “pretraining” has not been clearly defined within the realm of deep learning research in CV. This word is first used in convolution-based networks when we adjust the parameters on a more general dataset such as ImageNet, which can make other tasks train to start with a warm-up initialization and thus converge with faster speed. In contrast to early CNN-based transfer learning techniques that rely on pretrained datasets with supervised signals, our examination of PFM centers on SSL which utilizes human-designed labels, such as Jigsaw puzzles, or the comparison of different patches from images as pretext tasks. Five categories of pretext tasks for SSL are summarized in Appendix D. This allows learned representations to be generalized to various downstream tasks, including classification, detection, recognition, segmentation, etc.

However, it is costly to rely on data annotations when the learning tasks become more complicated, making the labeling process more arduous and time-consuming than the actual learning. This is where SSL is urgently needed and how it can further fuel the progress of deep learning methods. To reduce the dependency on data labeling, unlabeled data are trained with self-supervision by matching, contrasting, or generating in SSL.

The general pipeline of SSL is shown in Fig. 6. During the pretraining stage, a pretext task is designed for the encoder networks to solve. The artificial labels for this pretext task are automatically generated based on specific attributes of the data, such as image patches from the same origin being labeled as “positive” and those from different origins as “negative”. Then, the encoder networks are trained to solve the pretext task by supervised learning methods. Since shallow layers extract fine-grained details such as edges, angles, and textures, while deeper layers capture task-related high-level features such as semantic information or image contents, learned encoders on pretext tasks can be transferred to downstream supervised tasks. During this stage, the parameters of the backbone are fixed, and only a simple classifier, such as a two-layer Multi-Layer Perceptron (MLP), needs to be learned. Considering the limited workload in the downstream training stage, this learning

Fig. 7 Contrastive predictive coding [145]. The input sequence can represent both images and videos



process is commonly referred to as fine-tuning. In summary, the representations learned during the pretraining stage in SSL can be reused on other downstream tasks and achieve comparable results.

In this section, we introduce different tasks for pretraining PFMs in CV. The PFMs can be trained by specific pretext tasks, frame order, generation, reconstruction, memory bank, sharing, clustering and so on. We summarize the PFMs proposed in CV in Table 2.

4.1 Learning by specific pretext task

In the early stage of unsupervised learning, the network is trained by designing a special pretext task and predicting the answer to this task. Dosovitskiy et al. [134, 135] pretrain the Exemplar CNN to discriminate the different patches from the unlabelled data. The experiments prove the designs can learn useful representations transferred to the standard recognition assignments. In the method based on context prediction [136], a handcrafted supervised signal about the position information serves as the label for the pair classification. Inpainting [137] aims to pretrain models by predicting the missed center part. Because inpainting is a semantic-based prediction, another decoder is linked to the context encoder in this manner. Furthermore, the standard pixel-by-pixel reconstruction process of the decoder can be transferred to any other downstream inpainting tasks. Specifically, Colorization [138] is a method that evaluates how colorization as a pretext task can help to learn semantic representation for downstream tasks. It is also known as the *cross-channel encoding* since different image channels serve as input and the output is discriminated. Similarly, Split-Brain Autoencoder [139] also learns representations in a self-supervised way by forcing the network to solve cross-channel prediction tasks. Jigsaw [140] is proposed to pretrain the designed Context-Free Network (CFN) in a self-supervised manner by first designing the Jigsaw puzzle as a pretext task. Completing Damaged Jigsaw Puzzles (CDJP) [141] learns image representation by complicating pretext tasks furthermore, in which puzzles miss one piece and the other pieces contain incomplete color. Following the idea of designing efficient and effective pretext tasks, Noroozi

et al. [142] use counting visual primitives as a special pretext task and outperform previous SOTA models on regular benchmarks. NAT [143] learns representation by aligning the output of backbone CNN to low-dimensional noise. RotNet [144] is designed to predict different rotations of images.

4.2 Learning by frame order

The learning of sequence data such as videos always involves frame processing through time steps. This problem often connects with solving pretext tasks that can help to learn visual temporal representations. Contrastive Predictive Coding (CPC) [145] is the first model to learn data representations by predicting the future in latent space. This model can be fed with data in any modalities, like speech, images, text, etc. The components of CPC are shown in Fig. 7 from [145], where the x_t represents the input sequence of observations, z_t is a sequence of latent representations after the encoder g_{enc} , and c_t is a context latent representation that summarizes all the latent sequence $z_{\leq t}$ after an autoregressive model g_{ar} . Unlike the traditional model predicts future frames x_{t+k} by a generative model $p_k(x_{t+k}|c_t)$, CPC models a "density ratio" f_k to represent the mutual information between the context latent representation c_t and future frame x_{t+k} :

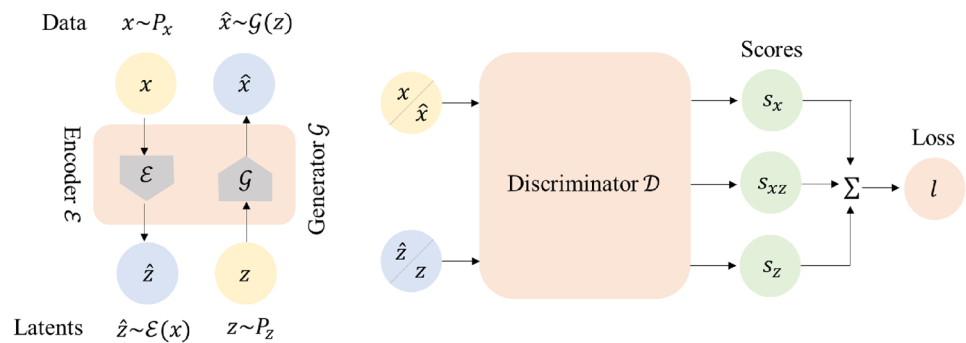
$$f_k(x_{t+k}, c_t) \propto p(x_{t+k}|c_t)/x_{t+k}. \quad (10)$$

After the encoding of recurrent neural networks, z_t and c_t can both be chosen for the downstream tasks as needed. The encoder and autoregressive model are trained by InfoNCE [145] as follows

$$\mathcal{L} = -\mathbb{E}_X[\log f_k(x_{t+k}, c_t) / \sum_{x_j \in X} f_k(x_j, c_t)], \quad (11)$$

where X denotes the training dataset containing both positive and negative samples. The density ratio f_k can be estimated by optimizing $\mathcal{L}$. CPC v2 revisits and improves CPC [146] by pretraining on unsupervised representations, and its representation generality can be transferred to data-efficient downstream tasks.

Fig. 8 The structure of the Big-BiGAN framework [147]



4.3 Learning by generation

Although many existing applications are popular after the development of the GAN-based approach, the representation abilities inside the GANs are not entirely exploited due to the absence of a feature encoder. Thus, Bidirectional Generative Adversarial Networks (BiGANs) [48] is proposed to project data back into the latent space, which is useful for auxiliary supervised discrimination tasks via serving as feature representations.

Based on BiGANs, BigBiGAN [147] first achieves the SOTA in unsupervised representation learning on ImageNet by adding an encoder and modifying the discriminator. As shown in Fig. 8 from [147], the traditional components of GANs (encoder $\mathcal{E}$ and generator $\mathcal{G}$) are used to produce data-latent pairs, denoted as $(\mathbf{x} \sim P_x, \hat{\mathbf{z}} \sim \mathcal{E}(\mathbf{x}))$ and $(\hat{\mathbf{x}} \sim \mathcal{G}(\mathbf{z}), \mathbf{z} \sim P_z)$. The final loss ℓ is defined as the sum of data-specific term s_x, s_z and data-joint term s_{xz} . The introduced discriminator $\mathcal{D}$ (Adversarially Learned Inference (ALI) [148], or BiGAN [48]) learns to discriminate between pairs from the raw data, latent distribution and encoded vector.

4.4 Learning by reconstruction

The iGPT [149] and ViT [40] models have demonstrated the feasibility of adapting the pretext task of masked prediction using auto-encoder from language to image data. BEiT [150] is the first to demonstrate that autoencoder-based masked prediction can outperform DINO [151], a conventional SOTA method without pretraining techniques. Specifically, BEiT consists of two stages: token embedding with discrete variational autoencoder (dVAE) [152], and tokenizer training with masked image prediction. In the first stage, the original image is split into some patches and encoded using discrete tokens, which is different from BERT since image patches don't have off-the-shelf tokens as words in NLP. In the second stage, the BEiT encoder takes a corrupted image containing unmasked and masked patches, and then the visual tokens of the masked patches are outputted to match the corresponding visual tokens from the fixed tokenizer.

Despite its success, the separation between masked prediction and autoencoder training induces that the whole framework is not end-to-end and hinders learning effectiveness and efficiency.

To mitigate this issue, MAE [153] proposes an end-to-end simple solution by predicting the masked patches directly from the unmasked ones with the Mean Squared Error (MSE) loss. It's worth noting that MAE uses a masking ratio of 75%, which is significantly higher than that of BERT (typically 15%). Ablation study suggests that higher masking ratios are beneficial for both fine-tuning and linear probing. Concurrently, SimMIM [154] proposes a similar autoencoder-based solution as MAE, in which they also confirm that a higher marking ratio and leveraging random masking strategy helps improve performance. The major difference is how they partition the responsibility of representation encoding and pretext prediction in the autoencoder. Since the decoder of SimMIM is simple, the encoder of SimMIM synchronously conducts both of them. On the contrary, the encoder in MAE solely undertakes the role of representation encoding, and the decoder is responsible for pretext prediction. Recently, Meta AI announces the Segment Anything Model (SAM) [155] which prompts users to specify what to segment in an image, allowing for a wide range of segmentation tasks without the need for additional training. SAM employs an MAE pretrained ViT-H [40] image encoder that runs once per image and produces an image embedding, as well as a prompt encoder that embeds input prompts such as clicks or boxes. Following that, a lightweight transformer-based mask decoder predicts object masks from image and prompt embeddings. The results show that SAM can generate high-quality masks from a single foreground point that are typically just modestly inferior to the manually annotated ground truth. It routinely achieves strong quantitative and qualitative outcomes on a wide range of downstream tasks using a zero-shot transfer approach and prompt engineering.

Leveraging ViT in MAE poses a serious inefficiency issue, where decreasing the patch size results in a quadratic increase in computing resources. To address the problem, there are two important solutions: (1) hierarchical ViT and (2) local attention. In the first direction, hierarchical ViT

Fig. 9 The general pipeline for the Memory Bank Method [156]

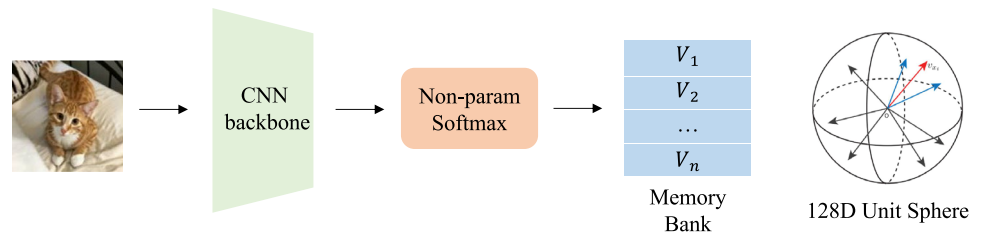
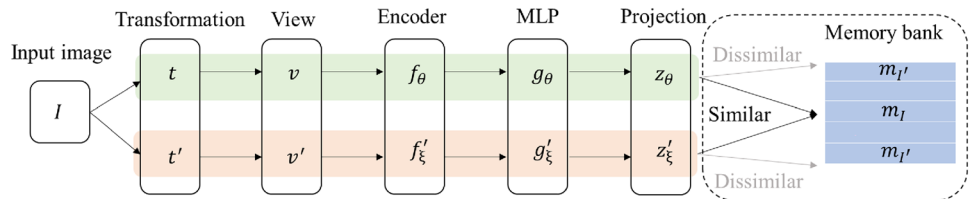


Fig. 10 Summary of all two-stream models, including contrastive learning and memory-bank-based methods



(hViT) was introduced, which utilizes a shrinking pyramid structure and techniques like shifted windows [157] to reduce computational demands. Unfortunately, hViT cannot be directly applied to enable MAE pretraining because the local window attention used in hViT makes it difficult to handle randomly masked patches as in MAE. Recently, Uniform Masking MAE (UM-MAE) [158] is proposed to empower MAE with hViTs, which introduces a two-stage pipeline: sampling and masking. It starts by randomly sampling a portion of patches (25% reported in the paper) from each block, and then follows by masking additional patches on top of the sampled ones. The first step helps to maintain common elements across different local windows, while the second step prevents shortcuts for pixel reconstruction from nearby low-level features, making the task more difficult. Another direction to improve efficiency focuses on reducing the input size by putting the attention of the network into some local small windows of the image. Motivated by the observation that local knowledge is sufficient for reconstructing masked patches, Local masked reconstruction (LoMaR) [159] was proposed. Rather than using the entire image for mask reconstruction, LoMaR samples a number of small windows and focuses attention on local regions, which outperforms MAE on downstream tasks in terms of learning efficiency.

4.5 Learning by memory bank

Non-Parametric Instance Discrimination (NPID) [156] is the first method that utilizes the instances to learn representations for downstream tasks. The detailed pipeline is shown in Fig. 9. The feature representations are stored in the memory bank for the convenience of computation because the instance-level classification objective needs all images in the training dataset. For any image x with feature

representation $\mathbf{v} = f_\theta(x)$, its probability of being recognized as i th example is:

$$P(i|\mathbf{v}) = \exp(\mathbf{v}_i^T \mathbf{v} / \tau) / \sum_{j=1}^n \exp(\mathbf{v}_j^T \mathbf{v} / \tau), \quad (12)$$

where $\mathbf{v}_i$ or $\mathbf{v}_j$ is the representation of i th or j th sample, which serves as a substitute for the parametric class prototype (i.e., weights of a classifier). Additionally, τ is the temperature parameter borrowed from the knowledge distillation [160].

Local Aggregation (LA) [161] is another method that trains a CNN encoder to embed raw images into a lower dimension space—embedding space. When a metric of local aggregation is maximized, similar data instances move together in the embedding space while dissimilar instances move apart.

Based on NPID, Pretext Invariant Representation Learning (PIRL, pronounced as “pearl”) [162] is proposed to argue that semantic representations are invariant under pretext transformation tasks. Suppose the original view and transformed view of images are denoted as I and I' , respectively. These sample views are fed into a CNN encoder, and the total empirical loss on the training dataset $\mathcal{D}$ can be defined as:

$$\mathcal{L}_{total}(\theta; \mathcal{D}) = \mathbb{E}_{I \sim \mathcal{T}} \left[\frac{1}{|\mathcal{D}|} \sum_{I' \in \mathcal{D}} \mathcal{L}(V_I, V_{I'}) \right], \quad (13)$$

where $\mathcal{T}$ denotes the different transformations of images. The loss encourages the representation of image I to be similar to that of I' , and the representation of I' to be dissimilar to that of different images I' , as shown in the dotted box of Fig. 10. Therefore, more negative sample pairs contribute to improving the scalability of the gradient and lead to the final learned encoder with a stronger representation ability. That is the reason why the memory bank is introduced to store more previous representations for subsequent comparison.

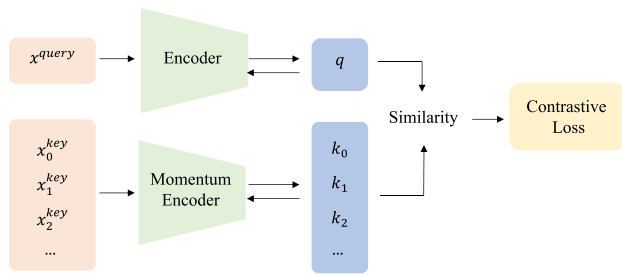


Fig. 11 The general pipeline of MoCo [163], which is also a two-stream framework with different parameters

4.6 Learning by sharing

SSL prefers using two encoder networks for the different data augmentation, and then pretrains the parameters by maximizing the distance between negative pairs or minimizing the distance between positive pairs. Figure 10 shows the two-stream models for all contrastive learning frameworks. The transformation t on the original input image I generates the view v , similarly, its counterpart t' generates v' . In general, two different or same encoders f_θ and f'_ξ are used to extract contrastive representations. The subsequent MLP heads g_θ and g'_ξ are used to learn more combinations that are beneficial to the contrastive loss. It is noticed that MLP and memory bank could be removed or preserved under different settings. In terms of the shared encoder, SSL can be divided into two categories: (1) Soft Sharing that two encoders share with similar but different parameters ($f_\theta \neq f'_\xi$); (2) Hard Sharing that two encoders maintain the same architectures and parameters ($f_\theta = f'_\xi$).

Soft sharing

Facebook AI Research (FAIR) presents Momentum Contrast (MoCo) [163] by using momentum to control the slight difference between two encoders. As shown in Fig. 11, one of the encoders serves as a dictionary look-up task that generates a queue of encoded data samples $\{k_0, k_1, \dots\}$. Another encoder generates encoded query $\{q_0, q_1, \dots\}$ with the training batch updated. The similarity is measured by the dot product of the new coming encoded query q and the encoded keys stored in the dictionary queue. Suppose there are K keys stored in the queue before the new key comes. The K keys are treated as negative samples to the query of the new key. To combine the contrastive loss on both negative and positive samples, InfoNCE Loss [145] is used for the pretraining in MoCo. The key design in MoCo for soft parameter sharing is called momentum update. He et al. [163] suggest that the direct parameter change of key encoder (i.e., momentum encoder) to query encoder loses the necessary consistency

and yields poor results. The momentum encoder parameter θ_k is updated as:

$$\theta_k = m\theta_k + (1 - m)\theta_q, \quad (14)$$

where the query encoder parameter θ_q is learned directly from the gradients of new coming instance, and $m \in [0, 1]$ is a hyper-parameter that controls the consistency (θ_k is more consistent if m is closer to 1).

Inspired by the design of SimCLR [164], in MoCo v2 [164], the FAIR team introduces an MLP projection head after encoders and utilizes more data augmentation techniques to improve the performance. The further improvements are from that: (1) embedded linear classifier bridges the gap between unsupervised and supervised pre-training representations; (2) more contrastive samples are feasible from both the larger training batch and stronger data augmentation.

DeepMind proposed Bootstrap Your Own Latent (BYOL) [165] that contains representation, projection, and discrimination stages to achieve a new SOTA without using negative samples. They understand the discrimination between different views of raw images as necessary prevention from collapse during the pretraining. However, they argue that many negative samples are not indispensable to prevent this collapse. As shown in the left part of Fig. 10, there are two streams in BYOL with different parameters. The online network (top green) updates parameters by comparing the prediction generated itself and the regression target provided by the target network. Then the parameters of the target model (bottom red) are updated the same as Eq. (14), i.e., $\xi \leftarrow \tau\xi + (1 - \tau)\theta$, where τ is the target decay rate to control the degree of parameter changing in the target network. Therefore, the target network can also be understood as a momentum encoder. Here, ξ in the target model is the parameter θ_k in momentum encoder, and θ in the online network denotes the parameter θ_q in the query encoder.

Hard sharing

SimCLR [166] is proposed by Brain Team in Google Research which utilizes the hard parameter-sharing architecture. This simple framework can also be concluded in Fig. 10, in which we can see that representations of different views of the same image are learned in the network $f(\cdot)$. This base encoder shares the parameters with each other. Thus, memory bank and momentum setting to learn key and query encoders are not necessary, which contributes to a simpler backbone architecture and easier learning strategy. The loss function to maximize the similarity between different views of the same image (positive pairs) is defined as

$$\ell_{i,j} = -\log \exp(\text{sim}(z_i, z_j)/\tau) / \sum_{k=1}^{2N} \mathbb{I}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau), \quad (15)$$

where (i, j) is a pair of positive samples, τ is an introduced hyper-parameter called temperature parameter [156], and $\mathbb{I}_{[k \neq i]} \in \{0, 1\}$ is an indicator function to control the denominator containing only negative pairs.

To avoid the dependence on a large number of explicit pairwise feature comparisons, Swapping Assignments between multiple Views of the same image (SwAV) [167] is proposed as an online algorithm by Inria and FAIR. SwAV introduces clustering to substitute the previous comparison between pairs, which gains more memory with the help of non-queue architecture. In this method, the clustering prototype joins the computation of the defined loss function. This prototype is encoded as the concatenation of vectors learned through the backpropagation in CNNs. Thus, there is no need for SwAV to compare the encoded representations between different views.

Based on the existing SwAV, a novel model called Self-supervised (SEER) [168] aims to learn a pretrained encoder from any random image and unbounded dataset in the wild. The base network is RegNetY architectures [169] trained with the SwAV SSL method [167]. This method proves that the SSL is not specific to a curated dataset such as ImageNet, and the scalability of recent RegNet releases the limitation of traditional backbones such as ResNet. In addition, this method encourages the research community to explore more backbones suitable for universal SSL.

Attracting the attention in the recent SSL, FAIR conducts empirical experiments on the SSL by utilizing the structure of Simple Siamese (SimSiam) networks. This method [170] can avoid the design of negative sample pairs, large batches (or memory banks), and momentum encoders in traditional contrastive learning. The two encoders in Fig. 10 with identical parameters that process two different views t and t' of image x are substituted by the only siamese network. MLP predictor g is used for one of the view representations, and then the stop-gradient operation is applied to another view representation.

4.7 Learning by clustering

DeepCluster [49] is the first model that adopts the clustering algorithm for large-scale dataset learning. This method groups the representations into different clusters and labels these clusters as supervised signals to pretrain the parameters of the backbone network. It demonstrates SOTA performance on a wide range of standard transferred tasks used in unsupervised learning.

When it comes to the connection between contrastive learning and clustering, SwAV [167] has utilized prototypes

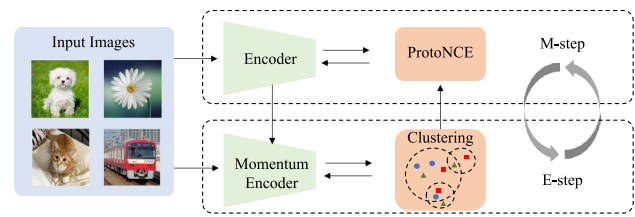


Fig. 12 The key pipeline for the DeepCluster model [49]

that serve as a clustering center to help classify the sample pairs during pretraining, while Prototypical Contrastive Learning (PCL) [171] first targets bridging contrastive learning with clustering. Compared to instance discrimination as pretext tasks learning low-level representations, clustering can help to encode more semantic information. Then more semantic-based downstream tasks will benefit from it. As shown in Fig. 12, prototypical contrastive learning uses prototypes to substitute one of the views of generated samples in NCE loss (Eq. 15), which is the proposed ProtoNCE loss in PCL. In addition, PCL is also a method based on soft parameter sharing, in which the momentum encoder is updated as Eq. (14).

4.8 Summary

This section extensively investigates recent progress in PFMs on images for representation learning, from the early perspective of designing pretext tasks for self-labeling to present contrastive loss-based SSL. The pipelines of the main methods are clearly illustrated. We hope this section can prepare the incoming researchers to acquire a basic understanding of this novel area and some worthwhile research direction. We believe the powerful generalization ability of PFMs would extremely reduce training computation overhead by “pretraining once and transferring forever”. Recent transformer-based PFMs have gradually outperformed traditional training from scratch on target datasets. This discovery will spur further exploration and research into this exciting field.

5 PFMs for graph learning

With the development of deep learning in graphs, the parameters (i.e., graph embedding) of the model began to increase rapidly. Therefore, large-scale labeled data is needed for training the models to avoid under-fitting or over-fitting. However, the cost of constructing large-scale labeled datasets for graphs is too subjective, expensive, and time-consuming, especially in domains that require professional knowledge and timeliness. While some semi-supervised approaches have temporarily mitigated the

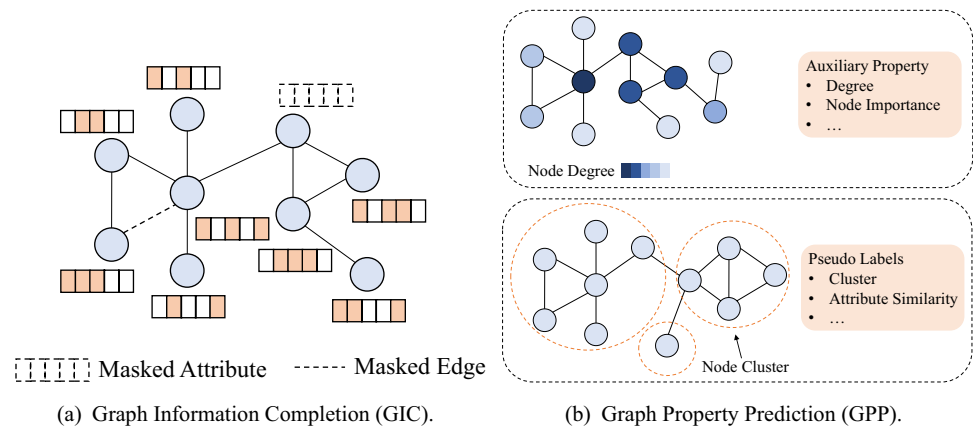
Table 2 Summary of the PFMs in CV

Year	Conference	Method	Pretext task	Architecture	Downstream task ¹	Code
2014	NeurIPS	Exemplar-CNN [134, 135]	Discrimination	CNN	cla, rec	https://lmb.informatik.uni-freiburg.de/...
2015	ICCV	Context [136]	Context prediction	CNN	cla, det, clu	https://github.com/.../deepc
2016	CVPR	Inpainting [137]	Inpainting	GAN, CNN	cla, det, seg, inp	https://github.com/.../cont
2016	ECCV	Colorization [138]	Colorization	CNN	cla, det, seg	https://github.com/.../color
2016	ECCV	Jigsaw [140]	Jigsaw puzzles	CNN	cla, det, seg, ret	https://github.com/.../Jigsa
2017	CVPR	Split-Brain [139]	Channel prediction	CNN	cla, det, seg	https://richzhang.github.io/splitbrainauto
2017	ICCV	Counting [142]	Counting	CNN	cla, det, seg, ret	https://github.com/clvrai/...
2017	ICML	NAT [143]	Noise	CNN	cla, det	–
2017	ICLR	BiGAN [48]	Generation	GAN, CNN	cla, det, seg	https://github.com/.../bigan
2018	WACV	CDJP [141]	Jigsaw puzzles	CNN	cla, det, seg	–
2018	ICLR	RotNet [138]	Rotation	NIN, CNN	cla, det, seg	https://github.com/gidar
2018	arXiv	CPC [145]	Patch overlapping	CNN, GRU	cla	–
2018	CVPR	NPID [156]	Instance discrimination	CNN	cla	https://github.com/.../lemni
2018	ECCV	DeepCluster [49]	Clustering	CNN	cla, det, seg	https://github.com/.../deepc
2019	ICCV	LA [161]	Local aggregation	CNN	rec, det	https://github.com/.../Local
2019	NeurIPS	BigBiGAN [147]	Generation	GAN, CNN	gen, cla	https://tfhub.dev/.../bigbigan
2019	CVPR	AET [172]	Transformation	CNN	cla	https://github.com/.../AET
2019	NeurIPS	AMDIM [173]	Discrimination	CNN	cla	https://github.com/.../amd
2020	CVPR	ClusterFit [174]	Clustering	CNN	cla, seg	–
2020	ICML	CPC v2 [146]	Patch overlapping	CNN	cla, det	–
2020	CVPR	PIRL [162]	Jigsaw puzzles	CNN	cla, rec, dec	https://github.com/.../PIRL
2020	CVPR	MoCo [163]	Discrimination	CNN	cla, rec, dec, pos, seg	https://github.com/.../moco
2021	ICLR	PCL [171]	Clustering	CNN	cla, det	https://github.com/.../PCL
2020	arXiv	MoCo v2 [164]	Discrimination	CNN	cla, dec	https://github.com/.../moco
2020	ICLR	SeLa [175]	Self-labelling	CNN	cla, det, seg	https://github.com/.../self
2020	ICML	SimCLR [166]	Discrimination	CNN	cla	https://github.com/.../simclr
2020	NeurIPS	SimCLR v2 [176]	Self-distillation [160]	CNN	cla	https://github.com/.../simclr
2020	ECCV	CMC [177]	View matching [178]	CNN	cla, seg	https://hobbitlong.github.io/CMC
2020	NeurIPS	InfoMin [179]	Discrimination	CNN	cla, det, loc, seg	https://hobbitlong.github.io/InfoMin
2020	NeurIPS	SwAV [167]	Cropping	CNN, Transformer	cla, det	https://github.com/.../swav
2020	NeurIPS	BYOL [165]	Discrimination	CNN	cla, det, seg	https://github.com/.../byol
2021	arXiv	MoCo v3 [180]	Discrimination	CNN, Transformer	cla	–
2021	ICLR	ReLIC [181]	discrimination	CNN	cla, rel	–
2021	ICLR	PCL v2 [171]	clustering	CNN	cla, det	https://github.com/.../PCL
2021	CVPR	SimSiam [170]	discrimination	CNN	cla, det, seg	https://github.com/.../simsi
2021	ICML	DirectPred [182]	discrimination	CNN	cla	https://github.com/.../ssl
2021	ICCV	DINO [151]	discrimination	CNN, Transformer	cla, seg	https://github.com/.../dino

Table 2 (continued)

Year	Conference	Method	Pretext task	Architecture	Downstream task ¹	Code
2021	arXiv	MoBY [183]	discrimination	CNN, Transformer	cla, det, seg	https://github.com/.../Transformer-SSL
2021	NeurIPS	MST [184]	Token prediction	CNN, Transformer	cla, det, seg	–
2022	ICLR	BEiT [185]	Token prediction	Transformer	cla, seg	https://github.com/.../beit
2022	CVPR	MAE [153]	Reconstruction	Transformer	cla, det, seg	https://github.com/facebookresearch/mae
2022	CVPR	SimMIM [154]	Reconstruction	Transformer	cla, det, seg	https://github.com/microsoft/SimMIM
2022	ArXiv	UM-MAE [158]	Reconstruction	Transformer	cla, det, seg	https://github.com/implus/UM-MAE
2022	ArXiv	LoMaR [159]	Reconstruction	Transformer	cla, det, seg	https://github.com/junchen14/LoMaR
2022	Arxiv	CAE [186]	Reconstruction	Transformer	cla, det, seg	https://github.com/lxtGH/CAE
2023	AAAI	PeCo [187]	Reconstruction	Transformer	cla, det, seg	–
2023	ArXiv	SAM [155]	Reconstruction	Transformer	det, gen, seg	https://github.com/facebookresearch/segment-anything

¹ Downstream task types: classification (cla), recognition (rec), detection (det), localization (loc), segmentation (seg), clustering (clu), inpainting (inp), retrieval (ret), generation (gen), pose estimation (pos), reinforcement learning (rel)

Fig. 13 Graph Information Completion (GIC) and Graph Property Prediction (GPP)

reliance of graph embedding models on label scale, they have not fundamentally resolved this problem. In recent times, researchers have turned their attention towards the application of PFMs in the field of graphs, inspired by their success in CV and NLP. However, for most graphs, obtaining large-scale pretraining data directly is challenging due to the unique nature of information such as nodes and edges. Therefore, recent studies have focused on utilizing the inherent information of a graph's attributes, topology, and community to enhance the effectiveness of the node's features. We have summarized the graph-related PFMs in Table 3.

5.1 Learning by graph information completion

The essential motivation of pretraining based on graph information completion (GIC) is to mask part of the information of the input graph data and recover the masked information based on the unmasked graph data, so as to pretrain the graph embedding, as shown in Fig. 13. Similar ideas appeared earlier in the field of image and text processing. For instance, in image processing, information such as image pixels and colors are recovered to pretrain the image encoder; in text processing, many methods implement pretraining of word embeddings and encoders by recovering part of the information in a sentence based on context words. These methods inspire the design of graph completion tasks on graph PFMs.

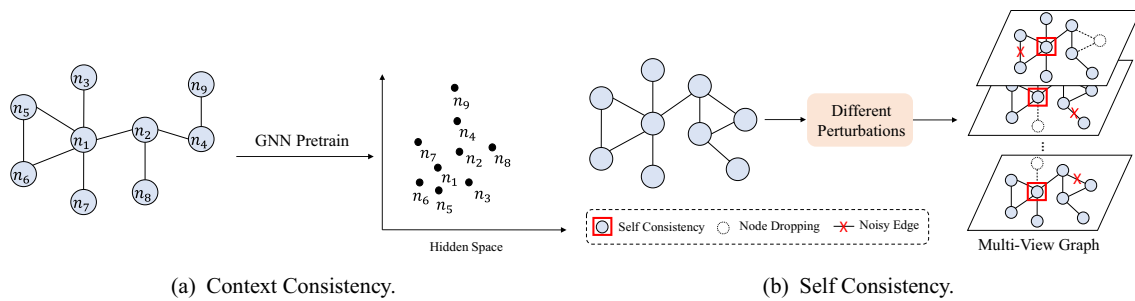


Fig. 14 Graph consistency analysis (GCA)

Among them, You et al. [188] are inspired by image inpainting, and first propose to cover them by removing the features of the target nodes, and then recovering/predicting the features of the masked nodes. In order to recover/predict the masked information, GraphCompetition [188] is achieved by providing GCNs with unmasked node features (limited to the 2-layer GCNs of the second-order neighbors of each target node). The purpose of GraphCompetition's pretraining is to help the model better perform feature representation and teach the model to extract features from the context. You et al. [188] propose the attribute mask task (namely, AttributeMask), which masks node attributes randomly, and then requires the self-supervising module to reconstruct the masked attributes. Jin et al. [189] think deeply about SSL on graph data, and propose the edge mask task (namely, EdgeMask), seeking to develop self-supervision in pairs based not only on a single node itself but on the connection between two nodes in the graph. In particular, EdgeMask randomly masks some edges and then asks the model to reconstruct the masked edges. In short, EdgeMask is expected to help GNN learn local connectivity information. Hu et al. [190] propose a PFM that masks node and edge attributes and then predicts this masked information based on the adjacent structure.

5.2 Learning by graph consistency analysis

Different from the aforementioned methods that focus on individual elements in the graph, graph consistency analysis (GCA) mainly explores the consistency of the distribution of two elements in the graph. Specifically, the consistency of two elements with similar semantics should be significantly stronger than two elements with unrelated semantics, and this characteristic can be used to pretrain the graph model. According to the judgment object of consistency, such methods can be roughly divided into the following three categories.

Context consistency

Based on the early homogeneity assumption, a mass of graph models tends to project contextual nodes to similar

positions in semantic space. Such consistency of the context in the graph is also applied to the pretraining graph model, which attempts to adjust the node representation by capturing the distribution characteristics of the nodes in the context, as shown in Fig. 14a.

Random walk is an efficient method to acquire context. It can capture the distribution characteristics of different perspectives in the context by designing a variety of walk strategies. The DeepWalk [191] adopts a truncated random walk strategy to represent the node context as the form of a sequence of nodes. By introducing the idea of NLP into the network embedding model, DeepWalk regards the node sequence as a “sentence” and models it based on the skip-gram model, providing an unsupervised and scalable training method for node representation. Furthermore, on the basis of DeepWalk, node2vec [192] uses two different parameter-controlled random walk strategies to obtain deviated node sequences to fully capture the context information.

Different from randomly sampling nodes from the context, some recent methods directly consider the relationship between the node's k -order neighbor distribution (as positive examples) and non-adjacent nodes (as negative examples), and use this to train the graph model. LINE [193] respectively proposes first- and second-order proximity to describe the local similarity between pairs of nodes in the graph from different perspectives, and uses it to optimize node representation. Meanwhile, LINE uses negative sampling and edge sampling techniques to optimize the second-order traversal and excessive training storage overhead. VGAE [194] introduces a variational autoencoder to encode graph structure data, and model the node first-order neighbor through a GCN encoder and a simple inner product decoder.

Self consistency

In the field of NLP and CV, contrastive learning as an efficient self-supervised mechanism is widely used in the pretraining of models. In fact, the internal comparison mechanism of such methods is based on the mutual information estimation of the original graph data and the augmented graph data to maintain the consistency of the data itself, as

Table 3 Summary of PFM in GL

Year	Conference	Method	Pretext task	Encoder	Code
2014	KDD	DeepWalk [191]	GC-C	Shallow NN	https://github.com/phanein/deepwalk
2015	WWW	LINE [193]	GC-C	Shallow NN	https://github.com/tangjianpku/LINE
2016	NeurIPS	VGAE [194]	GC-C	GCN	–
2016	KDD	node2vec [192]	GC-C	Shallow NN	https://github.com/aditya-grover/node2vec
2017	NeurIPS	GraphSage [214]	GC-C	Shallow NN	https://github.com/williamleif/GraphSAGE
2018	ICLR	DGI [200]	GC-CS	GCN/SAGE	https://github.com/PetarV-/DGI
2020	ICML	GraphCompetition [188]	GIC	GCN	https://github.com/Shen-Lab/SS-GCNs
2020	ICLR	AttMasking [190]	GIC	GCN	http://snap.stanford.edu/gnn-pretrain
2020	ICML	AttributeMask [188]	GIC	GCN	https://github.com/Shen-Lab/SS-GCNs
2020	arXiv	EdgeMask [189]	GIC	GCN	https://github.com/ChandlerBang/SelfT ask-GN
2020	arXiv	NodeProperty [189]	GPP-PR	GCN	https://github.com/ChandlerBang/SelfT ask-GN
2020	AAAI	M3S [203]	GPP-PC	GCN	–
2020	ICML	Node Clustering [188]	GPP-PC	GCN	https://github.com/Shen-Lab/SS-GCNs
2020	ICML	Graph Partitioning [188]	GPP-PC	GCN	https://github.com/Shen-Lab/SS-GCNs
2020	NeurIPS	CSP [204]	GPP-PC	GCN	–
2020	NeurIPS	MP [204]	GPP-PC	GCN	–
2020	NeurIPS	SELR [215]	GC-C	GNN	https://github.com/mlvlab/SELR
2020	KDD	GCC [195]	GC-S	GIN	https://github.com/THUDM/GCC
2020	NeurIPS	GraphCL [197]	GC-S	GCN	https://github.com/CRIPAC-DIG/GCA
2020	ICML	CMVRL [201]	GC-CS	GCN	–
2020	ICDM	SUBG-CON [202]	GC-CS	GCN	https://github.com/yzjiao/Subg-Con
2020	ICLR	InfoGraph [216]	GC-CS	GCN	https://github.com/fanyun-sun/InfoGraph
2020	AAAI	DMGI [217]	GC-CS	GCN	https://github.com/pcy1302/DMGI
2020	arXiv	Graph-Bert [210]	Hybrid	Transformer	https://github.com/jwzhanggy/Graph-Bert
2020	WWW	GMI [211]	Hybrid	GCN	–
2020	KDD	Gpt-GNN [212]	Hybrid	GNN	https://github.com/acbull/GPT-GNN
2021	ICML	JOAO [218]	GC-S	GCN	https://github.com/Shen-Lab/GraphCL_ Automated
2021	AAAI	CSSL [219]	GC-S	GCN	https://github.com/UCSD-AI4H/GraphSSL
2021	PAKDD	GIC [198]	GC-CS	GCN	https://github.com/cmavro/Graph-InfoC lust-GIC
2021	WWW	SUGAR [199]	GC-CS	GCN	https://github.com/RingBDStack/SUGAR
2021	ICML	GraphLoG [220]	GC-CS	GCN	https://github.com/DeepGraphLearning/ GraphLoG
2021	WWW	SLiCE [221]	GC-CS	GCN	https://github.com/pnnl/SLICE
2021	WSDM	BiGI [222]	GC-CS	GCN	https://github.com/caojiangxia/BiGI
2021	WWW	GCA [196]	GC-S	GCN	https://github.com/CRIPAC-DIG/GCA
2021	KDD	HeCo [223]	GC-CS	GCN	https://github.com/liun-online/HeCo
2021	AAAI	CG ³ [209]	Hybrid	GCN	–
2021	ICLR	SuperGAT [224]	GC-C	GAT	https://github.com/dongkwan-kim/Super GAT
2021	KDD	MoCL [225]	Hybrid	GNN	https://github.com/illidanlab/MoCL-DK
2022	ArXiv	MGAE [205]	Maksed Edge Reconstruction	GCN	–
2022	KDD	GMAE [206]	Maksed Node Reconstruction	Transformer	https://github.com/THUDM/GraphMAE
2022	Arxiv	MaskGAE [207]	Partial Maksed Node Reconstruction	Transformer	https://github.com/EdisonLeeeee/MaskGAE
2022	Arxiv	HGMAE [208]	Metapath Masking Reconstruction	Transformer	–

shown in Fig. 14b. Inspired by contrastive learning, some studies have begun to generate augmented samples of original data samples in the graph model. Among them, two augmented samples from the same original sample are regarded as positive pairs, and two augmented samples from different original samples are regarded as negative pairs.

For node-level tasks, GCC [195] devises the pretext task as subgraph instance discrimination in and across networks. GCC also enhances the ability of GNNs to learn intrinsic and transferable structural representations by introducing contrastive learning. Specifically, GCC samples subgraphs from the whole graph as augmentations via random walk with restart and artificially designs positional node embedding as node initial features. As a novel graph representation learning model, GCA [196] incorporates various priors for topological and semantic aspects of the graph to achieve adaptive contrastive augmentation. Specifically, GCA devises an enhancement scheme based on node centrality measures to highlight important connection structures, while corrupting node features by adding noise to specific nodes to lead the pretraining model to recognize underlying semantic information.

For graph-level tasks, some studies have attempted to introduce more diverse contrastive learning strategies. Among them, You et al. [197] introduce four common graph augmentation tasks (i.e., node dropping, edge perturbation, attribute masking, and subgraph sampling) into the GL model based on underlying prior and propose a unified comparative learning framework: GraphCL. Meanwhile, GraphCL discusses in depth the role of data augmentation in comparative learning and gives experimental demonstration that joint multiple augmentation strategies can improve model performance.

Cross scale consistency

Unlike the above two methods that consider the consistency of elements in the same scale, contrasting elements in graph data of different scales can also be used to train graph models, e.g., node-subgraphs. Most of such methods have the idea of maximizing mutual information [198, 199]. Specifically, the readout function is usually used to obtain the summary of the graph/subgraph, and the MI estimator can be calculated using the Jensen–Shannon divergence.

As a representative method, DGI [200] relies on maximizing the MI between the patch representation and the summary of the corresponding high-level graphs, which are all derived using the established graph convolutional network architecture, to learn the node representation. To generate negative samples on a single graph, DGI corrupts the original graph by randomly scrambling node features while keeping the structure unchanged. Similarly, Hassani and Khasahmadi propose CMVRL [201], which generates

an additional structural view of a sample graph based on graph diffusion. The sample graph and a regular view are sub-sampled together, and the node representation and graph representation are learned based on two shared MLPs, and then contrast learning is achieved through the consistency loss provided by the discriminator.

SUBG-CON [202] samples a series of context subgraphs from the original graph and inputs them to the encoder to obtain the pooled central node and subgraph representation. For the specified node, the context subgraph is expressed as a positive sample, and other randomly sampled subgraphs are expressed as a negative sample. The contrast loss of the latent space will force the encoder to identify positive samples and negative samples in order to distinguish different nodes based on regional structure information.

5.3 Learning by graph property prediction

Considering the attribute and structural information of the graph as the target of information completion, pretraining based on graph property prediction (GPP) can also be used to build the graph model in different forms. One of the most common methods is to generate self-supervised signals by exploring the auxiliary property in the graph data and to take the graph property prediction task as the pretraining task of the graph model. According to the different settings of the pretext task, it can roughly classify two categories: property regression and property classification.

Property regression (PR)

In the graph model, different from the GIC mentioned above, property regression primarily focuses on mining the relationship between the broader numerical structure and property attributes within the graph. Specifically, this branch of methods extracts richer self-supervised signals in graph data for pretraining graph models.

For example, similar but different from masking node attributes, the goal of NodeProperty [189] is to predict each node's auxiliary property in the graph, e.g., degree, local node importance, and local clustering coefficient. In other words, NodeProperty is used to encourage GNN to capture richer local structural information while optimizing the specific downstream tasks. Specifically, NodeProperty regards the node degree as a representative local node property, i.e., self-supervised signal, and takes other node properties as future work. Meanwhile, NodeProperty emphasizes that the intuition of devising self-supervised pretext tasks related to local node property is to ultimately guide the feature embedding of GNN (i.e., node representation) to save this information, which relies on the assumption that the node property information is relevant to the particular task.

Property classification (PC)

Different from the property regression task, the task of property classification is usually implemented by defining pseudo-labels based on a certain distribution in the graph data, which is a typical self-supervised method. Among them, the structure density, similarity of node attributes, and difference between local and global distributions are the most commonly used. We will briefly introduce the application of such methods in GL pretraining.

Among these methods, clustering is the most common and effective source of pseudo-labels. Among them, M3S [203] designs a multi-stage training strategy, using the idea of graph clustering to iteratively train the graph encoder, achieving enlarged labeled data with virtual labels in the case of very small samples. You et al. [188] further propose two pretraining strategies. Among them, Node Clustering assigns K (hyper-parameter) pseudo labels to nodes based on attribute clustering and pretrain node representation by node classification. In addition, You et al. also present Graph Partitioning based on the topology density assumption. In Graph Partitioning, the nodes of a graph are divided into approximately equal K (hyper-parameter) subsets to minimize the number of edges connecting nodes among subsets, and then pseudo labels are provided for nodes.

In addition to clustering methods, some researchers generate pseudo labels based on other statistical characteristics of graph data. For instance, in the molecular field, Rong et al. [204] use the molecular bonds of subgraphs and related statistical information to guide GNN to learn Context-Sensitive Properties (CSP) and then apply them to prediction. Rong et al. [204] propose a Motif Prediction (MP) task, which can be expressed as a multi-label classification problem, in which each motif corresponds to a label. Specifically, let's assume that K motifs in molecular data are considered. For a specific molecule (abstracted as graph G), they use RDKit to detect whether each motif appears in G , and then take it as the target of the motif prediction task.

5.4 Learning by masked autoencoder

The masked autoencoder (MAE) is first applied in MAGE [205], the masked autoencoders for self-supervised learning on graphs. Following MAE [153], MGAE operates on a partial network structure (without masked edges) that is based on convolutions. Besides, the decoder of MGAE is designed to model the cross-correlation between the head and tail nodes of an anchor edge. Empirical results demonstrate that MGAE performs better than traditional graph autoencoders and graph SSL approaches. Furthermore, GMAE [206] extends this approach by using a transformer instead of convolutions and reconstructing the features of masked nodes rather than masked edges. In addition to empirical improvements, MaskGAE [207] further

provides theoretical justifications for the potential benefits of masked graph modeling. Designing algorithms to accommodate graphs of various complex properties is a promising direction. For instance, to tackle the heterogeneous graphs scenario, HGMAE [208] proposes meta-path masking and adaptive attribute masking with a dynamic mask to enable effective and stable learning on complex graph structure. Moreover, several training strategies are developed, including meta-path-based edge reconstruction to incorporate complex structural information, target attribute restoration to utilize various node attributes, and positional feature prediction to encode node positional information. Besides dealing with more complex graph structures, how to improve the learning efficiency of MAE on graph data remains an open question.

5.5 Other learning strategies on graph data

In addition to the above methods, there are lots of pretraining methods that use relatively novel or hybrid strategies. For example, CG³ [209] generates an improved node representation by designing a semi-supervised consistency loss to maximize the consistency between different views of the same data or data from the same category. Next, CG³ uses the graph generation loss related to the input feature to extract the potential deterministic relationship between the data feature and the input graph topology as a supplementary supervision signal for SSL.

Based on the attention mechanism, Graph-Bert [210] trains itself to reconstruct node attributes and topological structure with sampled linkless subgraphs within their local contexts. GMI [211] extends the traditional mutual information computing idea from the vector space to the graph domain and proposes to jointly maximize feature mutual information (between the node's embedding and raw features of its neighbors) and edge mutual information (embedding of two adjacent nodes) for graph representation learning. GPT-GNN [212] proposes a self-supervised graph generation task to guide itself to capture the topological and semantic attributes of the graph. GPT-GNN roughly divides the possibility of graph generation into attribute generation and edge generation to untangle the intrinsic dependence between node attributes and graph topology.

5.6 Summary

In the graph model, as traditional feature learning methods are often accompanied by information loss in the process of feature learning, and the information taken into consideration is relatively one-sided, the obtained graph representation is relatively rough and loses mass information. People began to focus on the distribution of data and attributes in the graph data as self-supervised signals to pretrain the

graph model so that it can capture more valuable information. By transforming the distribution of nodes, attributes, and edges in the graph into different pretext tasks, and using GNNs for modeling, the graph model can fully fit the original distribution of the input graph. In lots of unsupervised or semi-supervised scenarios, such pretrained graph models have been proven to benefit downstream tasks. Besides, federated training large graph models [213] can be a promising solution for building pretrained foundation models. Currently, with the in-depth study of contrastive learning strategies, some work has attempted to apply contrastive learning in different forms to the pretraining of graph models. Through the consistency analysis of context, self, and cross-scale, this kind of method greatly improves the performance of the pretrained graph model on different graphs.

6 PFMs for other data modality and learning method

With the rapid development of the PFMs, except for text, image, and graph, the PFMs have also carried out a lot of research on speech, video, text-image, and cross-data. Besides, researchers have started investigating the unified PFMs that incorporate all three mentioned domains recently. Therefore, in this section, we introduce some other advanced and unified PFMs for other data modalities and learning methods.

6.1 PFMs for speech

In the field of speech, wav2vec [226] obtains speech representation by capturing contextual information on large-scale unlabeled datasets, and fine-tuning on a few samples by noise comparison binary classification task, which greatly improves the performance of downstream tasks. Furthermore, vq-wav2vec [227] and wav2vec 2.0 [228] propose a discrete unsupervised pretraining method on the basis of wav2vec, discretizing the original continuous speech signal, so that the methods in the mature NLP community can be migrated and applied. Meanwhile, lots of research have tried to design different mechanisms to use the representation obtained by speech pretraining as the initial input, and apply it to different tasks, e.g., automatic speech recognition [228, 229], phoneme recognition [230], and speech synthesis [231]. In particular, the extensive application of spoken language understanding has promoted the research of joint pretraining of speech and text. For example, SpeechBERT [229] applies MLM to speech and text pairs to perform representation learning on discrete information. Unlike [232], which relies on a large amount of labeled data for joint pretraining, SPLAT [233] uses unlabeled speech data to pretrain the speech embedding module,

and proposes a label-level alignment method suitable for label-level downstream tasks based on sequence alignment. MusicBERT [234] is a pretrained model designed for processing music data. It was developed by training on a vast symbolic music corpus consisting of over one million songs. To improve the pretraining process with symbolic music data, MusicBERT employs several mechanisms, such as OctupleMIDI encoding and a bar-level masking strategy. Huang et al. [235] suggest incorporating a metrical structure in the input data, which allows Transformers to better recognize the hierarchical structure of music at the beat-bar-phrase level. AudioTransformer [236] is a model that enhances the performance of Transformer architectures by implementing certain techniques, such as pooling, which were previously used in convolutional networks. Verma et al. [236] demonstrate how they leverage multi-rate signal processing ideas based on wavelets to improve the Transformer embeddings and obtain better results.

6.2 PFMs for video

Video is similar to the RGB features of image and sequence information of the text. Many meaningful explorations in self-supervised video representation learning can not only perform efficiently well in video datasets but also generalize to the learning in other areas. Odd-One-Out Networks (O3N) [237] is a technique that targets to predict the odd video subsequence among real subsequences sampled from a video in a training dataset. The experiments are conducted by using different video-clip encoders for O3N to prove consistent improvements of this pretraining design. Similarly, Shuffle and Learn [238] aims to learn the correct temporal order from a sequence of frames in a video. However, Kim et al. [239] designed a new self-supervised task called Space-Time Cubic Puzzles to train 3D CNNs. This task requires a pretrained backbone to arrange permuted 3D spatiotemporal crops. The performance of downstream tasks proves that effective video representations have been learned while solving such puzzles.

Inspired by the contrastive learning in images, many pretraining models in the video also utilize the contrastive loss to learn video presentations for downstream tasks. Inter-Intra Contrastive (IIC) framework [240] can learn video representations by using positive and negative pairs generated from different videos. Specifically, different modalities in the same video are treated as positive pairs, and video clips from different videos as negative ones. Temporal Contrastive Pretraining (TCP) [241] is another contrastive method based on CPC to learn video representations. Different from the existing GAN-based method that generates future frames for the video directly, TCP can predict the latent representation of future frames of the video, which is better for long-term predictions. Sequence Contrastive Learning (SeCo) [242]

is a novel method considering both intra- and inter-frame instance discrimination in sequence order-based task.

6.3 PFM for multimodal

The multimodal PFM among text and image can be divided into two categories: single-stream model and cross-stream model. The single-stream model refers to integrating text information and visual information at the beginning of the model. The Cross-stream model refers to text information and visual information encoded by two independent coding modules, respectively. Then different modal information is fused by mutual attention mechanism.

Single-stream model

VisualBERT [243] inputs text and images into the model simultaneously, which are aligned and fused using Transformer's self-attention mechanism. The input of the text is the same as BERT, and the input of the image is the image features extracted by Fasters-RCNN. VisualBERT also does pretraining and then fine-tuning the specific task. It adopts two pretraining tasks, namely MLM and sentence-image prediction, determining whether the input sentence describes the corresponding image. The structure of Unicoder-VL [244] is very similar to VisualBERT, except for the processing of the image. Unicoder-VL extracts the image feature through Faster-RCNN and concatenates the feature with image position-encoding mapping to the same space. It enhances the image label prediction task, which predicts the categories of images. The pretraining task of VL-BERT [245] is the same as Unicoder-VL. The image input of VL-BERT includes four parts: the image region features extracted by Fasters-RCNN, the location of the region in the original image, location coding, fragment encoding, and [IMG] encoding.

Cross-stream model

In ViLBERT [246], the text and image modes are first encoded separately, and their outputs go through a standard attention module. This module is based on the Transformer structure. Still, in the self-attention mechanism, each module uses its query to calculate attention with the value and key of another module to integrate the information between different modules. The model is pretrained on two tasks. The first task is the mask task, which is the same as BERT. On the image side, the goal of the task is that when the region image is masked, the classification distribution of the output of the model can be as consistent as possible with the output distribution of the model used to extract the region features (such as Faster-RCNN). The second task is the language image matching task. DALL-E is a series of deep learning models developed by OpenAI to generate images from natural language prompts. The first version of DALL-E uses

a transformer-based architecture, similar to the one used in the GPT LMs, to process the textual prompts and generate image-like representations. The model is trained on a dataset of images and their associated textual descriptions based on GPT-3. DALL-E 2 [247] is the improved version by employing contrastive language-image pretraining (CLIP) [248] for capturing semantic association between image-text pairs and GLIDE diffusion model [249] for text-conditional image synthesis. Furthermore, GPT-4 is proposed by OpenAI recently. It is a large-scale multimodal model which adopts RLHF and demonstrates human-level performance on various professional and academic benchmarks.

Based on the multi-modal data containing more available information than previous single-modality data, thus the performance of these models gets enhanced by combining with the SSL on the benchmark dataset. Cross and Learn [250] is the first method that reveals crossmodal information as an alternative source of supervision and obtains powerful feature representations from combining crossmodal loss and diversity loss in both RGB and optical flow modalities. Different from the existing methods that learn feature representations from only a single task in cross-domain datasets, Ren and Lee et al. [251] propose a novel deep multi-task network to learn more generalizable visual representations to overcome the domain difference and further utilize the cross-domain information in different tasks. In that paper, the cross-domain datasets are real and synthetic datasets generated by a GAN-based network, while the multiple tasks are the predictions of the surface normal, depth, and instance contour in RGB images. This model performs better than any previous single-task-based SSL methods by learning general-purpose visual representations from cross-domain multi-task feature learning. Tian et al. [252] believe that a powerful representation is one that models cross-view factors from the perspective of humans view to understand the world. They propose Contrastive Multiview Coding (CMC) to learn a video representation by maximizing mutual information between different views of the same scene.

6.4 PFM for code generation

Code generation with LLMs involves using pretrained language models to automatically generate code based on natural language descriptions of a desired program. This approach has the potential to greatly improve the efficiency of software development by reducing the need for manual coding and allowing developers to focus on higher-level tasks.

The technique involves training large-scale language models on vast amounts of natural language text and then fine-tuning the models on specific programming tasks. By inputting natural language descriptions of code, the model can generate code snippets that are syntactically and

semantically correct. Code generation with LLMs has been applied in various programming domains, including web development, NLP, and data analysis. The models used for code generation include GPT-4, T5, and Codex, among others. For example, Andrei et al. [253] have investigated and assessed the fine-tuning of transformer models for personalized code generation. Specifically, they have evaluated the effectiveness of various personalization techniques in the domain of generating unit tests for Java methods and learning to personalize for a specific software project. Shailja et al. [254] assess the capacity of LLMs to generate Verilog that is useful. To achieve this, pretrained LLMs are fine-tuned on Verilog datasets collected from GitHub and Verilog textbooks. An evaluation framework is then constructed, consisting of test benches for functional analysis and a flow for testing the syntax of Verilog code generated in response to problems of varying degrees of difficulty. An open-source CodeGen LLM that has undergone fine-tuning has been shown to outperform the current leading commercial Codex LLM. The CodeGen [255] is a group of LLMs that have up to 16.1B parameters and can handle both natural language and programming language data. Additionally, they have released the training library JAX FORMER as open-source. Their work demonstrates that the model can perform as well as the previous state-of-the-art zero-shot Python code generation on HumanEval, showcasing the practical applications of the trained model. Synchromesh, introduced in the study by Poesia et al. [256], adopts a novel approach called Target Similarity Tuning (TST) to retrieve a small set of examples from a training bank. Then, Synchromesh utilizes these examples to train a pretrained language model and generates programs by applying Constrained Semantic Decoding (CSD). CSD is a general framework that can restrict the output to valid programs in the target language. In this work, the authors show that the combined use of CSD and TST results in significant improvements in prediction accuracy, as well as preventing runtime errors.

However, there are still some limitations to code generation with LLMs, such as the models' tendency to generate overly verbose or inefficient code and their inability to handle complex programming tasks. Nevertheless, the technology has shown significant promise and has the potential to revolutionize the software development industry.

6.5 SOTA unified PFMs

A big convergence of PFMs handling multiple modalities is emerging, such as backbone architecture, pretraining task, and model scaling up [29]. Therefore, many unified PFMs proposed by researchers arise. A unified PFM is a unified model pretrained on unimodal and multimodal data with single or multiple transformers as the backbone, which has the ability to perform a large variety of downstream AI tasks,

including unimodal tasks and multimodal tasks. There are currently three types of SOTA unified models based on model architectures. We defined them as the single-transformer model, multi-transformer model, and comb-transformer model. A single-transformer model refers to a PFM model which only has a large-scale transformer as its backbone, whereas a multi-transformer model refers to a PFM model having multiple transformers. A comb-transformer model is the PFM model with the combination of both single and multiple transformer structures.

Single-transformer model

UNITER [257] is a large-scale PFM for joint image-text embedding, which consists of an Image Embedder, a Text Embedder, and a multi-layer Transformer. It first encodes visual features and bounding box features for image regions using Image Embedder and tokens and positions using Text Embedder. Then, a Transformer module is applied to learn generalizable contextualized embeddings for images and text through four pretraining tasks. Instead of applying random joint masking to both modalities, conditional masking on pretraining tasks is used. Six vision-language tasks are selected as the downstream tasks.

Uni-Perceiver [258] is a single Siamese model with shared parameters having the ability to process different modalities regarding vision and language tasks. Different task inputs and targets are encoded into unified token sequences with modality-specific tokenizers, which are then decoded by a modality-agnostic weight-sharing Transformer encoder into the shared representation space. Any perception task is modeled as finding the maximum likelihood target for each input through the similarity of their representations. Uni-Perceiver is pretrained on unimodal and multimodal tasks. The evaluation results on various downstream tasks show that the performance is close to SOTA methods by conducting prompt tuning on 1% of downstream task data.

Gato [259] builds a single large transformer sequence model that works as a multimodal, multi-task, multi-embodiment generalist policy. It can perform various tasks using a single neural network with the same set of weights. Gato is trained on 604 tasks, where different types of data, such as images, text, proprioception, joint torques, and other discrete and continuous observations and actions, are serialized into a flat sequence of tokens, batched, and processed by the transformer. During deployment, sampled tokens are assembled into different actions based on the context.

OFA [26] is a simple sequence-to-sequence learning framework with a unified instruction-based task representation that unifies various tasks. In the pretraining and finetuning stages, OFA requires no extra task-specific layers for downstream tasks to achieve Task-Agnostic. The Modality-Agnostic compute engine is a Transformer with the constraint that no learnable task- or modality-specific

components are added to downstream tasks. OFA is pre-trained on small-size image-text pairs to achieve crossmodal tasks while attaining highly competitive performances on unimodal tasks.

UNIFIED-IO [27] is a sequence-to-sequence model using a unified architecture that performs large and diverse tasks. UNIFIED-IO is a transformer model where both the encoder and decoder are composed of stacked transformer layers. The unified architecture does not need specific task or modality branches, which is accomplished by homogenizing the input and output of each task into a sequence of discrete vocabulary tokens. It trains a single transformer-based architecture on over 90 diverse datasets in the vision and language fields. UNIFIED-IO is the first model to perform various tasks and produce strong results across 16 diverse benchmarks without finetuning.

BEiT-3 [29] is a general-purpose multimodal pretrained model on language, vision, and vision-language tasks. The big convergence of BEiT-3 can be seen from three aspects, including backbone architecture, pretraining task, and model scaling up. It introduces a shared Multiway Transformer as a backbone network performing masked data modeling on both unimodal and multimodal data. To process different modalities, every Multiway Transformer block has a shared self-attention module, and a pool of feed-forward networks. It is a giant-size foundation model that contains 1.9B parameters. Experimental results show that BEiT-3 can outperform SOTA models on both vision and vision-language tasks.

Multi-transformer model

FLAVA [28] is an alignment model that targets all modalities at once and aims at solving vision and language tasks, and vision-language tasks. It utilizes a common transformer model architecture to learn strong representations from unimodal and multimodal data. An image encoder transformer is used to capture unimodal image representations. A text encoder transformer is adopted to process unimodal text information. A multimodal encoder transformer takes both encoded unimodal images and text as inputs and integrates their representations for multimodal reasoning. During pretraining, masked image modeling (MIM) and MLM losses are applied to the image and text encoders, respectively. On the other hand, masked multimodal modeling (MMM) and image-text matching (ITM) loss are used over paired image-text data. For downstream tasks, classification heads are applied to the outputs from the image, text, and multimodal encoders, respectively, for visual recognition, language understanding, and multimodal reasoning tasks. FLAVA shows good performance on 35 tasks across different domains. A noticeable advantage is that smaller datasets it used compared with other models.

Comb-transformer model

UNIMO [260] can learn both single modality and multimodalities with one model to achieve robust and generalizable representations. It employs multi-layer self-attention Transformers to learn general textual and visual representations simultaneously and unifies them into the same semantic space via cross-modal contrastive learning (CMCL). The main idea behind CMCL is to keep paired image and text representations close to the representation space while keeping non-paired representations far away. All of them are encoded by the same unified-modal Transformer in pairs or individually, and the representations of images and texts are extracted to compute the contrastive loss.

6.6 PFM for reinforcement learning

The success of pretraining learning methods in the supervised learning domain has spurred interest in the reinforcement learning (RL) domain to study whether the same paradigms can be adapted to RL algorithms. General pretraining RL can include broad directions, such as Reward-Free RL [261–264], Goal-condition RL [265–267], and Representation Learning in RL [268–271]. Here we focus the Representation Learning in RL. Specifically, this direction seeks to *improve the performance by pretraining the visual perception competent of RL agent, i.e., the state encoder, with some large-scale datasets using unsupervised/self-supervised data augmentation techniques*. The pretraining process empowers the state encoder to capture the essential structure information from the raw inputs (pixel-level input for CV). An RL policy network is built based on the pretrained state encoder to learn the specific downstream control tasks in the fine-tuning stage. Recent studies have demonstrated that can greatly benefit both sample efficiency and learning effectiveness from unsupervised [272–274], semi-supervised [275], and self-supervised [276, 277] learning techniques. Specifically, this direction could be roughly classified into the following two categories: Model-based Pretraining RL and Contrastive-like Pretraining RL.

Model-based pretraining RL

Model-based Pretraining RL aims to first pretrain a generative world model to capture the underlying structure of the environment and then leverage the world model as a state encoder or simulator during fine-tuning. World Models [272] is the first work that proposes to learn a compressed spatial and temporal representation of the environment in an unsupervised manner using a simple Variational Autoencoder, which greatly improves the sample efficiency compared to training from scratch. However, learning the world model without being aware of the environment's dynamic might lead to ignorance of some key information in the environment. Dreamer [278, 279] proposed to learn latent dynamics

by approximating the representation, transition, and reward model. They then train RL agents purely by imagination in a latent space, which is more efficient since it brings a low memory footprint and enables fast predictions of thousands of imagined trajectories in parallel. Furthermore, DreamerPro [280] proposes a reconstruction-free approach based on prototypical representations to migrate the task-irrelevant visual distractions problem in the latent dynamics modeling. DreamerPro significantly outperforms previous SOTA methods when there are complex background distractions. To verify whether learning accurate world models for the real world is promising, Daydreamer [281] applies Dreamer to the real-world physical robots problem and empirically demonstrates significant learning efficiency gains.

Contrastive-like pretraining RL

Contrastive-like Pretraining RL techniques seek to improve the representation ability of state encoders by pretraining the state encoder with a large amount of out-of-domain data or adding some auxiliary loss using unsupervised learning or data augmentation techniques. CURL [282] combines instance contrastive learning and by using MoCo [163] mechanism, which significantly improves the data efficiency of RL agents. Furthermore, RAD [283] proposes an implicit approach that directly trains the RL objective on multiple augmented observations views, which outperforms CURL on some of the environments in the DeepMind Control Suite. Concurrent to RAD, DrQ [284] introduces a simple regularization term, which applies image augmentation to compute current and target Q values. They demonstrate that data efficiency can be significantly improved after applying it to DQN. DrQ-v2 [285] further extends this approach to solve complex humanoid locomotion tasks by inserting similar techniques into the DDPG algorithm. Orthogonal to this direction, [268, 269, 286, 287] demonstrate that pretraining the vision part of RL agent using supervised or unsupervised methods on out-of-domain data can improve the learning efficiency of downstream RL control tasks. Besides ensuring consistency across different views of observation, SPR [271] additionally trains a dynamics model that enforces the representations to be temporally predictive. Based on SPR, SGI [270] proposes to pre-train representations using a combination of latent dynamics modeling, unsupervised goal-conditioned, and inverse dynamics modeling. Compared to previous methods, SGI can better capture the environment's dynamics and facilitate downstream RL control task training.

7 Other advanced topics on PFMs

As the number of parameters of the pretraining model increases, the pretraining model requires more memory and computing resources. It increases the training cost of

PFMs and limits their deployment on resource-constrained devices. Therefore, to improve the efficiency of the pretraining model, PFM improves computational efficiency from the following two aspects: model efficiency and model compression. The model efficiency and compression of the PFM refer to simplifying the redundancy of model parameters and structure. Under the condition that the task completion degree is not affected, the model with fewer parameters and a more concise structure is obtained.

7.1 Model efficiency

Model efficiency devotes to exploring more efficient pre-training methods to pretrain large-scale PFMs with a lower-cost solution. More efficient learning algorithms require more effective training methods and more efficient model architecture. Traditional pretraining tasks may be inefficient. For example, the commonly used masked token prediction task requires the model to predict masked tokens based on context. However, the masked tokens in the samples are usually a subset of the input tokens, and the model can only learn from this part of the tokens, so the training efficiency is low. To solve this problem, ELECTRA [30] proposes an RTD task that predicts whether each input marker is replaced by other tokens, which enables the ELECTRA to train against all input tokens. In addition to effective training methods, more efficient architecture can also improve the efficiency of PFMS. For most PFMS based on the Transformer algorithm, a more efficient model architecture can be obtained by reducing the complexity of the Transformer algorithm.

7.2 Model compression

Model compression requires less computing resources and memory. It is a potential approach to reduce the model size and enhance computation efficiency. The model compression strategy can be divided into two ways: parameter compression and structure compression.

The methods of parameter compression include parameter pruning, parameter quantization, low-rank decomposition, and parameter sharing. Parameter pruning refers to designing evaluation criteria for model parameters to delete redundant parameters based on a sizeable PFM. For example, Compressing BERT [35] prunes BERT before training while maintaining the performance equivalent to that of the original model. Parameter quantization is the quantization of model parameters from 32-bit full-precision floating-point numbers to lower-order numbers. For example, Q8BERT [84] uses 8-bit quantization to compress parameters fourfold with little impact on model performance. Low-rank decomposition is to reduce the dimension of a high-dimensional parameter vector into a sparse low-dimensional

vector. Parameter sharing refers to the structured matrix or clustering methods to map model parameters and reduce the number of parameters. For example, the ALBERT [36] uses decomposition-embedded parameterization and cross-layer parameter sharing to reduce the parameters in the model.

Structure compression refers to compact networks and knowledge distillation. A compact network means reducing the number of parameters and calculations by designing a new compact network structure. Knowledge distillation refers to the transfer of knowledge from the larger teacher model to the smaller student model through the use of a soft label, etc. DistilBERT [85], for example, uses the knowledge distillation method to compress BERT, reducing the size of the BERT model by 40% while retaining 97% of its language comprehension.

7.3 Security and privacy

The security risks, social bias, and data privacy in PFMs become important research topics. Qiu et al. [5] recognize that deep neural networks can be attacked by adversarial samples, which mislead the model to produce false predictions. Due to the excellent portability of pretraining models, they have been widely used in NLP, CV, and GL. However, it has been found that the pretraining model is susceptible to the influence of adversarial samples. A tiny interference of the original input may mislead the pretraining model to produce specific false predictions. Meanwhile, it is possible to recover the data samples by querying the PFMs which can cause privacy leakage.

Generation adversarial samples

The adversarial sample originates from the image. Despite of the physical attacks in the image domains [288–290], the adversarial samples of the image are hard to recognize with an invisible change. For example, only one pixel of the image is modified. Human beings do not easily detect such disturbance, but the neural network can identify the modified image, which is the original purpose of the adversarial sample. Some work has found that pretrained LMs are vulnerable in some scenarios. Jin et al. [291] successfully attack the three target models of BERT, CNN, and RNN by generating natural adversarial samples, which indicates that the current language processing model still has a large room for improvement in terms of security. However, it is difficult to achieve due to the distinct discreteness of languages in NLP. In particular, the generation of adversarial samples in the text must take into account linguistic characteristics to ensure that the sample's syntax and fluency are not harmed while affecting the model's output. For example, [291] uses adversarial samples to attack the fine-tuning stage of the BERT model for text classification and entailment successfully. [292] combines the sememe-based word substitution

method and search algorithm based on particle swarm optimization to generate adversarial samples.

Model defects

Some unrelated human factors can also mislead the PFM to make wrong predictions. For example, [33] discovers that the performance of BERT is limited in the reasoning task due to utilizing false statistical information in the dataset, which dramatically affects the performance by destroying this property. [293] defines universal adversarial triggers. When triggers are connected to any input, it can induce the model to generate specific predictions.

Backdoor attacks

There are still many methods to manipulate the predicted results of the pretraining model employing a backdoor attack. [294] demonstrates that it is possible to construct a weight poisoning attack in which pretrained weights are injected. After the fine-tuning stage, the backdoor is exposed. Attackers manipulate model predictions easily by injecting arbitrary keywords. [295] shows that PFMs in NLP can be manipulated by modifying the model corpus. The “meaning” of new words or existing words can be controlled by changing their weight parameters.

Defense against attacks

The human-in-the-loop method [31, 32] has been proposed and applied to generate more natural, efficient, and diversified adversarial samples. Some defense approaches have been proposed to defend against such attacks. [296] designs an auxiliary anomaly detection classifier and uses a multi-task learning procedure to defend against adversarial samples. On the other hand, some defects in the PFM may be inherited by the custom models in transfer learning, such as the adversarial vulnerabilities and backdoors mentioned above. To mitigate this issue, [297] proposes a relevant model slicing technique to reduce defect inheritance during transfer learning while retaining useful knowledge from the PFM.

Data privacy in PFMs

LLMs and other PFMs have been trained on private datasets [298]. The researchers have discovered that by querying the massive LMs, it is feasible to recover specific training samples. An adversary may, for instance, obtain IRC discussions and personally identifiable information. Even worse, because large models have so many parameters, it is simple for PFM to memorize or learn private information, making larger models more prone to attack than smaller ones. Many PFMs such as the LLMs have been trained on private datasets. The researchers have found that it is possible to recover individual training examples by querying the LLMs. For instance, an adversary can extract examples including

personally identifiable information, and Internet Relay Chat (IRC) conversations. Even worse, because of the billion parameters of large models, it is easy for PFM to learn private information, making the larger model more vulnerable than smaller models. We must take privacy-preserving measures into account during all PFM processes, including data processing, model training, model inference, and system deployment, in order to reduce the risks of privacy leakage.

8 Future research challenges and open problems

The PFM can avoid training models from the scratch, which is a breakthrough from weak AI to general AI. At present, due to the characteristics of PFM such as large-scale parameters, a large amount of training data, and high computational complexity, there are still many technical challenges in PFMs. We summarize the future research challenges of PFMs from four perspectives: data, foundation, model design, and upstream and downstream tasks. Meanwhile, we point out some open problems in the future research direction.

8.1 Challenges on data

Most pretrained datasets are for single modes and single languages. It is very important for the development of PFMs to construct pretrained datasets for multimodal, multi-lingual, and graph data. For the characteristics of these data, the existing technical challenges are as follows:

Data deficiencies

Unlike NLP and CV, except for the reusable nodes in a few molecular and protein networks, most of the nodes and edges in the graph data do not have a large amount of unlabeled data for pretraining. Meanwhile, the pretraining research of the graph model is still in its initial state. Besides, data from the Internet of Things (IoT) will be enormous and contains rich physical world information. For example, inertial measurement unit sensor data can capture users' social activity information [299, 300]. The theoretical basis, various definitions of the pretext task, and the augmented design of contrastive learning are all imperfect, and new research urgently needs to be supplemented.

Multimodal PFM

Some research work has been done on multimodal PFMs, such as text and image, text and audio, etc. These are mostly PFMs between two modalities. At present, the learning of multimodal PFMs requires new multimodal datasets, which

demand the establishment of the data between different modes. Thus, the construction of multimodal datasets is also an urgent problem to be solved.

Multi-lingual PFM

The multi-lingual PFM solves the resource shortage problem in multiple languages, and it aids in the achievement of new improvements in QA, text summarization, low-resource neural machine translation, and so on. However, the current PFM is still a mask LM. To improve the performance of the multi-LM, some suitable new tasks need to be added. In addition, multi-lingual vocabularies are much larger than single-language vocabularies, resulting in a sharp increase in model parameters to be learned.

8.2 Challenges on foundation

For a PFM, a theoretical foundation is essential to model performance, whether it is a “black box” or “white box” method. The foundation studied mainly includes theoretical foundation, semantic understanding, and explicable exploration.

Lack of theoretical foundation

There is no profound theory to support all kinds of tentative experiments, and further exploration has no handbook. Although there are several theoretical analysis that tries to understand the collapse of pretraining or the generalization ability of the learning representation, the lack of theoretical foundation is still a huge cloud upon the head of SSL.

Semantic understanding

Does the pretrained LM learn the meaning of the language, or just rely on corpus learning? Many models perform well on various datasets with helpful information that can be extracted, where some approaches even exceed human levels. However, the performance is poor on domain datasets or relatively small datasets. The models cannot reach a better level of stability and match different downstream tasks. This means that the model cannot serve the real purpose of human language use.

8.3 Challenges on model design

Most existing structures of PFMs are tried for text, image, and graph. The primary method is to increase data, improve computation power, and design training procedures to achieve better results. How to make a trade-off between data, computing resources, and predictive performance is worth studying.

Model variety

There are many attempts at model design, such as generation-based models in the CV area. However, GAN-based approaches are not popular for the following two reasons: (1) the discriminator has learned meaningful feature representations, but they are forgotten during training [301]; (2) the mode collapse causes the generator to output samples in singular mode to cheat the discriminator. As a result, although researchers attempt to apply GAN-based approaches on SSL for pretraining, the difficulties in the convergence of discriminator and divergence of generator hinder development and progress in this area.

Model compression

With the wide application of the Transformer and the pretraining model showing a general trend of growth, the computational complexity of the pretraining model has become the focus of attention. Due to the huge hardware requirements of model training and other reasons, the high threshold makes it difficult for researchers to train from scratch. BERT-base and GPT-3 contain about 108 million parameters and 175 billion parameters, respectively. It is not conducive to the development of relevant research work. There are some works for pretraining model compression, such as ALBERT having fewer parameters and better effect than BERT-base. The improvement models still require powerful computing equipment, making them difficult to apply universally. Reducing the high computing cost is one of the main challenges in future research.

Model robustness

Although many researchers have designed different pretext tasks for the pretraining, the main problem remains on how to design robust pretext tasks and judge the performance before large-scale computations. In addition, how to compare these proposed methods fairly is also a big issue. As for NLP, deep neural networks are vulnerable to adversarial inputs because of their linear characteristics. Although pretraining models perform well on different NLP tasks, most are based on deep neural networks, which generally have poor robustness. Operations such as cutting and rotating do not change the nature of the image in CV. In contrast, operations such as adding, deleting, and substituting a word in the text are likely to affect the semantics of the text. Therefore, how to improve the robustness of the PFM in NLP is a technical challenge.

Model anti-attack

The PFMs are vulnerable to attack by adversarial examples, which can easily mislead the model to produce specific false predictions. It is difficult to process due to the unique discreteness of language in the NLP field. Thus, the current PFMs have huge room for improvement in model anti-attack.

8.4 Challenges on finetuning and prompt

The pretrained model in NLP, CV, and GL fields can achieve good performance in most upstream tasks, but not all good in downstream tasks for fine-tuning and prompt. How to achieve consistent results both on upstream and downstream tasks is still a challenge for the PFMs.

Saturation phenomena

Google Research [302] observed the nonlinear relationship between the performance of upstream and downstream tasks. The higher training accuracy with more data on the upstream tasks does not always lead to better performance on the target downstream tasks. This observation challenges the most intuitive understanding of the pretraining process. Even in the most extreme case, the performance of upstream and downstream is at odds.

Pretext task

There are too many self-supervised tasks, also known as pretext tasks. The pretext task can be used for any downstream tasks, such as detection and classification. It is difficult to match the relationship between pretext tasks and downstream tasks.

Task-based graph

Much of the pretraining on the graph is based on the task graph. Different tasks construct different graphs, where nodes need to be reused. This makes it impossible to pretrain on the graph by introducing as much data as NLP and CV.

8.5 Open problems for future PFMs

First of all, a big convergence of text, image, graph, and multimodal pretraining is expected. Till the survey is written, no work has considered the graph in their unified PFMs. All of the SOTA unified models mainly focus on the language, vision, and language-vision tasks, while neglecting the importance of the graph in the data domain. Second, a unified backbone architecture for unified PFMs in future research will become more popular. It can be seen that a unified PFM model which only has a large-scale transformer as its backbone, i.e., a single-transformer model, is more focused by researchers than other types of unified PFMs. Third, a unified PFM is expected to achieve SOTA transfer performance for all different tasks in all data domains, including text, image, graph, and multimodalities. Most unified PFMs are only outstanding in a single data domain, whereas the performance in other domains is not competitive. BEiT-3 [29] shows a great example in both vision and vision-language tasks towards this research direction. Besides, in terms of RL usage in PFMs, even though ChatGPT builds the milestone in NLP, CV and GL do not have

significant research published yet. More work in this direction is expected in the future.

9 Conclusion

Existing PFMs in text, image, and graph domains are principally summarized in this survey. Firstly, we introduce the basic components of NLP, CV, and GL. Then, we provide a summary of existing models designed for pretraining in the three domains and summarize the necessary information in terms of model structures. Furthermore, we study some other research for PFMs, including other advanced and unified PFMs, model efficiency and compression, and security and privacy. Finally, we present the main challenges and open problems in PFM research.

Appendix A Basic components

A.1 Basic components on NLP

A.1.1 Language model

With the rapid development of deep learning, LMs are more and more applicable to the pretraining of NLP models. The LM can estimate the probability of rationality of a paragraph of the text. There are two main types of LMs: statistical LM and neural network LM.

Statistical LM The statistical LM is a mathematical model to solve the context-related characteristics of natural language from the perspective of probability and statistics. The core of statistical LMs is to determine the probability of a sentence appearing in a text. As the theoretical basis of the probabilistic LM, the N-gram model profoundly influences the subsequent LM. It plays a pivotal role in the field of the LM. The N-gram LM introduces the Markov hypothesis, which assumes that the probability of the occurrence of the current word only depends on the nearest $n - 1$ words. The maximum likelihood probability of a word w_i can be calculated by

$$p(w_i | w_1, w_2, \dots, w_N) = p(w_i | w_{i-n+1}, w_{i-n+2}, \dots, w_{i-1}) = \frac{C(w_{i-n+1}, w_{i-n+2}, \dots, w_i)}{\sum_N C(w_{i-n+1}, w_{i-n+2}, \dots, w_{i-1})}, \quad (16)$$

where $T = [w_1, w_2, \dots, w_N]$ is the text sequence and $C(w_{i-n+1}, w_{i-n+2}, \dots, w_i)$ is the co-occurrence frequency of $(w_{i-n+1}, w_{i-n+2}, \dots, w_i)$. The $p(w_i | w_1, w_2, \dots, w_N)$ is calculated according to the chain rule

$$p(w_1, w_2, \dots, w_N) = \prod_{i=1}^N p(w_i | w_1, w_2, \dots, w_{i-1}). \quad (17)$$

N-gram uses the probabilities of each word in the sequence to represent the co-occurrence probability of the whole text sequence. When N is large, it indicates a more vital constraint on the occurrence of the next word in the sequence and leads to more sparse frequency information. When N is small, the statistical results have higher reliability and better generalization ability, but the constraint will be weaker.

Neural LM The statistical LM adopts maximum likelihood estimation, which is intuitive and easy to understand. However, there are still problems such as a lack of long-term dependence, the rapid growth of parameter space and sparse data. Therefore, the neural network is introduced to map the LM to a continuous space. Neural LMs use distributed representations of words to model natural language sequences. Unlike class-based N-gram models, neurolinguistic models are able to recognize two similar words without losing the ability to encode each word as different from the other. It can be directly used for NLP tasks. It mainly introduces Forward Feedback Neural Networks (FFNN), Recurrent Neural Networks (RNN), and pretrained LMs.

As shown in Fig. 15a, FFNN according to the all former words of $x = [w_1, \dots, w_{i-1}]$ calculates the probability of w_i . In order to predict the conditional probability of w_i , x is sharing the projection matrix $M \in R^{|V| \times m}$ to a continuous feature vector space according to the projection index, $|V|$ is word library size, m is the dimension of the feature vector. The output is represented as

$$y = b_2 + H_x^{1,3} + H_x^{2,3} \tanh(b_1 + H_x^{1,2}), \quad (18)$$

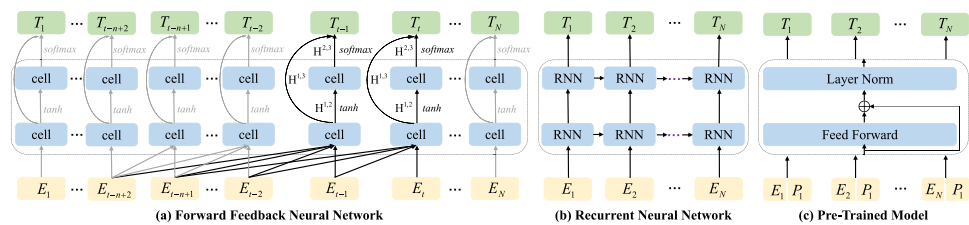
where $H^{1,2}$, $H^{2,3}$ and $H^{1,3}$ are the weight matrices used to connect each layer, and b_1 and b_2 are the bias values of the hidden layer and the output layer respectively.

The structure of the FFNN contains only limited information about the foregoing and has some limitations on the length of the input sequence. Therefore, the RNN LM comes into being. As shown in Fig. 15b, RNN can accept input of any variable length. When the input window is moved, its internal state mechanism can avoid repeated calculation, and parameter sharing further reduces the number of model parameters. Therefore, compared with FFNN, RNN has a great advantage.

The pretraining LM is to get a set of model parameters by pretraining some tasks. It initializes the model with these parameters and then trains to improve the model performance effectively. The commonly used pretraining models are fixed embedding (Word2vec [12], Glove [69], etc), variable embedding (Embeddings from LMs (ELMO) [303], Generative pretrained Transformer (GPT) [50] and Bidirectional Encoder Representations from Transformers

Table 4 Commonly used notations on NLP and graph

NLP		Graph	
Notations	Descriptions	Notations	Descriptions
N	The length of input text	$ \cdot $	The length of a set
w_i	The i th word in input text	$\mathcal{G}$	The set of graphs
$ V $	The word corpus size	G	A graph
H_x	The representation of the input sequence	V	The set of nodes in the graph
$\overline{\theta}_f$	The parameters for forward modeling	v	A node
$\overline{\theta}_b$	The parameters for backward modeling	E	The set of edges in the graph
θ	The shared parameter in all permutations	e_{ij}	An edge between v_i and v_j
Z_N	The set of all possible permutations of T	A	The adjacency matrix of a graph
$z_{T=t}$	The t th element of z	$\mathcal{T}$	The set of node types in a graph
$z_{T< t}$	The $[1, 2, \dots, t-1]$ elements of z	X	The feature matrix of a graph
z	A permutation of T	$\mathcal{Y}$	The set of ground truth labels in a graph
m	The dimension of the feature vector	D	The given graph data
b_1, b_2	The bias values of the hidden layer and the output layer	M_{GL}	The GL model

Fig. 15 The model architectures of forward feedback neural network, recurrent neural network and Pretrained LMs. $H^{1,2}$, $H^{2,3}$ and $H^{1,3}$ are the weight matrices used to connect each layer

(BERT) [13], etc). Here, we give an example of the GPT model, as shown in Fig. 15c. It adopts a two-stage process. In the first stage, the Transformer decoder is used as the basic unit of the model to perform text prediction. In the second stage, the GPT is initialized differently for different downstream tasks, training the model and fine-tuning the parameters.

A.2 Basic components on GL

Due to the extensive use of graph data in many fields, some communities (e.g., chemistry, protein, and social network) have recently focused on the study of graph pretraining. These pretraining models encode graph attributes, structures, and other information into node representations from multiple perspectives by designing different pretext tasks, which are used to optimize downstream tasks. In this section, we introduce the definition of the basic concepts of graphs, and then provide a formal definition of the PFM on the graph.

A.2.1 Notations and definitions of graphs

Unless particularly specified, the notations used in this article are illustrated in Table 4. We use $\mathcal{G} = \{G_i\}_i^N$ to represent a set of graphs, where N represents the number of graphs.

Depending to the graph's definition of the edges and nodes, graph data can be classified into the following types.

Definition 1 An **unattributed graph** is $G = (V, E)$, where $v \in V$ is a node, $e \in E$ is an edge, and naturally $E \subseteq V \times V$. Adjacency matrix $A \in \mathbb{R}^{n \times n}$ represents the topology of graph G , where $n = |V|$. $A_{i,j} = 1$ denotes there is an edge between node v_i and v_j , otherwise $A_{i,j} = 0$.

Definition 2 An **attributed graph** is $G = (V, E, X_v, X_e)$, where $X_v \in \mathbb{R}^{n \times d_v}$ and $X_e \in \mathbb{R}^{m \times d_e}$ are the feature matrices of nodes and edges, $|V| = n$, $|E| = m$, d_v and d_e denotes the feature dimensions of node and edge. In fact, in most application scenarios, only nodes have attributes, and edges have no attributes or only weights.

Definition 3 An **undirected graph** is $G = (V, E)$, where $e_{i,j} \in E$ means an unordered node pair (v_i, v_j) . In particular, the adjacency matrix A of the undirected graph is a symmetric matrix (i.e., $A_{i,j} = A_{j,i}$).

Definition 4 A **directed graph** is $G = (V, E)$, where $e_{i,j} \in E$ means an ordered node pair (v_i, v_j) .

Definition 5 G has a node-type mapping function $f_v : V \rightarrow \mathcal{T}$ and an edge-type mapping function

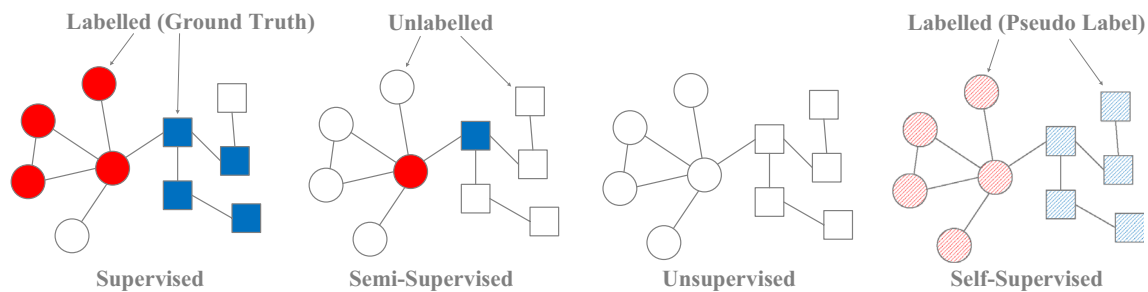


Fig. 16 Schematic of different supervision modes

$f_e : E \rightarrow \mathcal{T}^e$. When $|\mathcal{T}^v| = |\mathcal{T}^e| = 1$, the graph $G = (V, E)$ is a **homogeneous graph**. In other words, all nodes in G belong to a type, and all edges also belong to one type.

Definition 6 When $|\mathcal{T}^v| > 1$ and/or $|\mathcal{T}^e| > 1$, the graph $G = (V, E)$ is a **heterogeneous graph**. In particular, a heterogeneous graph must be an attributed graph.

A.2.2 Learning settings on graphs

GL methods are usually used to solve machine learning tasks on graph data, and we introduce different settings (supervision mode and learning mode) for GL.

Before that, we first provide the notations of the corresponding mathematical formulation of GL. $C = \{c_1, c_2, \dots, c_K\}$ is a set of target components defined in a graph set $\mathcal{G}$ ($G^{c_i} \in \mathcal{G}$), and c_i is associated with a corresponding ground truth label $y_i \in \mathcal{Y} = \{1, 2, \dots, N_y\}$, where K denotes the total number of target components, and N_y is the number of classes being predicted. Then the graph data can be represented as $D = \{c_i, G^{c_i}, y_i\}_i^K$, and a complete GL model M_{GL} can also be determined by $y_i = M_{GL}(c_i, G^{c_i})$. For instance, in a node classification task, c_i is the node to be classified, y_i denotes c_i 's label in graph G^{c_i} . Similarly, in a node clustering task, c_i is the node to be clustered, y_i denotes the corresponding cluster label in graph G^{c_i} .

Supervision mode

Depending on the source and scale of the training data, the supervision settings of GL can be divided into four types as shown in Fig. 16. **Supervised** GL is the most common mode in the real scenario. Given the target component c_i and the corresponding ground truth label y_i , the goal is to minimize the loss function between the predicted label of the GL model (i.e., $y_i^{pred} = M_{GL}(c_i, G^{c_i})$) and the expected label y_i of all c_i . Compared with supervised learning, **unsupervised** GL refers to situations in which no labeled data is provided, only the attributes and structure distribution of graph data (i.e., (c_i, G^{c_i})) can be used. **Self-supervised** GL is a special case of both supervised and unsupervised learning.

Specifically, self-supervised learning mainly uses pretext tasks (e.g., clustering, completion, and partition) to mine its own supervised information (i.e., pseudo-labels) from large-scale unsupervised graph data, and trains the GL model M_{GL} through the self-supervised information, so that it can learn to the valuable features of downstream tasks. In other words, the supervised information of self-supervised learning is not manually labeled, but the pretext tasks automatically construct supervised information from large-scale unsupervised data for supervised learning or training. **Semi-supervised** learning is a combination of unsupervised and supervised learning, which aims at learning data distribution to predict unlabeled data to solve the problem of difficulty in obtaining labeled data in real scenarios. In GL, semi-supervised learning refers to the realization of pattern recognition given a few labeled data and mass unlabeled data.

Learning mode The GL model M_{GL} is optimized by the given training samples, and adjusted on the validation samples to participate in the test. According to the visibility of the graph data at different stages, the learning settings of GL model M_{GL} can be classified into two categories: inductive learning and transductive learning.

Definition 7 Inductive Learning, which is the most common setting in machine learning tasks, trains the model on labeled data and then tests on samples that have never appeared in the training stage. Formally, given a training sample $\{(c_i, G^{c_i}, y_i)\}_{i=1}^{N_l}, \{(c_j, G^{c_j})\}_{j=1}^{N_u}$, where N_l and N_u are the numbers of labeled/unlabeled samples. Inductive learning learns a function $f^{ind} : \mathcal{G} \mapsto \mathcal{Y}$ so that f^{ind} is expected to be a good classifier on the future graph data $\{(c_k, G^{c_k})\}$, beyond $\{(c_j, G^{c_j})\}_{j=1}^{N_u}$.

Definition 8 Transductive Learning is different from inductive learning in that all samples are visible during both the training and testing stages. Formally, given a training sample $\{(c_i, G^{c_i}, y_i)\}_{i=1}^{N_l}, \{(c_j, G^{c_j})\}_{j=1}^{N_u}$, transductive learning learns a function $f^{trans} : \mathcal{G}^{l+u} \mapsto \mathcal{Y}^{l+u}$ so that f^{trans} is

expected to be a good classifier on the unlabeled data $\{(c_j, G^{c_j})\}_{j=1}^{N_u}$.

Under the supervised setting (including semi-/self-supervised), the unified classifier optimization methods of inductive learning and transductive learning can be written as:

$$\mathcal{L} = \frac{1}{K} \sum_{i=1}^K \mathcal{L}(f_{\theta}^{(\cdot)}(c_i, G^{c_i}), y_i), \quad (19)$$

where $\mathcal{L}$ is the cross-entropy loss, c_i can be node, edge or subgraph of its associated graph G^{c_i} , and $f_{\theta}^{(\cdot)}$ denotes inductive/transductive function with parameter θ .

Compared with using only one pretext task, some methods have designed some integration mechanisms to incorporate the advantages of multiple pretext tasks into a unified framework.

B Traditional learning methods

B.1 Traditional text learning

NLP is a research field that integrates linguistics and computer science. Its main research tasks include part-of-speech tagging, named entity recognition, semantic role labeling, machine translation, question answering, sentiment analysis, text summarization, text classification, relationship extraction, event extraction, etc. The LM can be considered the cornerstone of the downstream NLP tasks. It experiences four processes: grammar rule LM, probabilistic LM, neural network LM, and pretraining LM. A PFM trains on a large benchmark dataset to obtain a model that can solve new similar tasks, which has become a new hotspot in current LM research.

Word representations play a significant role in downstream tasks, which is the basis of NLP. The N-gram model preprocesses text features and encodes adjacent N words as a group, which makes it overly dependent on the richness of the training corpus. Otherwise, data-sparse is likely to occur, and the computational complexity will increase exponentially with the increase of N . Neural Network LM (NNLM) [11] adopts the idea of word vector for the first time, and the low-dimensional word vector of distributed representation can solve the discrete problem caused by word embedding well. However, it is still challenging to solve the problem of high computational complexity. The computational complexity of the word2vec model is independent of the selected window size but is determined by the dictionary size and the word vector dimension. Many downstream tasks can be significantly improved by training on a large corpus using word vector embedding after initial training. However, the problem of polysemy for the static

word vector is still unsolved, and it still belongs to the shallow LM [304, 305]. Therefore, more effective models are urgently needed to deal with the data set more flexibly. To capture high-level concepts of context, such as polysemy elimination, syntactic structure, etc. Neelakantan et al. [306] propose to learn multiple embeddings per word type. Zhou et al. [307] integrate the features on both dimensions of the matrix to enrich semantics by using subword information. Based on the Continuous Bag Of Words (CBOW) [12] in word2vec, Hui et al. [308] fine-tune the generated word vectors for emotion and obtain the word vectors containing both semantic meaning and emotional tendency, which significantly improved the performance in the Weibo sentiment classification task. Liu et al. [309] propose a model of hierarchical translation for machine translation. It uses the neural LM based on RNN as the word vector generation model. Liang et al. [310] propose an approach based on the double-layer self-attention mechanism for machine reading comprehension, and the model is divided into three parts: single document encoder, multi-document encoder, and answer prediction. In the single document encoder, the problem of the context information is represented by the Gated Recurrent Unit (GRU) model. Zhang et al. [311] propose an INDependent RNN (INDRNN) and attention mechanism for user intention classification, using word vectors generated by word2vec as input. The model introduces a word-level attention mechanism to effectively quantify the contribution of domain vocabulary to the intention category.

B.2 Traditional image learning

There are several types of neural networks in the deep learning era, from the beginning of most famous convolutional neural networks (CNNs) to the subsequent Attention- and Transformer-based networks. A deep neural network refers to an artificial neural network with more hidden layers, and more parameters are used to represent the target model, which leads to the SOTA performance on the benchmark dataset from image to video. Here, we introduce the milestone networks in CV chronologically.

B.2.1 Convolution-based networks

ImageNet [312], as one of the most important databases in computer vision, has aroused many milestone network architectures in image classification, including AlexNet [313], NIN [314], VGG [315], GoogLeNet [316], ResNet [317], DenseNet [318], etc. When it comes to object detection and semantic segmentation, researchers explore R-CNNs [319–322], FCN [323], SSD [324], YOLOs [325–329], SegNet [330], PSPNet [331], Deeplabs [332–335], RefineNet [336], etc. on common

benchmark datasets, such as PASCAL VOC [337, 338], MS COCO [339], etc.

There are several shared features among these popular convolution-based networks: (1) *data augmentation*. Deep models require much more data to fit a complicated model, thus the data augmentation technique such as flipping, rotation, cropping, scaling, translation, and even adding noises enlarges the training dataset; (2) *convolution*. The convolutional kernel is used to extract the features of original image data, which maintains the spatial structure for the adjacent pixels; (3) *deep architecture*. The deep architecture contains more parameters, which enhance the capability of the model. These common features have contributed to the SOTA performance of convolutional neural networks (CNNs) in computer vision for nearly recent 10 years.

B.2.2 Recurrent neural networks

Different from CNNs targeting 2D-dimensional image applications, recurrent neural networks (RNNs) [340–342] try to use recursive cells to process pictures in sequence, i.e., video data. However, the weaknesses of gradient explosion and long-term dependencies restrict further development of this model. To handle these problems embedded inside the RNN-based models, long short-term memory (LSTM) [343] was proposed by Hochreiter and Schmidhuber in 1997. In addition, the improved capability of LSTMs produces popularity and attracts attention both in NLP and CV [344–348].

B.2.3 Generation-based networks

Generative Adversarial Networks (GANs) [349] have provided a paradigm to learn representations for unlabelled data, and spawn many GAN-based approaches on downstream tasks. In image translation, pix2pix software [350] first proposes the conditional adversarial networks as a solution to the image-to-image translation problems, and achieves reasonable results on real-world datasets. Markovian Generative Adversarial Networks (MGANs) [351] is a method to generate texture synthesis, which can be applied to style transfer and video stylization. CycleGAN [352] provides a learning algorithm to translate an original image from the source domain to a target domain without containing pairs of images in datasets for supervised learning. StyleGAN [353] is a style-based generator to serve as an alternative architecture for traditional GANs. Pixel Recurrent Neural Networks (PixelRNN) [354] aims to complete images by modeling full dependencies between the color channels. DiscoGAN [355] is designed to learn relations between different domains.

GANs have also provided a novel direction to study data synthesis because it perfectly simulates the distribution of the original data. Laplacian Pyramid of Adversarial

Networks (LAPGAN) [356] uses a cascade of convolutional networks to generate images in a coarse-to-fine fashion. Similarly, Stacked Generative Adversarial Networks (SGAN) [357] decompose variations into multiple levels and gradually resolve uncertainties by stacking several GANs in a top-down way.

B.2.4 Attention-based networks

Based on the success of CNNs in the area of CV, the attention module is designed to be equipped with the popular CNNs. For example, SENet [358] proposes a channel attention module, which won first place in the competition of ILSVRC2017. In addition, CBAM [359] sequentially infers attention maps along both channel and spatial dimensions. Many innovative works, such as GCNet [360] and CCNet [361], are inspired by this idea of soft-attention mechanism, which outperforms the traditional CNNs on major benchmarks for both recognition and segmentation tasks. In particular, the self-attention mechanism [362], calculating the response at a position among all entities in a sequence by attending to all positions within the same sequence, is proposed to estimate the relevance of one position to other positions in feature maps. To control the expected entities and model more complex relations among different elements in the sequence, masked self-attention and multi-head attention [38] are the key components proposed to substitute the function of convolutions in the era of transformers.

B.2.5 Transformer-based networks

Recently, inspired by the self-attention mechanism and subsequent success of the transformer in NLP, researchers in CV also have tried to use the transformer as an alternative to the convolution. Self-attention-based transformer models always operate in a two-stage training mechanism: (1) pretraining on a primitive dataset (always big but not well labeled) by defining pretext tasks; (2) transferring the pretrained weights to the downstream tasks and adjusting the parameters on the target domain dataset by finetuning. Vision Transformer (ViT) [40] is applied on CV and achieves the SOTA performance on major benchmark datasets. Data-efficient image Transformers (DeiT) [363] was proposed by Facebook AI to train image transformers more efficiently and maintain the SOTA performance simultaneously. DETection TRansformer (DETR) [364] significantly outperforms competitive baselines in both object detection and semantic segmentation. LeViT [365] outperforms existing benchmarks with respect to balancing the accuracy and training speed. Image GPT [149] is inspired by a sequence transformer in NLP, which can compete with several self-supervised benchmarks on ImageNet. On the basis of this research, DeepViT [366]

explores a deeper architecture to improve performance consistently by making the transformer go deeper. Moreover, many researchers try to apply the transformer to more specific tasks. Pyramid Vision Transformer (PVT) [367] introduces the pyramid structure to overcome the difficulties of porting the transformer to various dense prediction tasks, and achieves the SOTA performance on major benchmark datasets. M3DeTR [368] is a novel research on multi-representation, multi-scale, and mutual-relation 3D object detection with transformers. Medical Transformer (MedT) [369] has focused on medical image segmentation and outperforms previous CNN-based and transformer-based architecture. In conclusion, the transformer has become a novel and popular research area in CV and its performance is proved by many existing works.

B.3 Traditional graph learning

GL aims to embed the graph as a low-dimensional representation while preserving the desired properties of the original graph data. Classical GL methods are usually implemented using statistical methods or artificially designed components.

Dimension reduction As a commonly used method in feature engineering, dimension reduction aims to reduce the dimension of high-dimensional attribute graph data into a lower-dimensional representation. In GL, it highlights the remaining information at the cost of losing part of the attributes. According to different dimensionality reduction strategies, such methods can be classified into two types. The first type is subspace learning under the linear assumption. Based on the assumption that the principal components [370] related to the larger variance represent important structural information, and those smaller variances represent noise, principal component analysis calculates a low-dimensional representation that maximizes the variance of the data. Linear Discriminant Analysis (LDA) [371] achieves dimension reduction by maximizing the ratio of inter-class scattering and intra-class scattering to obtain a linear projection matrix. Multi-Dimensional Scaling (MDS) [372] is a distance-maintaining manifold learning method. It produces a mapping in a lower dimension to preserve dissimilarities between nodes as much as possible. The second type is nonlinear dimension reduction, which aims to automatically learn nonlinear topology to achieve manifold learning. Isomap [373] first constructs a neighborhood graph on the manifold and calculates the shortest path between pairs of nodes, and then uses MDS to construct a low-dimensional embedding. Locally Linear Embedding (LLE) [374] first allocates neighbors for each node. Then, it calculates the weighted W_{ij} , the best linear reconstruction feature X_i from its neighbors. Finally, calculate the low-dimensional embedding for the optimal reconstruction of W_{ij} .

Matrix factorization Greatly influenced by the idea of dimension reduction, the models based on matrix factorization emerged in the early research of GL. Such models aim to reconstruct the adjacency matrix of the graph to achieve dimension reduction while maintaining structural information. Although these models have significant limitations, in fact, their ideas still inspire many current studies. Depending on how the matrix is constructed, such methods often append specific constraints. Graph Laplacian eigenmaps [375] minimizes a loss function to ensure that nodes close to each other on the manifold are mapped into the low-dimensional space and still maintain the local distances. Node proximity matrix factorization [376] minimizes the objective function $|W - YY^c|$ through matrix factorization to approximate the proximity of nodes in the low-dimensional space, where Y and Y^c are the embeddings for nodes and context nodes, and W is the default node proximity matrix. GraRep [377] aims to preserve the high-order proximity of graphs in the embedding space, thus it derives a k -th order transition matrix, A^k , by multiplying the adjacency matrix to itself k times. The transition probability from node v_i to node v_j is the entry in the i -th row and j -th column of the k -th order transition matrix, i.e., $p_k(v_i|v_j) = A^k_{ij}$. Then GraRep defines the loss function using the skip-gram model and negative sampling. To capture the high-order proximity between node pairs, HOPE [378] preserves asymmetric transitivity in approximating the high-order proximity. Specifically, the goal of HOPE is to minimize the objective function $\|S - WC^T\|_F^2$, where the elements $s_{ij} \in S$ represent a certain edge feature (e.g., Katz index, the Rooted Page-Rank, the Common Neighbors, and the Adamic-Adar) between the corresponding node pairs (v_i, v_j) , W is the node representation matrix, and C is the embedding of the node as the context. To reconstruct the matrix S more simply and elegantly, HOPE proposes to obtain W and C directly based on the low-rank singular value decomposition (SVD).

Graph kernel The kernel method is an important algorithm in pattern recognition and machine learning. Its basic idea is to give the graph embedding $x \in X$ in the original low-dimensional space X , and maps the embeddings to a high-dimensional feature space H through a nonlinear function f^{ker} . Then the nonlinear problem in X can be solved by constructing a linear algorithm in H . There are two main types of kernel methods on graph data. The first type uses the embedding method to convert the graph data into vectorial representation, and then directly implements the application based on the kernel function. However, due to the loss of mass graph structure information when transforming graphs into vectorial representation, such methods do not perform well in real scenarios. The second type of method introduces

the graph kernel function to solve this problem. Based on retaining the advantages of the original kernel function, it directly represents the structural information of the graph data in the high-dimensional Hilbert space. The definition of the traditional method of graph kernel comes from R-convolution. According to the difference between the contrast substructure and the decomposition method of the graph structure, a large number of methods based on graph kernel have been proposed. For example, the work of [379, 380] proposed a random-walk kernel based on calculating the number of common synchronization between two graph structures. To reduce the computational complexity and optimize the random walk strategy, a graph kernel based on comparing the shortest path information between two graph structures is proposed. To capture more complex topological information, the Weisfeiler-Lehman subtree graph kernel is proposed, which is based on a one-dimensional Weisfeiler-Lehman isomorphism test algorithm to find isomorphic subtree structures in a bunch of graph structures [381].

C PFM's theory

Since pretraining has received great attention from the research community, the investigation in the theory-backed explanation is similarly eye-catching. During the unsupervised pretraining era before SSL, Erhan et al. [382, 383] shed some light on the theoretical explanation for the confirmation and clarity of learning difficulties. [382] researches the influence of pretraining with respect to architecture depth, model capacity, and the number of training samples, and demonstrates the robustness of pretraining from the perspective of both the optimization and the regularization. [383] further prove the regularizer role of the unsupervised pretraining in the downstream supervised tasks.

C.1 Different perspectives

Pretext tasks. [384] posits a mechanism based on approximate conditional independence (CI) to connect pretext and downstream task data distributions, which suggests that pretext tasks can self-supervisedly learn the representations from unlabelled data that reduce the sample complexity of downstream supervised tasks. The experiments both on CV and NLP tasks support this theory. Representation Learning via Invariant Causal Mechanisms (ReLIC) [181] also provides a theoretical understanding from the perspective that the explicit invariance constraints across augmentations can yield improved generalization guarantees.

Multi-view redundancy. From the perspective of a multi-view setting, [385] understands contrastive learning as exploiting multiple views of data for representation learning.

This theory provides a theoretical analysis that the linear functions of these representations from pretraining are still competitive compared with the non-linear optimal predictor of the label. In other words, the linear functions of the learned representations are nearly optimal on downstream prediction tasks whenever the different views provide redundant information about the label.

C.2 Different categories

Contrastive learning. Although experimental results show us that previous designs such as contrastive loss or momentum updating can produce impressive performance in SSL. However, one of the most important questions that remain in SSL is why these methods can maintain representation consistency during the pretraining process. A naive view is the minimization between positive pairs can boost invariance learning, while the maximization between negative pairs contributes to avoiding representational collapse. [386] shows that contrastive learning can achieve competitive bound via intra-class concentration, thus leading to the reduction of sample complexity on downstream tasks from the benefit of transferred representations. This research also provides a framework that can be utilized both on the guarantees of the quality of learning representations during the pretraining phase and the future assumptions added to the framework that allow tighter guarantees.

Non-contrastive learning. While contrastive learning shows an effect by capturing the similarity and dissimilarity among the unlabelled examples, and further converging to an average local optimum which represents the general representations, recent non-contrastive SSL methods such as BYOL and SimSiam also shows the SOTA performance without the design of comparison between negative pairs. Based on the analysis of the eigenspaces, Tian et al. [182] study the behavior of non-contrastive SSL training and prove that the effects are from both the predictor and stop-gradient signal. Based on this theory, a novel and simple **Direct-Pred** method is proposed as a by-product of this theoretical exploration.

D Pretext task taxonomy on CV

Pretext tasks are always designed to use pseudo labels generated from the data itself to pretrain the proxy model. There are five categories of pretext tasks for self-supervised: (1) generation-based methods; (2) transformation-based methods; (3) context-based methods; (4) semantic-based methods; (5) view-based methods.

Generation-based methods This type of method is GAN-based in the deep learning era. For image generation, there are several applications including image colorization [138,

387], image super-resolution [388], image editing [389], context encoders [137], image-to-image translation [352], etc. On the other hand, video generation tasks contains future prediction [145], video action recognition [241], video generation [390, 391], and video representation [392].

Transformation-based methods Transformation is a typical technology that serves as a data augmentation method to enlarge the training dataset in traditional DL. However, if transformations of the same image are labeled as positive samples and others as negative samples, this pretext task can be used for self-supervised pretraining [166]. Popular transformation in self-supervised learning (SSL) contains color transformation (such as Jitter, Gaussian blur, and adjusting brightness) and geometric transformation (such as flipping, cropping, scaling, and rotation).

Context-based methods Basically, the design and construction of many artificial tasks, such as solving Jigsaw puzzles [140], comparing context similarity, and discriminating sequence order. Solving Jigsaw puzzles is defined as identifying the correct position of patches from an image. This task can help the model to learn an encoder for transfer learning [141, 393], and the feature representations are effective after the pretrained dataset is big enough. In addition, the design of video Jigsaw is also proposed for unsupervised learning [394]. Differently, context similarity tries to label the patches from the same images as positive samples and others as negative samples, then use a predefined similarity function to scale the distance between different pairs [49].

Semantic-based methods Semantic-based methods contain object detection, semantic segmentation, and depth prediction. These tasks also involve pretext tasks because their pixel-based labels can learn a more robust feature representation than simpler tasks. These pre-text tasks always establish on video dataset [395, 396].

View-based methods This type of method contains both single-modal data and multi-modal data. For the single-modal data, the original data is treated as the anchor and different viewpoints generate its positive pair samples. Sometimes the time slices in sequence-based data are treated as negative pairs because the scene is changed as time goes [397]. In addition, multi-modal data is usual in view-based methods, which are also called cross-modal-based methods here. Such as audio-video cooperative learning [398], RGB and optical flow cross-modal distance training [250].

E Evaluation metrics

Classification task The classification task, according to a labeled training document, determines the relationship between document features and document categories. The

learned relationship model is then used to determine the category of new documents.

Accuracy and error rate The key metrics for a text classification model are Accuracy and Error Rate. The terms Accuracy and Error Rate are defined as follows:

$$Accuracy = \frac{(TP + TN)}{N}, \quad (20)$$

$$ErrorRate = 1 - Accuracy = \frac{(FP + FN)}{N}, \quad (21)$$

where TP and FP denote true positive and false positive, TN and FN stand for true negative and false negative.

Precision, Recall and F1 Regardless of the standard type and error rate, there are very important metrics used for unbalanced testing sets. These metrics are similar to the concept of the class label in the testing samples. F1 is defined as the harmonic average of Precision and Recall. Thus, Accuracy, Recall, and F1 can be represented as:

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}, \quad (22)$$

$$F1 = \frac{2Precision \times Recall}{Precision + Recall}. \quad (23)$$

When the accuracy, F1, and recall values hit 1, the desired results are obtained. On the other hand, when the values turn 0, we get the worst consequence. For the multi-class classification task, the precision and recall values of each class can be determined independently, and then the individual and overall performance can be analyzed.

Micro-F1 The *Micro-F1* [399] is a metric that measures all labels' overall accuracy and recall. We denote *Micro-F1* as:

$$Micro-F1 = \frac{2P_t \times R_t}{P + R}, \quad (24)$$

$$P = \frac{\sum_{t \in S} TP_t}{\sum_{t \in S} TP_t + FP_t}, \quad R = \frac{\sum_{t \in S} TP_t}{\sum_{t \in S} TP_t + FN_t}. \quad (25)$$

where TP_t and FP_t mean true and false positive of the t th label on a text.

Macro-F1 The *Macro-F1* calculates the average F1 of all labels by giving equal weight to them. *Macro-F1* is denoted as:

$$Macro-F1 = \frac{1}{S} \sum_{t \in S} \frac{2P_t \times R_t}{P_t + R_t}, \quad (26)$$

$$P_t = \frac{TP_t}{TP_t + FP_t}, \quad R_t = \frac{TP_t}{TP_t + FN_t}. \quad (27)$$

where TN_i and FN_i represent true and false negative of the i th label. $\mathcal{S}$ stands for the label set of all samples.

Mean reciprocal rank (MRR) The MRR is commonly used to evaluate the performance of ranking algorithms on Question Answering (QA) and Information Retrieval (IR) tasks. MRR is represented as

$$\text{MRR} = \frac{1}{Q} \sum_{i=1}^Q \frac{1}{\text{rank}_i}, \quad (28)$$

where rank_i is the ranking of the i th ground-truth answer. The number of predicted labels on each text is denoted by Q . Moreover, there are some metrics, such as EM, Hamming-loss [400], P@K and NDCG@K.

Generation task Generation task uses LMs to predict the next most likely word or sentence based on input data.

Bilingual evaluation understudy (BLEU) BLEU compares the generated sentences to the reference sentence and makes predictions using automatic machine translation algorithms. The language creation problem is also supported by deep learning technologies such as speech recognition, image caption generation, and text summarization. They can't discover anything better, but it has a few advantages: it's simple to comprehend, correlates well with human judgment, and is language-independent. As a bilingual evaluation aid, BLEU is mainly used to evaluate the quality of machine translation [401]. BLEU compares the degree of overlap between the N-gram in the candidate text and the N-gram in the reference text. The higher overlap indicates better translation quality. The formula for the computation is:

$$\text{BLEU} = BP \times \exp \left(\sum_{n=1}^N W_n \log P_n \right), \quad (29)$$

where N represents N-gram, BP is penalty factor, P_N is multivariate precision, and $W_N = 1/N$ is the corresponding weight of multivariate precision. r represents the length of the shortest reference translation, and c represents the length of the candidate translation, then the specific calculation method of penalty factor BP is as follows:

$$BP = \begin{cases} 1, & l_t > l_a \\ e^{1-l_a/l_t}, & l_t \leq l_a \end{cases}, \quad (30)$$

where l_t is the number of words in machine translation and l_a is the number of words in reference answer. The penalty factor is mostly used to penalize large gaps between machine and reference translations.

ROUGE (recall-oriented understudy for gisting evaluation) ROUGE stands for N-gram co-occurrence statistics, which are used in automatic evaluation methods. It is expanded on the similarity of N-grams, which means that

an N-gram is a subsequence of the main document text in terms of N words. There are four types of ROUGE, including ROUGE-N, ROUGE-L, ROUGE-W, and ROUGE-S. The first two are commonly used, and the N in rouge-N refers to N-gram, which is calculated similarly to BLEU, except BLEU is based on accuracy, while ROUGE is based on recall. L in ROUGE-L refers to the Longest Common Subsequence, which is calculated as the Longest Common Subsequence between the candidate abstract and the reference abstract. Thus, the longer the length, the higher the score, based on the F value. The calculation formula of ROUGE-N and ROUGE-L is mainly introduced. The calculation formula of ROUGE-N is as follows:

$$\begin{aligned} \text{ROUGE-N} &= \frac{\sum_{S \in \{\text{ReferenceSummaries}\}} \sum_{\text{gram}_n \in S} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{S \in \{\text{ReferenceSummaries}\}} \sum_{\text{gram}_n \in S} \text{Count}(\text{gram}_n)}, \end{aligned} \quad (31)$$

where N stands for N-gram, $\text{Count}(\text{gram}_n)$ represents the frequency of occurrence of an N-gram, and $\text{Count}_{\text{match}}(\text{gram}_n)$ represents the frequency of co-occurrence of an N-gram. The calculation formula of ROUGE-L is as follows:

$$\text{ROUGE-L} = F_{\text{lcs}} = \frac{(1 + \beta^2) R_{\text{lcs}} P_{\text{lcs}}}{R_{\text{lcs}} + \beta^2 P_{\text{lcs}}}, \quad (32)$$

$$R_{\text{lcs}} = \frac{\text{LCS}(X, Y)}{M}, \quad (33)$$

$$P_{\text{lcs}} = \frac{\text{LCS}(X, Y)}{N}, \quad (34)$$

where X is the candidate abstract, Y represents the reference abstract, $\text{LCS}(X, Y)$ table indicates the length of the Longest Common Subsequence (LCS) of the candidate abstract and references abstract, M stands for the length of reference abstract, and N denotes the length of the candidate abstract. The ROUGE method is characterized by N-gram co-occurrence statistics, based on recall rate (ROUGE-N) and F-value (ROUGE-L). They are often used in text summaries. It is worth noting that ROUGE is word-based correspondence rather than semantic-based correspondence, but this can be mitigated by increasing the number of reference summaries.

METEOR METEOR, also known as an explicitly sorted translation evaluation metric [402], is an improved version of the BLEU standard that aims to address some flaws in the BLEU standard. Using WordNet to calculate matching relationships between specific sequences, synonyms, roots, affixes, and definitions improves BLEU performance and makes it more relevant to manual discrimination. The calculation formula is as follows:

$$METEOR = (1 - Pen) \times F_m, \quad (35)$$

$$F_m = \frac{PR}{\alpha P + (1 - \alpha)R}, \quad (36)$$

$$P = \frac{m}{\sum_k h_k(c_i)}, \quad (37)$$

$$R = \frac{m}{\sum_k h_k(s_{ij})}, \quad (38)$$

where $Pen = \gamma(\frac{ch}{m})^\theta$ is a penalty factor, which punishes the word order in candidate translation that is different from that in reference translation. ch refers to the number of chunks, which are clustered units of matched units adjacent to each other in both the candidate translation and the candidate reference translation. α, β, θ is the adjustable parameter, m is the number of unary groups that can be matched in the candidate translation, c is the length of the candidate translation, $h_k(c_i)$ is the number of occurrences in candidate translations c_i , and $h_k(s_{ij})$ is the number of occurrences in reference translations s_{ij} .

Perplexity Perplexity is also called the degree of confusion [403]. Its core idea is: first, according to the testing sentence, learn an LM P . Then, according to the LM P , the score of the optional sentence is calculated. Finally, the above scores are standardized according to sentence length. The calculation formula is as follows:

$$PPL(W) = P(w_1, w_2, \dots, w_M)^{-\frac{1}{M}}, \quad (39)$$

where W is the candidate translation, M is the length of the candidate translation, P is the LM obtained according to the reference translation, and $P(w_1, w_2, \dots, w_M)$ is the score calculated by the LM for the candidate translation. The Perplexity assessment indicator is based on a LM. The lower the degree of confusion, the better the translation quality, which is often used in machine translation and LMs. Its disadvantages are as follows: the larger the data set is, the faster the degree of confusion decreases; the punctuation in the data will impact the PPL of the model; and the interference of common words.

F Datasets

F.1 Downstream tasks and datasets on NLP

There are many available data sets in the NLP domain, divided according to different tasks. It mainly comprises two categories: the task of classification of texts and the task of generating texts. The text classification tasks mainly include

Sentiment Analysis (SA), News Classification (NC), Topic Labelling (TL), Natural Language Inference (NLI), Named Entity Recognition (NER), Question Answering (QA), Dialogue Act Classification (DAC), etc. The generation tasks mainly include text summaries and machine translation (Table 5).

Sentiment analysis (SA)

It consists of judging the emotional polarity and dividing it into several classes. Depending on the granularity of sentiments, the SA is divided into three categories: dichotomy (positive and negative), trichotomy (positive, negative, and neutral), and multiple categories. Here we introduce several datasets in detail.

Stanford sentiment treebank (SST) [474] The dataset is an extension of MR [475]. SST-1 is a version of SST. It is divided into five categories and the number of training texts and testing texts is 8544 and 2210, respectively. It also consists of 20 average tokens. The SST-2 [409] contains 9613 movie reviews including 6920 training texts, 872 development texts, and 1821 testing texts.

Semantic textual similarity benchmark (STS-B) [476] It is used in semantic textual similarity tasks organized in the SemEval context between 2012 and 2017 [477]. It consists of text from image titles, news titles and forums. On a scale of 1 to 5, STS-B displays the semantic similarity of two sentences. It includes 5749 training sets, 1379 development sets, and 1377 testing sets.

Multi-perspective question answering (MPQA) [478, 479] This is an opinion dataset which has two categories. It contains 10,606 sentences from various news sources that have been manually annotated for opinions and other private states. It is worth noting that there are 3311 positive articles and 7293 negative articles, having no labels for each article.

IMDB reviews [480] The dataset is the world's most authoritative source for binary sentiment classification of film reviews. The number of content in each class is the same and it can be divided into training and testing sets whose number of comments is 25,000 on average.

News classification (NC)

As one of the most vital information sources, news content exerts a critical effect on people. The NC facilitates users to acquire essential knowledge in real time. Its applications mainly include news topic identification and recommendation of relevant news based on user interests. Here we introduce several datasets in detail.

20 Newsgroups (20NG) [481] 20NG is a text dataset derived from newsgroups. There are 20 classes with the same number of articles per class, including 18,846 articles in total. The average number of tokens is 221.

AG news [424, 482] This is an academic news search engine, which is divided into four categories. It contains

Table 5 The statistics of the datasets on NLP. For the QA task, the class represents the sum number of candidate answers and the correct answer. For dialogue, class is the number of slots. Length means the average tokens in turn

Type	Task	Datasets	Class	Length	Number	Related papers
Classification	Sentiment analysis	MR	2	20	10,662	[404–408]
		SST-1	5	18	11,855	[404, 409–412]
		SST-2	2	19	9613	[13, 404, 409, 413, 414]
		MPQA	2	3	10,606	[404, 415, 416]
		IMDB	2	294	50,000	[14, 413, 414, 417–419]
	News Classification	20NG	20	221	18,846	[307, 407, 420–423]
		AG News	4	45/7	127,600	[14, 406, 424–426]
		R8	8	66	7674	[407, 423, 427]
		R52	52	70	9100	[407, 423, 427]
	Topic Labeling	DBPedia	14	55	630,000	[419, 424, 425, 428]
		Ohsumed	23	136	7400	[407, 423, 427]
		YahooA	10	112	1,460,000	[424, 429]
	Natural Language Inference	SNLI	3	–	570,152	[13, 55, 303, 430–432]
		MNLI	3	–	433,000	[13, 14, 36, 55, 433]
		QNLI	2	–	115,667	[13, 14, 36]
		WNLI	2	–	852	[36, 432]
		RTE	2	–	5768	[36]
		SICK	3	–	10,000	[434]
		MSRP	2	–	5801	[435]
		CoNLL 2003	4	–	2302	[13, 303, 436–439]
	Named Entity Recognition	OntoNotes 4.0	18	–	–	[440, 441]
		OntoNotes 5.0	18	–	2,945,000	[13, 436, 437, 439]
		MSRA	3	–	–	[13, 439–441]
		ACE 2004	7	–	443	[439, 442–445]
		ACE 2005	7	–	437	[439, 442–444, 446]
		KBP2017	–	–	–	[439, 446]
		QQP	2	–	799,266	[13, 36]
	Question Answering	MRPC	2	–	–	[36]
		SQuAD	–	5000	5570	[36, 55, 303]
		RACE	5	–	100,000	[14, 36, 432, 447]
		TREC	6	10	6400	[307, 405, 406, 413, 426, 428]
		WikiQA	–	873	243	[448, 449]
		DSTC 4	89	–	30,000	[450, 451]
		MRDA	5	–	62,000	[450, 452]
Generation	Dialog Act Classification	SwDA	43	–	1,022,000	[450, 453, 454]
		NYT	–	–	109,910	[455, 456]
		CNN	–	760	92,579	[457–461]
		Dailymail	–	653	219,506	[455, 458, 460, 462, 463]
	Text Summarization	Gigaword	–	–	3,991,000	[458, 464]
		WMT14	–	–	–	[465, 466]
		WMT16	–	–	–	[466, 467]
		WMT17	–	–	–	[465, 467–470]
		WMT18	–	–	–	[467–469]
	Machine Translation	DSTC2	–	–	3000	[471]
		MWOZ	35	15.03	10,438	[471–473]
		GSIM	–	–	3008	[471]
		OOS	151	–	23,700	[471]
	Dialogue					

news headlines and introductions. It includes 120,000 training texts and 7600 testing texts. The number of average tokens is 45/7.

R8 and R52 [483] They come from Reuters [484]. R8 contains 8 classes consisting of 66 average tokens and includes 2189 and 5485 testing and training courses. There are 52 classes in R52, which consists of 70 average tokens. It is divided into 6532 and 2568 training and testing texts.

Topic labeling (TL)

The task mainly obtains the meaning of the file by defining complex file themes. It is a critical component of topic analysis technology, which aims at simplifying topic analysis by assigning each article to one or more topics. Here, we introduce a few in detail.

DBpedia [485] It is a large-scale multilingual knowledge base generated by Wikipedia's most commonly used information boxes. It releases DBpedia every month, adding or removing classes and attributes in each version. The most popular version of DBpedia has 14 categories, separated into 560,000 training data and 70,000 testing data. The number of average tokens is 55.

Ohsumed [486] This is a biomedical literature database. The number of texts is 7400. It has 23 cardiovascular disease categories and consists of 136 average tokens. All texts are medical abstracts that are categorized into one or more classes.

Yahoo answers (YahooA) [424] The dataset is a topic labeling task having 10 categories. The number of average tokens is 136. There are 140,000 training data and 5000 testing data. Each text in YahooA has question titles, question contexts, and best answers.

Natural language inference (NLI)

This task is used to forecast whether the meaning of a text can be inferred from another. Interpretation is a broad form of NLI. By comparing the semantic similarity of sentence pairings, it determines whether a sentence is the interpretation of another one. Here we introduce several primary datasets in detail.

The Stanford natural language inference (SNLI) [430] It is commonly used in NLI takes. It contains 570,152 human-annotated sentence pairs, which are annotated with three sorts of relationships: neutral, derived, and conflicting. Multi-genre Natural Language Inference (MNLI) [487] has 3 categories and consists of 430,000 sentence pairs annotated with textual information, which is usually used in textual inference tasks. Question Natural Language Inference (QNLI) [488], whose task with 2 classes is to determine whether a given text pair is a question-answer. Winograd Natural Language Inference (WNLI) [489] which consists of 2 categories is a data set that captures the standard reference information between two paragraphs.

Microsoft research paraphrase (MSRP) [435] The dataset contains sentence pairs for the text-similarity task, including 1725 training and 4076 testing sets. A binary label annotates each pair, discriminating whether they are paraphrases.

Sentences involving compositional knowledge (SICK) [434] It includes nearly 10,000 English sentence pairs, marked with similarity, and the scale range is 1–5. It has neutral, entailment, and contradictory three categories.

Named entity recognition (NER)

This is a fundamental task of NLP to identify people, places, organizations, and other entities in text. It is a crucial primary tool for many NLP tasks, including information extraction, question answering, semantic parsing, machine translation, etc.

CoNLL 2003 [303] It consists of newswire text from the Reuters RCV1 corpus. It contains four different entity types (Location, Organization, Person, and Miscellaneous) and includes 1393 English news articles, and 909 German news articles.

OntoNotes 5.0 [13] The dataset consists of 174,5K English, 900K Chinese, and 300K Arabic text data. It comes from telephone conversations, news agencies, radio news, radio conversations, and blogs. It has 18 entity classes containing 11 types, seven values, and 2,945,000 text data.

MSRA [440] This is a Chinese dataset that is obtained from the news domain. It has three types of entities and is used as a shared task on SIGNAN back in 2006.

Question answering (QA)

There are two types of QA systems: the extraction guidance system and the generation guidance system. The extractive QA can be regarded as a particular case of text classification. Here we detail several datasets.

Microsoft research paraphrase corpus (MRPC) [490] It contains 5800 sentence pairs extracted from Internet news, and the task type is similar to the QQP dataset. Sentence pairs are derived from comments on the same news item and determine whether the two sentences are semantically the same. The assessment criteria were classification accuracy and F1 score.

Stanford question answering dataset (SQuAD) [303] This is a large-scale machine-reading comprehension dataset that contains two tasks. SQuAD 1.1 [488] provides questions and corresponding answers, and the data set contains 100,000 samples in total, while SQuAD 2.0 [491] adds unanswered questions and expands the scale to 150,000.

RACE [492] The dataset has 5 categories, containing nearly 100,000 questions extracted from middle and high school English tests, with corresponding answers given by experts. The average length of RACE text is more significant than 300, which is longer than other reading comprehension data sets (such as SQuAD) sequences.

Dialog act classification (DAC)

The dialogue act is a specific verbal component, which marks the dialogue according to the meaning category of the dialogue. DAC categorizes tags according to the meaning of the dialogue to help understand the speaker's intentions.

Dialog state tracking challenge 4 (DSTC 4) [451] It belongs to the dialog act classification task and mainly focuses on dialog state tracking on human-human dialogs. It is divided into 89 training classes and contains 24,000 training texts and 6000 test texts.

ICSI meeting recorder dialog act (MRDA) [452] It includes about 75 h of speech from 75 naturally occurring meetings among 53 speakers. The number of categories is 5, and it contains 51,000 training texts, 11,000 test texts, and 11,000 validation texts.

Switchboard dialog act (SwDA) [493] The dataset extends the dialogue behavior label with rounds/discourses. The label summarizes the sentence structure, and relevant and pragmatic information of the relevant turn. The SwDA is split into 43 training classes and includes 1,003,000 training texts, 19,000 test texts, and 112,000 validation texts.

Text summarization

Text summarization is a summary of a given single or multiple documents. It is kept as concise as possible while ensuring that it reflects the critical content of the original document. It can be divided into extractive summarization and generative summarization. Extractive summarization is generated by extracting and splicing the critical sentences in documents. Generative summarization is generated by a model, which summarizes documents according to the required content expressed in documents.

NYT [455] The dataset comes from the corpus annotated by the New York Time. The named entities are annotated using the Stanford NER tool in conjunction with the Free-base knowledge base. It contains 9076 articles, with the remaining 100,834 divided into a training set (96,834 examples) and a validation set (4000 samples).

CNN/daily mail [457] It is used for the passage-based question-answering task, and it is popular in assessing ATS systems. The dataset consists of CNN/Daily Mail news stories paired with multi-sentence human-generated summaries. There are 287,226 training instances, 13,368 validation instances, and 11,490 testing instances in total.

Gigaword [464] This is a data set of English news chapters consisting of nearly 950 pieces. Headlines – stories from multiple sources, including the New York Times – include some articles with a one-sentence, short news feed.

Machine translation (MT)

It refers to the task of translation from one language to another with its semantic equivalence by a computer. There are three categories, rule-based machine translation,

statistics-based machine translation, and neural network-based machine translation.

WMT14 [465] It is a grouping of datasets used in the Ninth Workshop on Statistical Machine Translation shared tasks, including a news translation task, a quality estimation task, a metrics task, and a medical text translation task.

WMT16 [466] This dataset is a grouping of datasets used in the First Conference on Machine Translation shared tasks. It has ten shared tasks, including a news translation task, an IT domain translation task, a biomedical translation task, an automatic post-editing task, a metrics task, a quality estimation task, a tuning task, a pronoun translation task, a bilingual document alignment task, and a multimodal translation task.

WMT17 [465] The dataset includes three MT tasks (news, biomedical, and multimodal), an automatic post-editing task, a quality estimation task, a task dedicated to the training of neural MT systems, a task on bandit learning for MT, an automatic post-editing task, and a metrics task.

WMT18 [468] It mainly features six shared tasks: a news translation task, a biomedical translation task, an automatic post-editing task, a metrics task, a quality estimation task, and a multimodal translation task. Participants must evaluate their approaches to the machine translation topic using the standard data sets created for the shared tasks.

Dialogue

As an essential way of man–machine interaction, the dialogue system offers a wide range of applications. The existing dialogue systems can be grouped into task-oriented dialogue systems and non-task-oriented dialogue systems from application scenarios. Among them, the non-task type of conversation system can also be called a chatbot.

DSTC2 [471] This is a multi-round dialogue data set of restaurant reservation fields, including 1612 training data, 506 verification data, and 1117 test data. It allows the user's goals to change compared to DSTC1. DSTC2 is also richer in terms of the conversation state representation, including the slot value pairs of the user's targets and the ways to find them.

MWOZ [471] It contains 8420/1000/1000 conversations for training, validation, and test sets, respectively. It contains 30 pairs in seven domains being a multi-domain fully-labeled corpus. Every sample includes a goal, multiple user and agent utterances, and annotations regarding slot values.

Out-of-scope (OOS) [471] The dataset includes 15,100 training, 3100 validation, and 5500 test sets, respectively. It contains 151 intent classes, containing 150 in-scope and one out-of-scope intent. The out-of-scope intent indicates that a user utterance failed to classify to given predefined objectives.

Table 6 The statistics of the datasets used on downstream tasks

Type	Name	Usage	Domain	Class	Size	Related papers
Classification	ImageNet	Pretrain &	–	1000+	1,200,000+	[136–143, 145, 146, 151, 156, 173, 174, 179, 182, 183, 494] [48, 49, 161–167, 170, 172, 175–177, 180, 181, 495, 496]
		Downstream				
	CIFAR-10	Downstream	–	10	60,000	[134, 135, 138, 165, 166, 172, 173, 175, 182]
	CIFAR-100	Downstream	–	100	60,000	[165, 166, 173, 175]
	STL-10	Downstream	–	10	6000	[134, 135, 173, 177, 179, 182]
	Caltech-101	Downstream	Object	101	9146	[134, 135, 165, 166]
	MNIST-10	Downstream	Digit	10	60,000	[48, 179]
	SVHN	Downstream	digit	10	73,257	[175]
	Places205	Downstream	Scene	205	2,448,873	[49, 138, 139, 142, 161, 162, 167, 172–175, 494, 496]
	SUN397	Downstream	Scene	899	130,519	[166]
	HMDB51	Downstream	Action	51	7000	[177]
	UCF101	Downstream	Action	101	–	[177]
	Food-101	Downstream	Food	101	101,000	[165, 166]
	Birdsnap	Downstream	Bird	500	49,829	[166]
	Cars	Downstream	Car	196	16,185	[165, 166]
	Aircraft	Downstream	Aircraft	102	10,200	[165, 166]
	Pets	Downstream	Pet	37	7400	[165, 166]
	Flowers	Downstream	Flower	102	8189	[165, 166]
	DTD	Downstream	Texture	47	5640	[165, 166]
	iNaturallist2018	Downstream	Species	8000+	450,000+	[162, 167, 174, 496]
	JFT-300 M	Pretrain	–	3000+	300,000,000+	[40, 496]
Detection	COCO	Downstream	object	80	200,000	[142, 163, 164, 167, 170, 179, 183, 496]
	VOC07	Downstream	Object	20	9963	[49, 137, 138, 138–143, 146, 161–165, 167, 170, 175, 494]
Segmentation	VOC12	Downstream	Object	20	2913	[49, 137–142]
	NYU-Depth V2	Downstream	scene	894	1,449	[139, 165, 177]
	VOC11	Downstream	Object	20	3334	[136]
	ADE20K	Downstream	Scene	3688	27,574	[183, 496]
	Cityscapes	Downstream	Scene	25	25,000+	[163]
	LVIS	Downstream	Vocabulary	1200+	160,000+	[163]
	DAVIS	Downstream	Scene	150	–	[151]
Inpainting	Paris StreetView	Downstream	Scene	–	15,000	[136, 137]
Sequence	Moving-MNIST	Downstream	Digit	–	10,000	[179]
–	YFCC100M	Pretrain	Multimedia	–	100,000,000+	[49]

Downstream tasks and datasets on CV

The datasets in CV mainly contain three types from the perspective of tasks: classification, detection, and segmentation. The popular datasets are concluded in Table 6, and some infrequently mentioned datasets in long tails are discussed in the text.

Classification

In this part, we first cover the popular large-scale datasets used frequently in both the pretext and downstream

tasks. Then the domain datasets only used for the downstream tasks are unfolded.

MNIST [497] It's a collection of handwritten digits that includes 60, 000 samples in training and 10, 000 in testing. The images are fixed-size with 28×28 pixels. The pixel values are from 0 to 255.0 in which pixel values smaller than 255.0 can be understood as background (white) and 255 means foreground (black). The labels are from 0 to 9 and only one of these digits exists in an image. Both traditional and deep learning methods are based on this most popular dataset despite advanced methods showing

perfect results. Thus, Geoffrey Hinton has described it as “the drosophila of machine learning”.

Street view house numbers (SVHN) [498] In the domain of digit numbers, it collects real-world digit numbers from house numbers in Google Street View images. It includes 73, 257 digits for training, 26, 032 digits for testing, and 531, 131 additional. All of them are 32×32 color images with both class labels and character-level bounding boxes.

CIFAR [499] As more advanced methods show perfect results on the simple datasets, more sophisticated datasets such as CIFAR-10 and CIFAR-100 are conducted. These two datasets are closer to the real-world object. The CIFAR-10 contains 50,000 training images and 10,000 testing images, with 6000 images per class and 32×32 pixels in each RGB color image. The CIFAR-100 is similar to the CIFAR-10 but with more detailed label information. There are 100 classes containing 500 training images and 100 testing images in each class. In addition, these 100 “fine” classes are grouped equally into 20 “coarse” classes. Researchers can adapt it to suitable learning methods.

STL-10 [500] Inspired by the CIFAR-10 dataset, STL-10 is another 96×96 color image dataset containing similar 10 real-world classes. Each class has 500 training images and 800 testing images. The biggest difference is that STL-10 has 100, 000 unlabeled images for unsupervised learning. More construction information can be seen in [501].

Caltech-101 [502] It collects roughly 300×200 color images of objects belonging to 101 categories, with 40–800 images per category and 50 on average. The outlines of the objects in the pictures are annotated for the convenience of different learning methods.

ImageNet [312] This is one of the most popular and large-scale datasets on computer vision. It is built according to the hierarchical structure of WordNet [503]. The full ImageNet dataset contains 14, 197, 122 images and 21, 841 synsets indexed, attaching on average 1000 images to demonstrate each synset. The most frequently-used subset of ImageNet is the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) dataset from 2010 to 2017, containing tasks of classification, localization, and detection. The number of samples in training and testing datasets and the labels of images are determined by the specific task, more details are seen in [504].

HMDB51 [505, 506] In addition to the popular MNIST, there still exist many domain datasets used for the downstream tasks in the classification problem. HMDB51 is an action video database for a total of 7000 clips in 51 action classes. It contains five types of facial actions and body movements.

UCF101 [507] It is another action video data set designed for more realistic action recognition. It is an extension of the UCF50 [508] data set containing only 50 action categories with 101 action categories, collected from YouTube. What

makes it a famous recognition dataset is the workshop in ICCV13 with UCF101 as its main competition benchmark.

Food-101 [509] This is a real-world food dataset of 101 food categories, with 750 and 250 images per class in training and testing dataset respectively.

Birdsnap [510] It is a fine-grained visual categorization of birds on a broad scale, with bounding boxes and the locations/annotations of 17 parts in the object. It contains 49,829 images of the 500 most common species in North America, with each species containing 69–100 images and most species having 100. In addition, some images are also labeled as male or female, immature or adult, and breeding or non-breeding plumage.

SUN397 To target the scene categorization, the extensive Scene UNderstanding (SUN) database [511, 512] fills the gap of the existing dataset with the limited scope of categories. This database has 899 categories and 130,519 images, and only images with more than 200×200 pixels were kept. SUN397 is a more well-sampled subset that maintains 397 categories with at least 100 images per category, in which other categories containing relatively few unique photographs are discarded.

Places205 Places205 [513] dataset is another large scale scene dataset consists of 2,448,873 images from 205 scene categories.

Cars [514] The dataset in the domain of cars contains 16,185 color images of 196 classes (at the level of Make, Model, Year) of cars. For convenience, this dataset is split into training and testing sets in roughly equal quantities.

Aircraft [515] It is another fine-grained visual classification designed for aircraft (also known as FGVC-Aircraft). A popular form of this dataset is the fine-grained recognition challenge 2013 (FGComp2013) [516] ran in parallel with the ILSVRC2013. There exist four-level hierarchies: Model, Variant, Family, Manufacturer, from finer to coarser to organize this database. The more detailed information is shown in [517].

Pets [518] It represents The Oxford-IIIT Pet Dataset that collects 37 pet categories with roughly 200 images per category. All images have an associated ground truth annotation of breed for classification, head ROI for detection, and pixel-level trimap for segmentation.

Flowers [519] Similarly, Flowers is another domain dataset in flowers also collected by Oxford; it contains Oxford-17 Flowers of 17 categories and Oxford-102 Flowers of 102 categories.

Describable textures dataset (DTD) [520] This is an evolving collection of textural images in the wild, which consists of 5640 images of 47 categories, with 120 images per category.

iNaturalist2018 [521] It is a large-scale species classification competition conducted on the FGVC5 workshop at CVPR2018. This dataset contains over 8000 species

categories, with more than 450,000 images in the training and validation dataset collected from iNaturalist [522].

JFT-300 M [523] JFT-300 M is an internal Google dataset introduced by Sun et al [523] and well-known from ViT Model [40]. It is labeled by algorithms that utilize human-computer communications and target classification tasks. This dataset finally contains 300 M images with over 1000 M labels, thus leading to the multiple labels attached to this large-scale dataset.

Detection

The detection is a popular task in the CV, and almost all the research is conducted on COCO and PASCAL VOC datasets.

COCO [339] This is a large-scale data set for object detection, segmentation, and caption; it contains 330,000 RGB images, with more than 200,000 labeled. There are 1.5 million object instances of 80 object categories involved. Thus, it is one of the most popular benchmark data set in detection and segmentation in parallel with the following PASCAL VOC.

PASCAL VOC [524] From 2005 through 2012, the dataset has run challenges assessing performance on object class recognition and has provided standardized image datasets for object class recognition. The main datasets used in self-supervised learning are VOC07, VOC11, and VOC12. Main competitions in VOC07 [525] contain classification and detection tasks; both of them consist of 20 objects and contain at least one object in each image. Thus, it is common to use VOC07 to serve as the downstream task for the detection.

Segmentation

The segmentation is a semantics-based pixel-level classification. These datasets are difficult to obtain and annotate, thus they are always used as a downstream task.

VOC11 [526] & **VOC12** [527] Both VOC11 and VOC12 contains classification, detection, and segmentation tasks in the main competition, thus leading to the common use of downstream task for the segmentation.

ADE20K [528, 529] It collects 27,574 images from both the SUN and Places205 databases, in which 25,574 for training and 2000 for testing. All 707,868 objects from 3688 categories existing in images are annotated. Especially, this dataset contains 193,238 annotated object parts and parts of parts, and additional attributes, annotation time, depth ordering for the benefit of the research community.

NYU-Depth V2 [530] This is a dataset consisting of images and video sequences from 464 indoor scenes that are recorded by both the RGB and Depth cameras from 3 cities. It contains 1449 images with the ground truth of depth, and the original RGB values are also provided. In addition,

there are 407,024 new unlabeled frames and additional class labels for the objects in images.

Cityscapes [531, 532] It is a dataset of urban street scenes from 50 cities with the ground truth of semantic segmentation. The main instances are vehicles, people, and construction. The high-quality dense pixel annotations contain a volume of 5000 images. In addition to the fine annotations, coarser polygonal annotations are provided for a set of 20,000 images. Moreover, the videos consist of not consistent images with high-quality annotations, and these annotated images with consistently changing views are provided for researchers.

LVIS [533] It is a dataset for large vocabulary instance segmentation. It features that (1) a category or word in one image is related to the only segmentation object; (2) more than 1200 categories are extracted from roughly 160,000 images; (3) long tails phenomenon exist in these categories; and (4) more than 2,000,000 high-quality instance segmentation masks.

Densely annotated video segmentation (DAVIS) [534] It is a video dataset designed for the in-depth analysis of the SOTA in video object segmentation, in which DAVIS 2017 [535] contains both semi-supervised (human-guided at the testing time) and unsupervised (human non-guided at test time) video sequences with multiple annotated instances.

Others

There are many datasets designed for special visual tasks such as inpainting. In addition, this part covers the data collection in the wild.

Paris StreetView [536] The dataset is designed for image inpainting task, which contains 14,900 training images and 100 testing images collected from street views of Paris. This dataset is collected from Google Street View and mainly focuses on the buildings in the city.

Moving-MNIST [537] Based on MNIST, it is a video dataset designed for evaluating sequence prediction or reconstruction, which contains 10,000 sequences. Each video is long of 20 frames and consisted of two digits (possibly overlapped) moving inside a 64×64 patch. The first benchmark is reported on [538] by the method of LSTMs.

Yahoo Flickr creative commons 100 million (YFCC100M) [539, 540] The dataset is the largest public multimedia collection that is allowed to search by users for their own targets; this dataset can browse both images and videos. It is free and for researchers to explore and investigate subsets of the YFCC100M in real time. Subsets of the complete dataset can be retrieved by any keyword search and reviewed directly. In addition, the text information attached to any image or video is abundant, such as containing location information and user tags. Briefly, it is more a multimedia library than a domain dataset.

Data in the wild More generalized dataset concept in the self-supervised learning era is composed of multimedia websites, APP, or search engines such as Instagram, Flickr, Google Images, etc. I think pictures in the wild will play a major role in the future study of CV because of the quantity of data, the computation source, and the learning power of PFM.

F.3 Downstream tasks and datasets on graph

The purpose of the pretraining graph model is to improve the performance of downstream tasks. According to the different analysis objects of the downstream tasks, they can be divided into nodes, edges, and graphs. Meanwhile, the PFMs of GL have been widely used in a mass of fields. In this section, we combine the downstream tasks to conduct statistics on the pretraining datasets and the downstream task datasets.

Node-level tasks

Nodes are the most basic element of the graph, so lots of downstream tasks mainly focus on the analysis of nodes.

Node classification Node Classification (NCF) is one of the most prevalent graph-based tasks, which has important analytical value in most of the different types of graph data. Different from the pseudo-labels assigned to nodes in the graph in self-supervised methods, the labels in NCF often come from external information such as manual annotation. Based on Definition 7 and 8, NCF can be divided into two types: transductive and inductive according to the visibility during training, verification, and testing. In addition, the result of NCF can be single-label or multi-label according to the mutual exclusion of labels. The statistical results of common NCF datasets are shown in Table 7.

Node clustering The goal of Node ClusterIng (NCI) is to divide a graph into different classes or clusters according to a certain standard so that the correlation of nodes in the same cluster is as large as possible, and the irrelevance of nodes that are not in the same cluster is also minimized. Although in the above-mentioned pretraining tasks, NCI is used as a pretext task has appeared, NCI still test pretraining graph models based on other pretext tasks.

Top-K search The goal of task Top-K Search (TKS) is to search the K nodes with the highest predefined associations for a given node in the graph. Usually, TKS is used for search tasks such as recommendation and alignment. The detailed statistical results of the datasets are shown in Table 7.

Link-level tasks

The edge is also an important part of the graph structure, which associates independent nodes and is the key to distinguishing graph data from non-relational data. Especially in some specific fields (e.g., molecules, proteins), edges

contain real information, so there are various tasks related to edges.

Link classification Similar to the NCF, the Link Classification (LC) also assigns one or more labels to a given edge. In fact, in LC, the nodes at both ends of the edge are still taken into consideration.

Link prediction Link Prediction (LP) is a common graph task (e.g., knowledge graph). The goal of LP is to predict edges that are removed or may exist in the graph. Similar to NCI, LP is also one of the pretext tasks in self-supervised learning, and its statistic results as shown in Table 8.

Top-K recommendation Top-K Recommendation (TKR) is exactly the same as the definition of TKS, the difference lies in the sorting goal.

Graph-level tasks

The graph-level task generally focuses on the distribution of nodes, edges, and attributes in a given graph, in order to infer the possible properties of the entire graph.

Graph classification Graph Classification (GC) is commonly used in social, molecular, and protein graph data, which aims to predict the property of the given community, chemical compound, and protein. The statistic results as shown in Table 9.

Data source

The PFMs of GL have been widely used in a mass of fields. We will describe the details of the pretraining datasets and the downstream task datasets.

Citation and co-author network A citation is a basic local representation, whose structure reflects the citation relationships of papers in a research direction or field. Specifically, a citation network is a kind of relational data composed of research papers as nodes and citation relations as edges. Among them, the citation network used in the GL model usually comes from local samples of common citation databases, e.g., Cora, Citeseer, and PubMed, and serves as downstream tasks. Similarly, the co-author network is a dataset of scientific collaboration that corresponds to a researcher's ego network, in which the researcher and their collaborators are nodes and an edge indicates collaboration between two researchers. According to different requirements of downstream tasks, such co-author networks can be used for various tasks, e.g., node classification and graph classification.

Molecular and protein network A molecular network usually refers to a compound composed of atoms and atomic bonds, and predicting the properties of the compound is usually regarded as a graph classification task. For example, MUTAG is a collection of nitroaromatic compounds whose goal is to predict their mutagenicity to *Salmonella typhimurium*. PTC uses a graph to show the structure of multiple compounds and aims to predict the carcinogenicity of different compounds in rats. The protein network is a

Table 7 The statistics of the datasets for node-level tasks. Homogeneous:Hom, Heterogeneous:Het

Task	Name	Usage	Source	Type	Nodes	Edges	Class	Features	Related paper
NCF	Academia	pretrain	Citation	Hom	138K	739K	–	–	[195]
	DBLP (SNAP)	pretrain	Citation	Hom	317K	2 M	–	–	[195]
	DBLP (NetRep)	pretrain	Citation	Hom	540K	30 M	–	–	[195]
	IMDB	pretrain	Movie	Hom	896K	8 M	–	–	[195]
	Facebook	pretrain	Social	Hom	3 M	47 M	–	–	[195]
	LiveJournal	pretrain	Social	Hom	4 M	86 M	–	–	[195]
	Cora	Downstream	Citation	Hom	2708	5429	7	1,433	[188, 194, 203, 214, 224] [198, 200–202, 209, 211]
	CiteSeer	Downstream	Citation	Hom	3327	4732	6	3703	[188, 194, 203, 224] [198, 201, 202, 209, 211]
	PubMed	Downstream	Citation	Hom	19K	44K	3	500	[188, 198, 201–203] [194, 200, 209, 211, 224]
	ACM	Downstream	Citation	Hom	8994	26K	4	1902	[215]
	Cora-Full	Downstream	Citation	Hom	20K	63K	70	500	[224, 541]
	Cora-ML	Downstream	Citation	Hom	2995	8158	7	2879	[224]
	Reddit-233K	Downstream	Social	Hom	233K	57 M	210	5414	[189, 201, 202, 214]
	BlogCatalog	Downstream	Social	Hom	10K	334K	39	–	[191, 192]
	YouTube	Downstream	Social	Hom	1 M	3 M	47	–	[191]
	Reddit-231K	Downstream	Social	Hom	231K	11 M	41	602	[200, 211, 542, 543]
	Amazon	Downstream	Social	Het	130 M	–	–	–	[212]
	PPI-30K	Downstream	Protein	Het	3890	77K	50	–	[192, 200]
	PPI-57K	Downstream	Protein	Het	57K	819K	121	50	[202, 211, 224, 542, 543]
	IMDB	Downstream	Movie	Hom	12K	37K	4	1256	[215]
	Four-Univ	Downstream	Movie	Hom	4518	3426	6	2000	[224]
	Chameleon	Downstream	Web	Hom	2277	36K	6	500	[224]
	Crocodile	Downstream	Web	Hom	12K	180K	6	500	[224]
	Flickr-89K	Downstream	Web	Hom	89K	450K	7	500	[202, 224]
	ogbn-arxiv	Downstream	Web	Hom	169K	117K	40	128	[224]
	Wiki-CS	Downstream	Web	Hom	12K	277K	10	300	[224, 541]
	DBLP	Downstream	Web	Hom	17K	53K	4	1639	[224, 543]
	Computers	Downstream	Co-purchase	Hom	14K	246K	10	767	[198, 209, 224, 541]
	Photo	Downstream	Co-purchase	Hom	7650	119K	8	745	[198, 209, 224, 541, 544]
	CS	Downstream	Co-author	Hom	18K	82K	15	500	[198, 209, 224, 541, 544]
	Physics	Downstream	Co-author	Hom	35K	248K	5	500	[198, 224, 541]
	H-index	Downstream	Co-author	Hom	5000	44K	–	–	[195]
	Flickr-81K	Downstream	Photo	Hom	81K	6 M	195	–	[191]
	Wikipedia	Downstream	Word	Hom	4777	185K	40	–	[192]
	US-Airport	Downstream	Airline	Hom	1,190	13K	–	–	[195]
	OAG	Downstream	Academic	Het	178 M	2B	–	–	[212]
NTKS	KDD-ICDM	Downstream	Co-author	Hom	2867/2607	7637/4774	697	–	[195]
	SIGIR-CIKM	Downstream	Co-author	Hom	2851/3548	6354/7076	874	–	[195]
	SIGMOD-ICDE	Downstream	Co-author	Hom	2626/2559	8304/6668	898	–	[195]

collection of proteins classified as either enzymes or non-enzymes. The amino acids are represented by nodes, and two nodes are connected by an edge if they are less than 6 Angstroms apart.

Social and movie network The social network is the social-relational data in the real network environment,

which usually represents the relationship between users or posts. For instance, Reddit is a graph dataset comprised of Reddit posts made in September 2014. BlogCatalog is a graph dataset that represents a network of social relationships between bloggers who are listed on the BlogCatalog website. The movie network is usually composed of actors

Table 8 The statistics of the datasets for LC. Homogeneous:Hom, Heterogeneous:Het

Name	Usage	Source	Type	Nodes	Edges	Class	Features	Related paper
Cora	Downstream	Citation	Hom	2708	5429	7	1433	[188, 189, 194, 203, 214, 224, 542, 543, 545, 546] [198, 200–202, 209–211, 544]
CiteSeer	Downstream	Citation	Hom	3327	4732	6	3703	[188, 189, 194, 200, 203, 224, 542, 543, 546] [198, 201, 202, 209–211, 541, 544]
PubMed	Downstream	Citation	Hom	19K	44K	3	500	[188, 189, 194, 200, 203, 224, 542, 543, 545, 546] [198, 201, 202, 209–211, 544]
ML-100K	Downstream	Movie	Hom	2625	100K	5	–	[545]
ML-1 M	Downstream	Movie	Hom	9940	1 M	5	–	[545]
BlogCatalog-5K	Downstream	Social	Hom	5196	172K	6	8189	[211, 542]
Amazon	Downstream	Social	Het	130 M	–	–	–	[212]
PPI-57K	Downstream	Protein	Het	57K	819K	121	50	[202, 211, 224, 542, 543]
Flickr-7K	Downstream	Photo	Hom	7575	240 M	9	12,047	[211, 542]
Last-FM	Downstream	Music	Hom	15K	73K	122	–	[215]
Book-Crossing	Downstream	Book	Hom	111K	443K	52	–	[215]
OAG	Downstream	Academic	Het	178 M	2B	–	–	[212]

Table 9 The statistics of the datasets for GC. Homogeneous:Hom, Heterogeneous:Het

Name	Usage	Source	Type	Graphs	Nodes	Edges	Class	Related paper
ZINC15	Pretraining	Molecule	Hom	2 M	–	–	–	[190, 204]
ChEMBL	Pretraining	Molecule	Hom	456K	–	–	–	[190, 204]
PPI-pre	Pretraining	Protein	Het	395K	–	–	–	[190]
MUTAG	Downstream	Molecule	Hom	188	–	–	2	[190, 199, 201, 216, 218, 225, 547, 548]
PTC	Downstream	Molecule	Hom	344	–	–	2	[190, 199, 201, 216, 547, 548]
BBBP	Downstream	Molecule	Hom	2039	–	–	2	[190, 204, 218, 220, 225, 549]
Tox21	Downstream	Molecule	Hom	7831	–	–	24	[190, 204, 218, 220, 225, 549]
ToxCast	Downstream	Molecule	Hom	8575	–	–	1234	[190, 204, 218, 220, 225, 549]
SIDER	Downstream	Molecule	Hom	1427	–	–	54	[190, 204, 218, 220, 225, 549]
ClinTox	Downstream	Molecule	Hom	1478	–	–	4	[190, 204, 218, 220, 225, 549]
MUV	Downstream	Molecule	Hom	93K	–	–	34	[190, 218, 220]
HIV	Downstream	Molecule	Hom	41K	–	–	2	[190, 218, 220, 549]
BACE	Downstream	Molecule	Hom	1513	–	–	2	[190, 218, 220, 225, 549]
PPI-88K	Downstream	Protein	Het	88K	–	–	80	[190]
IMDB-M	Downstream	Movie	Hom	1500	19K	99K	3	[195, 201, 216, 545, 547]
IMDB-B	Downstream	Movie	Hom	1000	19K	97K	2	[195, 201, 216, 218, 545, 547]
FreeSolv	Downstream	Molecule	Hom	642	–	–	–	[204]
ESOL	Downstream	Molecule	Hom	1128	–	–	–	[204]
Lipophilicity	Downstream	Molecule	Hom	4200	–	–	–	[204]
QM7	Downstream	Molecule	Hom	6830	–	–	–	[204]
QM8	Downstream	Molecule	Hom	22K	–	–	–	[204]
COLLAB	Downstream	Co-author	Hom	5000	373K	–	3	[195, 218, 547, 548]
RDT-B	Downstream	Co-author	Hom	2000	859K	–	2	[195, 216, 218, 548]
RDT-M	Downstream	Co-author	Hom	5000	3 M	–	5	[195, 216, 218, 548]
NCI1	Downstream	Molecule	Hom	4110	123K	132K	2	[197, 199, 218, 219, 547, 548]
NCI109	Downstream	Molecule	Hom	4127	123K	133K	2	[199]
PROTEINS	Downstream	Molecule	Hom	1113	44K	81K	2	[197, 199, 218, 219, 548]
D & D	Downstream	Molecule	Hom	1178	335K	843K	2	[199, 218]
Mutagenicity	Downstream	Molecule	Hom	4337	131K	134K	2	[219]
METR-LA	Downstream	Traffic	Hom	1	207	–	–	[550]

and their co-occurrence participation in the movie. For example, IMDB-B is a movie collaboration data set that contains a large number of self-networks of actors who play movie roles in IMDB. Nodes in each graph represent actors/actresses, and if they appear in the same film, an edge connects them. These graphs are based on action and romance genres. The difference between IMDB-M and IMDB-B is that a node in the graph represents one or more actors.

Others Some of the rarer graph data are used to test the universality of the PFM, such as word networks (Wikipedia), book networks (Book-crossing), and airline networks (US-Airport). In addition, there are also some special graph structures adapted to specific models, such as spatiotemporal graphs (METR-LA).

Data availability No new data were created or analyzed in this study. Data sharing is not applicable to this article.

References

- Bommasani R, Hudson DA, Adeli E, Altman R, Arora S, Arxiv S, Bernstein MS, Bohg J, Bosselut A, Brunskill E et al (2021) On the opportunities and risks of foundation models. arXiv preprint [arXiv:2108.07258](https://arxiv.org/abs/2108.07258)
- Chowdhury GG (2003) Natural language processing. *Annu Rev Inf Sci Technol* 37:51–89
- Forsyth D, Ponce J (2011) Computer vision: a modern approach. University of Illinois at Urbana-Champaign, USA
- Bondy JA, Murty MSR et al (1976) Graph theory with applications. Macmillan, London
- Qiu X, Sun T, Xu Y, Shao Y, Dai N, Huang X (2020) Pre-trained models for natural language processing: a survey. *Sci China Technol Sci* 63(10):1872–1897
- Li J, Tang T, Zhao WX, Wen J-R (2021) Pretrained language models for text generation: a survey
- Han K, Wang Y, Chen H, Chen X, Guo J, Liu Z, Tang Y, Xiao A, Xu C, Xu Y et al (2020) A survey on visual transformer. arXiv
- Sanchez S, Romero H, Morales A (2020) A review: comparison of performance metrics of pretrained models for object detection using the tensorflow framework. In: IOP conference series: materials science and engineering
- Hu W, Liu B, Gomes J, Zitnik M, Liang P, Pande V, Leskovec J (2019) Pre-training graph neural networks. arXiv
- Zhuang F, Qi Z, Duan K, Xi D, Zhu Y, Zhu H, Xiong H, He Q (2020) A comprehensive survey on transfer learning. In: Proceedings of the IEEE
- Bengio Y, Ducharme R, Vincent P, Janvin C (2000) A neural probabilistic language model. *Adv Neural Inf Proc Syst*
- Mikolov T, Chen K, Corrado G, Dean J (2013) Efficient estimation of word representations in vector space. In: 1st international conference on learning representations, ICLR 2013. Workshop track proceedings, Scottsdale, Arizona, USA, May 2–4, 2013
- Devlin J, Chang M, Lee K, Toutanova K (2019) BERT: pre-training of deep bidirectional transformers for language understanding. In: NAACL-HLT
- Yang Z, Dai Z, Yang Y, Carbonell JG, Salakhutdinov R, Le QV (2019) Xlnet: generalized autoregressive pretraining for language understanding. In: NeurIPS
- Chen M, Tworek J, Jun H, Yuan Q, Pinto HPdO, Kaplan J, Edwards H, Burda Y, Joseph N, Brockman G et al (2021) Evaluating large language models trained on code. arXiv preprint [arXiv:2107.03374](https://arxiv.org/abs/2107.03374)
- Neelakantan A, Xu T, Puri R, Radford A, Han JM, Tworek J, Yuan Q, Tezak N, Kim JW, Hallacy C et al (2022) Text and code embeddings by contrastive pre-training. arXiv preprint [arXiv:2201.10005](https://arxiv.org/abs/2201.10005)
- Christiano PF, Leike J, Brown T, Martic M, Legg S, Amodei D (2017) Deep reinforcement learning from human preferences. *Adv Neural Inf Process Syst*
- Stiennon N, Ouyang L, Wu J, Ziegler D, Lowe R, Voss C, Radford A, Amodei D, Christiano PF (2020) Learning to summarize with human feedback. *Adv Neural Inf Process Syst* 33:3008–3021
- Ouyang L, Wu J, Jiang X, Almeida D, Wainwright CL, Mishkin P, Zhang C, Agarwal S, Slama K, Ray A., et al. (2022) Training language models to follow instructions with human feedback. arXiv preprint [arXiv:2203.02155](https://arxiv.org/abs/2203.02155)
- Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A et al (2020) Language models are few-shot learners. arXiv
- Lester B, Al-Rfou R, Constant N (2021) The power of scale for parameter-efficient prompt tuning. In: Proceedings of the 2021 conference on empirical methods in natural language processing, pp 3045–3059
- Schick T, Schütze H (2021) Exploiting cloze-questions for few-shot text classification and natural language inference. In: Proceedings of the 16th conference of the European chapter of the association for computational linguistics: main volume, pp 255–269
- Zhang Z, Zhang A, Li M, Smola A (2023) Automatic chain of thought prompting in large language models. In: International conference on learning representations
- Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi EH, Le QV, Zhou D et al (2022) Chain-of-thought prompting elicits reasoning in large language models. *Adv Neu Inform Process Syst* 35:24824–24837
- OpenAI (2023) GPT-4 technical report
- Wang P, Yang A, Men R, Lin J, Bai S, Li Z, Ma J, Zhou C, Zhou J, Yang H (2022) Unifying architectures, tasks, and modalities through a simple sequence-to-sequence learning framework. arXiv preprint [arXiv:2202.03052](https://arxiv.org/abs/2202.03052)
- Lu J, Clark C, Zellers R, Mottaghi R, Kembhavi A (2022) Unified-io: A unified model for vision, language, and multi-modal tasks. arXiv preprint [arXiv:2206.08916](https://arxiv.org/abs/2206.08916)
- Singh A, Hu R, Goswami V, Couairon G, Galuba W, Rohrbach M, Kiela D (2022) Flava: a foundational language and vision alignment model. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 15638–15650
- Wang W, Bao H, Dong L, Bjorck J, Peng Z, Liu Q, Aggarwal K, Mohammed OK, Singhal S, Som S et al (2022) Image as a foreign language: Beit pretraining for all vision and vision-language tasks. arXiv preprint [arXiv:2208.10442](https://arxiv.org/abs/2208.10442)
- Clark K, Luong M, Le QV, Manning CD (2020) ELECTRA: pre-training text encoders as discriminators rather than generators. In: ICLR
- Wallace E, Rodriguez P, Feng S, Yamada I, Boyd-Graber J (2019) Trick me if you can: human-in-the-loop generation of adversarial examples for question answering. *Trans Assoc Comput Linguist* 7:387–401
- Nie Y, Williams A, Dinan E, Bansal M, Weston J, Kiela D (2020) Adversarial NLI: a new benchmark for natural language understanding. In: ACL
- Niven T, Kao H (2019) Probing neural network comprehension of natural language arguments. In: ACL

34. Wang G, Ivanov N, Chen B, Wang Q, Yan Q (2023) Graph learning for interactive threat detection in heterogeneous smart home rule data. In: 2023 ACM SIGMOD international conference on management of data. ACM
35. Gordon MA, Duh K, Andrews N (2020) Compressing BERT: studying the effects of weight pruning on transfer learning. In: RepL4NLP@ACL. <https://doi.org/10.18653/v1/2020.repl4nlp-1.18>
36. Lan Z, Chen M, Goodman S, Gimpel K, Sharma P, Soricut R (2020) ALBERT: a lite BERT for self-supervised learning of language representations. In: ICLR
37. Han X, Zhang Z, Ding N, Gu Y, Liu X, Huo Y, Qiu J, Zhang L, Han W, Huang M et al (2021) Pre-trained models: past, present and future. *AI Open* 2:225–250
38. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I (2017) Attention is all you need. *arXiv*
39. Guo M-H, Xu T-X, Liu J-J, Liu Z-N, Jiang P-T, Mu T-J, Zhang S-H, Martin RR, Cheng M-M, Hu S-M (2022) Attention mechanisms in computer vision: a survey. *Comput Visual Media* 8(3):331–368
40. Dosovitskiy A, Beyer L, Kolesnikov A, Weissenborn D, Zhai X, Unterthiner T, Dehghani M, Minderer M, Heigold G, Gelly S et al (2020) An image is worth 16x16 words: transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*
41. Yun S, Jeong M, Kim R, Kang J, Kim HJ (2019) Graph transformer networks. *Adv Neural Inf Process Syst* 32
42. Dehghani M, Djolonga J, Mustafa B, Padlewski P, Heek J, Gilmer J, Steiner AP, Caron M, Geirhos R, Alabdulmohsin I et al (2023) Scaling vision transformers to 22 billion parameters. In: International conference on machine learning. PMLR, pp 7480–7512
43. Chowdhery A, Narang S, Devlin J, Bosma M, Mishra G, Roberts A, Barham P, Chung HW, Sutton C, Gehrmann S et al (2022) Palm: scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*
44. Joshi M, Chen D, Liu Y, Weld DS, Zettlemoyer L, Levy O (2020) Spanbert: improving pre-training by representing and predicting spans. *Trans Assoc Comput Linguist* 8:64–77
45. Lewis M, Liu Y, Goyal N, Ghazvininejad M, Mohamed A, Levy O, Stoyanov V, Zettlemoyer L (2020) BART: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In: ACL. <https://doi.org/10.18653/v1/2020.acl-main.703>
46. Clark K, Luong M-T, Le QV, Manning CD (2020) Electra: pre-training text encoders as discriminators rather than generators. *arXiv*
47. Lan Z, Chen M, Goodman S, Gimpel K, Sharma P, Soricut R (2019) Albert: a lite bert for self-supervised learning of language representations. *arXiv*
48. Donahue J, Krähenbühl P, Darrell T (2016) Adversarial feature learning. *arXiv*
49. Caron M, Bojanowski P, Joulin A, Douze M (2018) Deep clustering for unsupervised learning of visual features. In: ECCV
50. Radford A, Narasimhan K, Salimans T, Sutskever I (2018) Improving language understanding by generative pre-training
51. Radford A, Wu J, Child R, Luan D, Amodei D, Sutskever I (2019) Language models are unsupervised multitask learners. OpenAI blog
52. Caruana R (1997) Multitask learning. *Mach Learn* 28:41–75
53. Peters ME, Neumann M, Iyyer M, Gardner M, Clark C, Lee K, Zettlemoyer L (2018) Deep contextualized word representations. *arXiv*
54. Wu Y, Schuster M, Chen Z, Le QV, Norouzi M, Macherey W, Krikun M, Cao Y, Gao Q, Macherey K et al (2016) Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv*
55. Liu Y, Ott M, Goyal N, Du J, Joshi M, Chen D, Levy O, Lewis M, Zettlemoyer L, Stoyanov V (2019) Roberta: A robustly optimized BERT pretraining approach. *CoRR arXiv:1907.11692*
56. Sennrich R, Haddow B, Birch A (2015) Neural machine translation of rare words with subword units. *arXiv*
57. Song K, Tan X, Qin T, Lu J, Liu T (2020) MpNet: masked and permuted pre-training for language understanding. In: NeurIPS
58. Li Q, Peng H, Li J, Xia C, Yang R, Sun L, Yu PS, He L (2022) A survey on text classification: from traditional to deep learning. *ACM Trans Intell Syst Technol (TIST)* 13(2):1–41
59. Song K, Tan X, Qin T, Lu J, Liu T-Y (2019) Mass: masked sequence to sequence pre-training for language generation. *arXiv*
60. Dong L, Yang N, Wang W, Wei F, Liu X, Wang Y, Gao J, Zhou M, Hon H-W (2019) Unified language model pre-training for natural language understanding and generation. *arXiv*
61. Sun Y, Wang S, Li Y, Feng S, Chen X, Zhang H, Tian X, Zhu D, Tian H, Wu H (2019) Ernie: enhanced representation through knowledge integration. *arXiv*
62. Sun Y, Wang S, Li Y, Feng S, Tian H, Wu H, Wang H (2020) Ernie 2.0: a continual pre-training framework for language understanding. In: AAAI
63. Cui Y, Che W, Liu T, Qin B, Yang Z (2021) Pre-training with whole word masking for Chinese BERT. *T-ASL*. <https://doi.org/10.1109/TASLP.2021.3124365>
64. Diao S, Bai J, Song Y, Zhang T, Wang Y (2020) ZEN: pre-training chinese text encoder enhanced by n-gram representations. In: EMNLP. <https://doi.org/10.18653/v1/2020.findings-emnlp.425>
65. Tsai H, Riesa J, Johnson M, Arivazhagan N, Li X, Archer A (2019) Small and practical bert models for sequence labeling. *arXiv*
66. Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O (2017) Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*
67. Thoppilan R, De Freitas D, Hall J, Shazeer N, Kulshreshtha A, Cheng H-T, Jin A, Bos T, Baker L, Du Y et al (2022) Lamda: language models for dialog applications. *arXiv preprint arXiv:2201.08239*
68. Mikolov T, Sutskever I, Chen K, Corrado G, Dean J (2013) Distributed representations of words and phrases and their compositionality. *arXiv*
69. Pennington J, Socher R, Manning CD (2014) Glove: global vectors for word representation. In: EMNLP
70. Dai AM, Le QV (2015) Semi-supervised sequence learning. *arXiv*
71. Liu P, Qiu X, Huang X (2016) Recurrent neural network for text classification with multi-task learning. *arXiv*
72. Bojanowski P, Grave E, Joulin A, Mikolov T (2017) Enriching word vectors with subword information. *TACL*
73. McCann B, Bradbury J, Xiong C, Socher R (2017) Learned in translation: Contextualized word vectors. *arXiv*
74. Dai Z, Yang Z, Yang Y, Carbonell J, Le QV, Salakhutdinov R (2019) Transformer-xl: attentive language models beyond a fixed-length context. *arXiv*
75. Kong L, d'Áutume CdM, Ling W, Yu L, Dai Z, Yogatama D (2019) A mutual information maximization perspective of language representation learning. *arXiv*
76. Wang W, Bi B, Yan M, Wu C, Bao Z, Xia J, Peng L, Si L (2019) Structbert: incorporating language structures into pre-training for deep language understanding. *arXiv*
77. Xiong W, Du J, Wang WY, Stoyanov V (2019) Pretrained encyclopedia: weakly supervised knowledge-pretrained language model. *arXiv*

78. Peters ME, Neumann M, Logan IV RL, Schwartz R, Joshi V, Singh S, Smith NA (2019) Knowledge enhanced contextual word representations. arXiv
79. Huang H, Liang Y, Duan N, Gong M, Shou L, Jiang D, Zhou M (2019) Unicoder: a universal language encoder by pre-training with multiple cross-lingual tasks. arXiv
80. Eisenschlos JM, Ruder S, Czapla P, Kardas M, Gugger S, Howard J (2019) Multifit: efficient multi-lingual language model fine-tuning. arXiv
81. Beltagy I, Lo K, Cohan A (2019) Scibert: a pretrained language model for scientific text. arXiv
82. Sun S, Cheng Y, Gan Z, Liu J (2019) Patient knowledge distillation for bert model compression. arXiv
83. Lample G, Conneau A (2019) Cross-lingual language model pretraining. arXiv
84. Zafrir O, Boudoukh G, Izsak P, Wasserblat M (2019) Q8bert: quantized 8bit bert. In: 2019 Fifth workshop on energy efficient machine learning and cognitive computing-NeurIPS edition (EMC2-NIPS). IEEE, pp 36–39
85. Sanh V, Debut L, Chaumond J, Wolf T (2019) Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. arXiv
86. Liu W, Zhou P, Zhao Z, Wang Z, Deng H, Ju Q (2020) Fastbert: a self-distilling bert with adaptive inference time. arXiv
87. Martin L, Müller B, Suárez PJO, Dupont Y, Romary L, Clergerie É, Seddah D, Sagot B (2020) Camembert: a tasty french language model. In: ACL. <https://doi.org/10.18653/v1/2020.acl-main.645>
88. Conneau A, Khandelwal K, Goyal N, Chaudhary V, Wenzek G, Guzmán F, Grave E, Ott M, Zettlemoyer L, Stoyanov V (2020) Unsupervised cross-lingual representation learning at scale. In: ACL. <https://doi.org/10.18653/v1/2020.acl-main.747>
89. Kitaev N, Kaiser L, Levskaya A (2020) Reformer: the efficient transformer. In: ICLR
90. Shen S, Dong Z, Ye J, Ma L, Yao Z, Gholami A, Mahoney MW, Keutzer K (2020) Q-bert: Hessian based ultra low precision quantization of bert. In: AAAI
91. Chi Z, Dong L, Wei F, Wang W, Mao X-L, Huang H (2020) Cross-lingual natural language generation via pre-training. In: AAAI
92. Liu W, Zhou P, Zhao Z, Wang Z, Ju Q, Deng H, Wang P (2020) K-bert: enabling language representation with knowledge graph. In: AAAI
93. Jiang Z, Yu W, Zhou D, Chen Y, Feng J, Yan S (2020) Convbort: improving BERT with span-based dynamic convolution. In: NeurIPS
94. Wang W, Wei F, Dong L, Bao H, Yang N, Zhou M (2020) Minilm: deep self-attention distillation for task-agnostic compression of pre-trained transformers. In: NeurIPS
95. Liu Y, Gu J, Goyal N, Li X, Edunov S, Ghazvininejad M, Lewis M, Zettlemoyer L (2020) Multilingual denoising pre-training for neural machine translation. Trans Assoc Comput Linguist 8:726–742
96. Sun T, Shao Y, Qiu X, Guo Q, Hu Y, Huang X, Zhang Z (2020) Colake: contextualized language and knowledge embedding. In: COLING. <https://doi.org/10.18653/v1/2020.coling-main.327>
97. Le H, Vial L, Frej J, Segonne V, Coavoux M, Lecouteux B, Allauzen A, Crabbé B, Besacier L, Schwab D (2020) Flaubert: unsupervised language model pre-training for French. In: LREC
98. Shen T, Mao Y, He P, Long G, Trischler A, Chen W (2020) Exploiting structured knowledge in text via graph-guided representation learning. In: EMNLP. <https://doi.org/10.18653/v1/2020.emnlp-main.722>
99. Jiao X, Yin Y, Shang L, Jiang X, Chen X, Li L, Wang F, Liu Q (2020) Tinybert: distilling BERT for natural language understanding. In: EMNLP. <https://doi.org/10.18653/v1/2020.findings-emnlp.372>
100. Delobelle P, Winters T, Berendt B (2020) Robbert: a dutch roberta-based language model. In: EMNLP. <https://doi.org/10.18653/v1/2020.findings-emnlp.292>
101. He B, Zhou D, Xiao J, Jiang X, Liu Q, Yuan NJ, Xu T (2020) Integrating graph contextualized knowledge into pre-trained language models. In: EMNLP. <https://doi.org/10.18653/v1/2020.findings-emnlp.207>
102. Raffel C, Shazeer N, Roberts A, Lee K, Narang S, Matena M, Zhou Y, Li W, Liu PJ (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. J Mach Learn Res 21(140):1–67
103. Schick T, Schütze H (2021) Exploiting cloze-questions for few-shot text classification and natural language inference. In: EACL
104. Wang X, Gao T, Zhu Z, Zhang Z, Liu Z, Li J, Tang J (2021) KEPLER: a unified model for knowledge embedding and pre-trained language representation. Trans Assoc Comput Linguist 9:176–194
105. Gao T, Yao X, Chen D (2021) Simcse: simple contrastive learning of sentence embeddings. CoRR [arXiv:2104.08821](https://arxiv.org/abs/2104.08821)
106. Du N, Huang Y, Dai AM, Tong S, Lepikhin D, Xu Y, Krikun M, Zhou Y, Yu AW, Firat O et al (2022) Glam: efficient scaling of language models with mixture-of-experts. In: International conference on machine learning. PMLR, pp 5547–5569
107. Chi Z, Huang S, Dong L, Ma S, Singhal S, Bajaj P, Song X, Wei F (2021) Xlm-e: cross-lingual language model pre-training via electra. arXiv preprint [arXiv:2106.16138](https://arxiv.org/abs/2106.16138)
108. Sanh V, Webson A, Raffel C, Bach SH, Sutawika L, Alyafeai Z, Chaffin A, Stiegler A, Scao TL, Raja A et al (2021) Multitask prompted training enables zero-shot task generalization. arXiv preprint [arXiv:2110.08207](https://arxiv.org/abs/2110.08207)
109. Rae JW, Borgeaud S, Cai T, Millican K, Hoffmann J, Song F, Aslanides J, Henderson S, Ring R, Young S et al (2021) Scaling language models: methods, analysis & insights from training gopher. arXiv preprint [arXiv:2112.11446](https://arxiv.org/abs/2112.11446)
110. Smith S, Patwary M, Norick B, LeGresley P, Rajbhandari S, Casper J, Liu Z, Prabhume S, Zerveas G, Korthikanti V et al (2022) Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model. arXiv preprint [arXiv:2201.11990](https://arxiv.org/abs/2201.11990)
111. Hoffmann J, Borgeaud S, Mensch A, Buchatskaya E, Cai T, Rutherford E, Casas DL, Hendricks LA, Welbl J, Clark A et al (2022) Training compute-optimal large language models. arXiv preprint [arXiv:2203.15556](https://arxiv.org/abs/2203.15556)
112. Zhang S, Roller S, Goyal N, Artetxe M, Chen M, Chen S, Dewan C, Diab M, Li X, Lin XV et al (2022) Opt: open pre-trained transformer language models. arXiv preprint [arXiv:2205.01068](https://arxiv.org/abs/2205.01068)
113. Wei J, Bosma M, Zhao V, Guu K, Yu AW, Lester B, Du N, Dai AM, Le QV (2022) Finetuned language models are zero-shot learners. In: International conference on learning representations
114. Honovich O, Scialom T, Levy O, Schick T (2022) Unnatural instructions: tuning language models with (almost) no human labor. arXiv preprint [arXiv:2212.09689](https://arxiv.org/abs/2212.09689)
115. Wang Y, Mishra S, Alipoormolabashi P, Kordi Y, Mirzaei A, Naik A, Ashok A, Dhanasekaran AS, Arunkumar A, Stap D et al (2022) Super-natural instructions: generalization via declarative instructions on 1600+ nlp tasks. In: Proceedings of the 2022 conference on empirical methods in natural language processing, pp 5085–5109
116. Mishra S, Khashabi D, Baral C, Hajishirz, H (2022) Cross-task generalization via natural language crowdsourcing instructions. In: Proceedings of the 60th Annual meeting of the association for computational linguistics (volume 1: long papers), pp 3470–3487
117. Wang Y, Kordi Y, Mishra S, Liu A, Smith NA, Khashabi D, Hajishirzi H (2022) Self-instruct: aligning language model with self generated instructions. arXiv preprint [arXiv:2212.10560](https://arxiv.org/abs/2212.10560)

118. Weidinger L, Mellor J, Rauh M, Griffin C, Uesato J, Huang P-S, Cheng M, Glaese M, Balle B, Kasirzadeh A et al (2021) Ethical and social risks of harm from language models. *arXiv preprint arXiv:2112.04359*
119. Kieglend S, Kreutzer J (2021) Revisiting the weaknesses of reinforcement learning for neural machine translation. In: *Proceedings of the 2021 conference of the North American chapter of the association for computational linguistics: human language technologies*, pp 1673–1681
120. Jaques N, Shen JH, Ghandeharioun A, Ferguson C, Lapedriza A, Jones N, Gu S, Picard R (2020) Human-centric dialog training via offline reinforcement learning. In: *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pp 3985–4003
121. Rennie SJ, Marcheret E, Mroueh Y, Ross J, Goel V (2017) Self-critical sequence training for image captioning. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp 7008–7024
122. Pang RY, He H (2021) Text generation by learning from demonstrations. In: *Proceedings of the international conference on learning representations*
123. Hausknecht M, Ammanabrolu P, Côté M-A, Yuan X (2020) Interactive fiction games: a colossal adventure. *Proc AAAI Conf Artif Intell* 34:7903–7910
124. Snell C, Kostrikov I, Su Y, Yang M, Levine S (2022) Offline rl for natural language generation with implicit language q learning. *arXiv preprint arXiv:2206.11871*
125. Lu X, Welleck S, Jiang L, Hessel J, Qin L, West P, Ammanabrolu P, Choi Y (2022) Quark: controllable text generation with reinforced unlearning. *arXiv preprint arXiv:2205.13636*
126. Uc-Cetina V, Navarro-Guerrero N, Martin-Gonzalez A, Weber C, Wermter S (2022) Survey on reinforcement learning for language processing. *Artif Intell Rev* 56:1543–1575
127. Ramamurthy R, Ammanabrolu P, Brantley K, Hessel J, Sifa R, Bauckhage C, Hajishirzi H, Choi Y (2022) Is reinforcement learning (not) for natural language processing?: benchmarks, baselines, and building blocks for natural language policy optimization. *arXiv preprint arXiv:2210.01241*
128. Wu J, Ouyang L, Ziegler DM, Stiennon N, Lowe R, Leike J, Christiano P (2021) Recursively summarizing books with human feedback. *arXiv preprint arXiv:2109.10862*
129. Nakano R, Hilton J, Balaji S, Wu J, Ouyang L, Kim C, Hesse C, Jain S, Kosaraju V, Saunders W et al (2021) Webgpt: browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*
130. Glaese A, McAleese N, Trębacz M, Aslanides J, Fiore V, Ewalds T, Rauh M, Weidinger L, Chadwick M, Thacker P et al (2022) Improving alignment of dialogue agents via targeted human judgements. *arXiv preprint arXiv:2209.14375*
131. Bai Y, Kadavath S, Kundu S, Askell A, Kernion J, Jones A, Chen A, Goldie A, Mirhoseini A, McKinnon C et al (2022) Constitutional AI: harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*
132. Chung HW, Hou L, Longpre S, Zoph B, Tay Y, Fedus W, Li E, Wang X, Dehghani M, Brahma S et al (2022) Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*
133. Kojima T, Gu SS, Reid M, Matsuo Y, Iwasawa Y (2022) Large language models are zero-shot reasoners. *Adv Neu Inf Process Syst* 35:22199–22213
134. Dosovitskiy A, Springenberg JT, Riedmiller M, Brox T (2014) Discriminative unsupervised feature learning with convolutional neural networks. *Adv Neu Inf Process Syst*
135. Dosovitskiy A, Fischer P, Springenberg JT, Riedmiller M, Brox T (2016) Discriminative unsupervised feature learning with exemplar convolutional neural networks. *TPAMI*. TPAMI-2015-05-0348.R1
136. Doersch C, Gupta A, Efros AA (2015) Unsupervised visual representation learning by context prediction. In: *ICCV*
137. Pathak D, Krahenbuhl P, Donahue J, Darrell T, Efros AA (2016) Context encoders: feature learning by inpainting. In: *CVPR*
138. Zhang R, Isola P, Efros AA (2016) Colorful image colorization. In: *ECCV*
139. Zhang R, Isola P, Efros AA (2017) Split-brain autoencoders: unsupervised learning by cross-channel prediction. In: *CVPR*
140. Noroozi M, Favaro P (2016) Unsupervised learning of visual representations by solving jigsaw puzzles. In: *ECCV*
141. Kim D, Cho D, Yoo D, Kweon IS (2018) Learning image representations by completing damaged jigsaw puzzles. In: *WACV*
142. Noroozi M, Pirsavash H, Favaro P (2017) Representation learning by learning to count. In: *ICCV*
143. Bojanowski P, Joulin A (2017) Unsupervised learning by predicting noise. In: *ICML*
144. Gidaris S, Singh P, Komodakis N (2018) Unsupervised representation learning by predicting image rotations. *arXiv*
145. Oord Avd, Li Y, Vinyals O (2018) Representation learning with contrastive predictive coding. *arXiv*
146. Henaff O (2020) Data-efficient image recognition with contrastive predictive coding. In: *ICML*
147. Donahue J, Simonyan K (2019) Large scale adversarial representation learning. In: *NeurIPS*, pp 10541–10551
148. Dumoulin V, Belghazi I, Poole B, Mastropietro O, Lamb A, Arjovsky M, Courville A (2016) Adversarially learned inference. *arXiv*
149. Chen M, Radford A, Child R, Wu J, Jun H, Luan D, Sutskever I (2020) Generative pretraining from pixels. In: *ICML*
150. Bao H, Dong L, Piao S, Wei F (2021) Beit: Bert pre-training of image transformers. In: *International conference on learning representations*
151. Caron M, Touvron H, Misra I, Jégou H, Mairal J, Bojanowski P, Joulin A (2021) Emerging properties in self-supervised vision transformers. In: *Proceedings of the IEEE/CVF international conference on computer vision*, pp 9650–9660
152. Ramesh A, Pavlov M, Goh G, Gray S, Voss C, Radford A, Chen M, Sutskever I (2021) Zero-shot text-to-image generation. In: *International conference on machine learning*. PMLR, pp 8821–8831
153. He K, Chen X, Xi S, Li Y, Dollár P, Girshick R (2022) Masked autoencoders are scalable vision learners. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp 16000–16009
154. Xie Z, Zhang Z, Cao Y, Lin Y, Bao J, Yao Z, Dai Q, Hu H (2022) Simmim: a simple framework for masked image modeling. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp 9653–9663
155. Kirillov A, Mintun E, Ravi N, Mao H, Rolland C, Gustafson L, Xiao T, Whitehead S, Berg AC, Lo W-Y et al (2023) Segment anything. *arXiv preprint arXiv:2304.02643*
156. Wu Z, Xiong Y, Yu SX, Lin D (2018) Unsupervised feature learning via non-parametric instance discrimination. In: *CVPR*
157. Liu Z, Lin Y, Cao Y, Hu H, Wei Y, Zhang Z, Lin S, Guo B (2021) Swin transformer: hierarchical vision transformer using shifted windows. In: *Proceedings of the IEEE/CVF international conference on computer vision*, pp 10012–10022
158. Li X, Wang W, Yang L, Yang J (2022) Uniform masking: enabling mae pre-training for pyramid-based vision transformers with locality. *arXiv preprint arXiv:2205.10063*
159. Chen J, Hu M, Li B, Elhoseiny M (2022) Efficient self-supervised vision pretraining with local masked reconstruction. *arXiv preprint arXiv:2206.00790*
160. Hinton G, Vinyals O, Dean J (2015) Distilling the knowledge in a neural network. *arXiv*

161. Zhuang C, Zhai AL, Yamins D (2019) Local aggregation for unsupervised learning of visual embeddings. In: ICCV
162. Misra I, Maaten L (2020) Self-supervised learning of pretext-invariant representations. In: CVPR
163. He K, Fan H, Wu Y, Xie S, Girshick R (2020) Momentum contrast for unsupervised visual representation learning. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 9729–9738
164. Chen X, Fan H, Girshick R, He K (2020) Improved baselines with momentum contrastive learning. arXiv
165. Grill J-B, Strub F, Altché F, Tallec C, Richemond PH, Buchatskaya E, Doersch C, Pires BA, Guo ZD, Azar MG et al (2020) Bootstrap your own latent: a new approach to self-supervised learning. arXiv
166. Chen T, Kornblith S, Norouzi M, Hinton G (2020) A simple framework for contrastive learning of visual representations. In: ICML
167. Caron M, Misra I, Mairal J, Goyal P, Bojanowski P, Joulin A (2020) Unsupervised learning of visual features by contrasting cluster assignments. arXiv
168. Goyal P, Caron M, Leflaudeux B, Xu M, Wang P, Pai V, Singh M, Liptchinsky V, Misra I, Joulin A et al (2021) Self-supervised pretraining of visual features in the wild. arXiv
169. Radosavovic I, Kosaraju RP, Girshick R, He K, Dollár P (2020) Designing network design spaces. In: CVPR
170. Chen X, He K (2021) Exploring simple siamese representation learning. In: CVPR
171. Li J, Zhou P, Xiong C, Hoi SCH (2021) Prototypical contrastive learning of unsupervised representations. In: ICLR. OpenReview.net
172. Zhang L, Qi G-J, Wang L, Luo J (2019) Aet vs. aed: unsupervised representation learning by auto-encoding transformations rather than data. In: CVPR
173. Bachman P, Hjelm RD, Buchwalter W (2019) Learning representations by maximizing mutual information across views. arXiv
174. Yan X, Misra I, Gupta A, Ghadiyaram D, Mahajan D (2020) Clusterfit: improving generalization of visual representations. In: CVPR
175. Asano YM, Rupprecht C, Vedaldi A (2019) Self-labelling via simultaneous clustering and representation learning. arXiv preprint [arXiv:1911.05371](https://arxiv.org/abs/1911.05371)
176. Chen T, Kornblith S, Swersky K, Norouzi M, Hinton G (2020) Big self-supervised models are strong semi-supervised learners. arXiv
177. Tian Y, Krishnan D, Isola P (2019) Contrastive multiview coding. arXiv
178. Cubuk ED, Zoph B, Shlens J, Le QV (2019) Randaugment: practical data augmentation with no separate search. arXiv
179. Tian Y, Sun C, Poole B, Krishnan D, Schmid C, Isola P (2020) What makes for good views for contrastive learning. arXiv
180. Chen X, Xie S, He K (2021) An empirical study of training self-supervised vision transformers. arXiv
181. Mitrovic J, McWilliams B, Walker JC, Buesing LH, Blundell C (2021) Representation learning via invariant causal mechanisms. In: ICLR
182. Tian Y, Chen X, Ganguli S (2021) Understanding self-supervised learning dynamics without contrastive pairs. In: ICML. Proceedings of machine learning research
183. Xie Z, Lin Y, Yao Z, Zhang Z, Dai Q, Cao Y, Hu H (2021) Self-supervised learning with swin transformers. arXiv
184. Li Z, Chen Z, Yang F, Li W, Zhu Y, Zhao C, Deng R, Wu L, Zhao R, Tang M et al (2021) Mst: masked self-supervised transformer for visual representation. Adv Neural Inf Process Syst 34
185. Bao H, Dong L, Piao S, Wei F (2022) BEit: BERT pre-training of image transformers. In: International conference on learning representations. <https://openreview.net/forum?id=p-BhZSz59o4>
186. Chen X, Ding M, Wang X, Xin Y, Mo S, Wang Y, Han S, Luo P, Zeng G, Wang J (2022) Context autoencoder for self-supervised representation learning. arXiv preprint [arXiv:2202.03026](https://arxiv.org/abs/2202.03026)
187. Dong X, Bao J, Zhang T, Chen D, Zhang W, Yuan L, Chen D, Wen F, Yu N (2021) Peco: perceptual codebook for bert pre-training of vision transformers. arXiv preprint [arXiv:2111.12710](https://arxiv.org/abs/2111.12710)
188. You Y, Chen T, Wang Z, Shen Y (2020) When does self-supervision help graph convolutional networks? In: ICML. Proceedings of machine learning research, pp 10871–10880
189. Jin W, Derr T, Liu H, Wang Y, Wang S, Liu Z, Tang J (2020) Self-supervised learning on graphs: deep insights and new direction. CoRR [arXiv:2006.10141](https://arxiv.org/abs/2006.10141)
190. Hu W, Liu B, Gomes J, Zitnik M, Liang P, Pande VS, Leskovec J (2020) Strategies for pre-training graph neural networks. In: ICLR
191. Perozzi B, Al-Rfou R, Skien S (2014) Deepwalk: online learning of social representations. In: ACM SIGKDD
192. Grover A, Leskovec J (2016) node2vec: scalable feature learning for networks. In: ACM SIGKDD
193. Tang J, Qu M, Wang M, Zhang M, Yan J, Mei Q (2015) LINE: large-scale information network embedding. In: WWW
194. Kipf TN, Welling M (2016) Variational graph auto-encoders. CoRR
195. Qiu J, Chen Q, Dong Y, Zhang J, Yang H, Ding M, Wang K, Tang J (2020) GCC: graph contrastive coding for graph neural network pre-training. In: KDD
196. Zhu Y, Xu Y, Yu F, Liu Q, Wu S, Wang L (2021) Graph contrastive learning with adaptive augmentation. In: WWW
197. You Y, Chen T, Sui Y, Chen T, Wang Z, Shen Y (2020) Graph contrastive learning with augmentations. In: NeurIPS
198. Mavromatis C, Karypis G (2021) Graph infoclust: maximizing coarse-grain mutual information in graphs. In: PAKDD
199. Sun Q, Li J, Peng H, Wu J, Ning Y, Yu PS, He L (2021) SUGAR: subgraph neural network with reinforcement pooling and self-supervised mutual information mechanism. In: WWW
200. Velickovic P, Fedus W, Hamilton WL, Liò P, Bengio Y, Hjelm RD (2019) Deep graph infomax. In: ICLR
201. Hassani K, Ahmadi AHK (2020) Contrastive multi-view representation learning on graphs. In: ICML. Proceedings of machine learning research, pp 4116–4126
202. Jiao Y, Xiong Y, Zhang J, Zhang Y, Zhang T, Zhu Y (2020) Sub-graph contrast for scalable self-supervised graph representation learning. In: ICDM, pp 222–231
203. Sun K, Lin Z, Zhu Z (2020) Multi-stage self-supervised learning for graph convolutional networks on graphs with few labeled nodes. In: AAAI, pp 5892–5899
204. Rong Y, Bian Y, Xu T, Xie W, Wei Y, Huang W, Huang J (2020) Self-supervised graph transformer on large-scale molecular data. In: NeurIPS
205. Tan Q, Liu N, Huang X, Chen R, Choi S-H, Hu X (2022) Mgae: Masked autoencoders for self-supervised learning on graphs. arXiv preprint [arXiv:2201.02534](https://arxiv.org/abs/2201.02534)
206. Hou Z, Liu X, Dong Y, Wang C, Tang J et al (2022) Graphmae: Self-supervised masked graph autoencoders. arXiv preprint [arXiv:2205.10803](https://arxiv.org/abs/2205.10803)
207. Li J, Wu R, Sun W, Chen L, Tian S, Zhu L, Meng C, Zheng Z, Wang W (2022) Maskgae: masked graph modeling meets graph autoencoders. arXiv preprint [arXiv:2205.10053](https://arxiv.org/abs/2205.10053)
208. Tian Y, Dong K, Zhang C, Zhang C, Chawla NV (2022) Heterogeneous graph masked autoencoders. arXiv preprint [arXiv:2208.09957](https://arxiv.org/abs/2208.09957)

209. Wan S, Pan S, Yang J, Gong C (2021) Contrastive and generative graph convolutional networks for graph-based semi-supervised learning. In: AAAI
210. Zhang J, Zhang H, Xia C, Sun L (2020) Graph-bert: only attention is needed for learning graph representations. *arXiv arXiv:2001.05140*
211. Peng Z, Huang W, Luo M, Zheng Q, Rong Y, Xu T, Huang J (2020) Graph representation learning via graphical mutual information maximization. In: WWW
212. Hu Z, Dong Y, Wang K, Chang K, Sun Y (2020) GPT-GNN: generative pre-training of graph neural networks. In: KDD
213. Wang G, Guo H, Li A, Liu X, Yan Q (2023) Federated iot interaction vulnerability analysis. In: 2023 IEEE 39th international conference on data engineering (ICDE). IEEE
214. Hamilton WL, Ying Z, Leskovec J (2017) Inductive representation learning on large graphs. In: NIPS
215. Hwang D, Park J, Kwon S, Kim K, Ha J, Kim HJ (2020) Self-supervised auxiliary learning with meta-paths for heterogeneous graphs. In: NeurIPS
216. Sun F, Hoffmann J, Verma V, Tang J (2020) Infograph: unsupervised and semi-supervised graph-level representation learning via mutual information maximization. In: ICLR
217. Park C, Kim D, Han J, Yu H (2020) Unsupervised attributed multiplex network embedding. In: AAAI, pp 5371–5378
218. You Y, Chen T, Shen Y, Wang Z (2021) Graph contrastive learning automated. *CoRR arXiv:2106.07594*
219. Zeng J, Xie P (2021) Contrastive self-supervised learning for graph classification. In: AAAI, pp 10824–10832
220. Xu M, Wang H, Ni B, Guo H, Tang J (2021) Self-supervised graph-level representation learning with local and global structure. *CoRR arXiv:2106.04113*
221. Wang P, Agarwal K, Ham C, Choudhury S, Reddy CK (2021) Self-supervised learning of contextual embeddings for link prediction in heterogeneous networks. In: WWW
222. Cao J, Lin X, Guo S, Liu L, Liu T, Wang B (2021) Bipartite graph embedding via mutual information maximization. In: WSDM
223. Wang X, Liu N, Han H, Shi C (2021) Self-supervised heterogeneous graph neural network with co-contrastive learning. *KDD arXiv:2105.09111*
224. Kim D, Oh A (2021) How to find your friendly neighborhood: graph attention design with self-supervision. In: ICLR. <https://openreview.net/forum?id=W5KUNlqWty>
225. Sun M, Xing J, Wang H, Chen B, Zhou J (2021) Mocl: contrastive learning on molecular graphs with multi-level domain knowledge. *CoRR arXiv:2106.04509*
226. Schneider S, Baevski A, Collobert R, Auli M (2019) wav2vec: unsupervised pre-training for speech recognition. In: INTERSPEECH
227. Baevski A, Schneider S, Auli M (2020) vq-wav2vec: self-supervised learning of discrete speech representations. In: ICLR
228. Baevski A, Zhou Y, Mohamed A, Auli M (2020) wav2vec 2.0: a framework for self-supervised learning of speech representations. In: NeurIPS
229. Chung Y, Glass JR (2020) Generative pre-training for speech with autoregressive predictive coding. In: ICASSP
230. Song X, Wang G, Huang Y, Wu Z, Su D, Hsu H (2020) Speech-xlnet: unsupervised acoustic model pretraining for self-attention networks. In: INTERSPEECH
231. Chung Y, Wang Y, Hsu W, Zhang Y, Skerry-Ryan RJ (2019) Semi-supervised training for improving data efficiency in end-to-end speech synthesis. In: ICASSP
232. Denisov P, Vu NT (2020) Pretrained semantic speech embeddings for end-to-end spoken language understanding via cross-modal teacher-student learning. In: Interspeech
233. Chung Y-A, Zhu C, Zeng M (2021) SPLAT: speech-language joint pre-training for spoken language understanding. In: ACL
234. Zeng M, Tan X, Wang R, Ju Z, Qin T, Liu T-Y (2021) Musicbert: symbolic music understanding with large-scale pre-training. *arXiv preprint arXiv:2106.05630*
235. Huang Y-S, Yang Y-H (2020) Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions. In: Proceedings of the 28th ACM international conference on multimedia, pp 1180–1188
236. Verma P, Berger J (2021) Audio transformers: Transformer architectures for large scale audio understanding. *adieu convolutions. arXiv preprint arXiv:2105.00335*
237. Fernando B, Bilen H, Gavves E, Gould S (2017) Self-supervised video representation learning with odd-one-out networks. In: CVPR
238. Misra I, Zitnick CL, Hebert M (2016) Shuffle and learn: unsupervised learning using temporal order verification. In: ECCV
239. Kim D, Cho D, Kweon IS (2019) Self-supervised video representation learning with space-time cubic puzzles. In: AAAI
240. Tao L, Wang X, Yamasaki T (2020) Self-supervised video representation learning using inter-intra contrastive framework. In: ACM multimedia
241. Lorre G, Rabarisoa J, Orcesi A, Ainouz S, Canu S (2020) Temporal contrastive pretraining for video action recognition. In: WACV
242. Yao T, Zhang Y, Qiu Z, Pan Y, Mei T (2020) Seco: Exploring sequence supervision for unsupervised representation learning. *arXiv*
243. Li LH, Yatskar M, Yin D, Hsieh C, Chang K (2019) Visualbert: a simple and performant baseline for vision and language. *CoRR arXiv:1908.03557*
244. Li G, Duan N, Fang Y, Gong M, Jiang D (2020) Unicoder-vl: a universal encoder for vision and language by cross-modal pre-training. In: AAAI
245. Su W, Zhu X, Cao Y, Li B, Lu L, Wei F, Dai J (2020) VL-BERT: pre-training of generic visual-linguistic representations. In: ICLR
246. Lu J, Batra D, Parikh D, Lee S (2019) Vilbert: pretraining task-agnostic visiolinguistic representations for vision-and-language tasks. In: NeurIPS
247. Ramesh A, Dhariwal P, Nichol A, Chu C, Chen M (2022) Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*
248. Radford A, Kim JW, Hallacy C, Ramesh A, Goh G, Agarwal S, Sastry G, Askell A, Mishkin P, Clark J, *et al.* (2021) Learning transferable visual models from natural language supervision. In: International conference on machine learning. PMLR, pp 8748–8763
249. Nichol A, Dhariwal P, Ramesh A, Shyam P, Mishkin P, McGrew B, Sutskever I, Chen M (2021) Glide: towards photorealistic image generation and editing with text-guided diffusion models. *arXiv preprint arXiv:2112.10741*
250. Sayed N, Brattoli B, Ommer B (2018) Cross and learn: cross-modal self-supervision. In: GCPR
251. Ren Z, Lee YJ (2018) Cross-domain self-supervised multi-task feature learning using synthetic imagery. In: CVPR
252. Tian Y, Krishnan D, Isola P (2020) Contrastive multiview coding. In: ECCV
253. Zlotchevski A, Drain D, Svyatkovskiy A, Clement CB, Sundaresan N, Tufano M (2022) Exploring and evaluating personalized models for code generation. In: Proceedings of the 30th ACM joint European software engineering conference and symposium on the foundations of software engineering, pp 1500–1508
254. Thakur, S., Ahmad B, Fan Z, Pearce H, Tan B, Karri R, Dolan-Gavitt B, Garg S (2022) Benchmarking large language models

- for automated verilog rtl code generation. arXiv preprint [arXiv:2212.11140](#)
255. Nijkamp E, Pang B, Hayashi H, Tu L, Wang H, Zhou Y, Savarese S, Xiong C (2022) Codegen: an open large language model for code with multi-turn program synthesis. arXiv preprint [arXiv:2203.13474](#)
 256. Poesia G, Polozov O, Le V, Tiwari A, Soares G, Meek C, Gulwani S (2022) Synchromesh: reliable code generation from pre-trained language models. arXiv preprint [arXiv:2201.11227](#)
 257. Chen Y-C, Li L, Yu L, El Kholy A, Ahmed F, Gan Z, Cheng Y, Liu J (2020) Uniter: universal image-text representation learning. In: European conference on computer vision. Springer, pp 104–120
 258. Zhu X, Zhu J, Li H, Wu X, Li H, Wang X, Dai J (2022) Uniperceiver: pre-training unified architecture for generic perception for zero-shot and few-shot tasks. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 16804–16815
 259. Reed S, Zolna K, Parisotto E, Colmenarejo SG, Novikov A, Barth-Maron G, Gimenez M, Sulsky Y, Kay J, Springenberg JT et al (2022) A generalist agent. arXiv preprint [arXiv:2205.06175](#)
 260. Li W, Gao C, Niu G, Xiao X, Liu H, Liu J, Wu H, Wang H (2020) Unimo: towards unified-modal understanding and generation via cross-modal contrastive learning. arXiv preprint [arXiv:2012.15409](#)
 261. Stadie BC, Levine S, Abbeel P (2015) Incentivizing exploration in reinforcement learning with deep predictive models. arXiv preprint [arXiv:1507.00814](#)
 262. Achiam J, Sastry S (2017) Surprise-based intrinsic motivation for deep reinforcement learning. arXiv preprint [arXiv:1703.01732](#)
 263. Pathak D, Agrawal P, Efros AA, Darrell T (2017) Curiosity-driven exploration by self-supervised prediction. In: International conference on machine learning. PMLR, pp 2778–2787
 264. Tang H, Houthooft R, Foote D, Stooke A, Xi Chen O, Duan Y, Schulman J, DeTurck F, Abbeel P (2017) # Exploration: a study of count-based exploration for deep reinforcement learning. Adv Neural Inf Process Syst 30
 265. Dey P, Medya S (2019) Manipulating node similarity measures in network. arXiv
 266. Han B, Zheng C, Chan H, Paster K, Zhang M, Ba J (2021) Learning domain invariant representations in goal-conditioned block mdps. Adv Neural Inf Process Syst 34:764–776
 267. Ding Y, Florensa C, Abbeel P, Phielipp M (2019) Goal-conditioned imitation learning. Adv Neural Inf Process Syst 32
 268. Shah R, Kumar V (2021) Rrl: Resnet as representation for reinforcement learning. arXiv preprint [arXiv:2107.03380](#)
 269. Xiao T, Radosavovic I, Darrell T, Malik J (2022) Masked visual pre-training for motor control. arXiv preprint [arXiv:2203.06173](#)
 270. Schwarzer M, Rajkumar N, Noukhovitch M, Anand A, Charlin L, Hjelm RD, Bachman P, Courville AC (2021) Pretraining representations for data-efficient reinforcement learning. Adv Neural Inf Process Syst 34:12686–12699
 271. Schwarzer M, Anand A, Goel R, Hjelm RD, Courville A, Bachman P (2020) Data-efficient reinforcement learning with self-predictive representations. arXiv preprint [arXiv:2007.05929](#)
 272. Ha D, Schmidhuber J (2018) World Models <https://doi.org/10.5281/zenodo.1207631> arXiv:1803.10122 [cs, stat]
 273. Jaderberg M, Mnih V, Czarnecki WM, Schaul T, Leibo JZ, Silver D, Kavukcuoglu K (2016) Reinforcement learning with unsupervised auxiliary tasks. arXiv. <https://doi.org/10.48550/arXiv.1611.05397>
 274. Higgins I, Pal A, Rusu AA, Matthey L, Burgess CP, Pritzel A, Botvinick M, Blundell C, Lerchner A (2018) DARLA: improving zero-shot transfer in reinforcement learning. arXiv. <https://doi.org/10.48550/arXiv.1707.08475>
 275. Finn C, Yu T, Fu J, Abbeel P, Levine S (2016) Generalizing skills with semi-supervised reinforcement learning. arXiv preprint [arXiv:1612.00429](#)
 276. Shah R, Kumar V (2021) RRL: Resnet as representation for reinforcement learning. arXiv
 277. Schwarzer M, Anand A, Goel R, Hjelm RD, Courville A, Bachman P (2021) Data-efficient reinforcement learning with self-predictive representations. arXiv. <https://doi.org/10.48550/arXiv.2007.05929>
 278. Hafner D, Lillicrap T, Ba J, Norouzi M (2019) Dream to control: learning behaviors by latent imagination. arXiv preprint [arXiv:1912.01603](#)
 279. Hafner D, Lillicrap T, Norouzi M, Ba J (2020) Mastering atari with discrete world models. arXiv preprint [arXiv:2010.02193](#)
 280. Deng F, Jang I, Ah, S (2022) Dreamerpro: reconstruction-free model-based reinforcement learning with prototypical representations. In: International conference on machine learning. PMLR, pp 4956–4975
 281. Wu P, Escontrela A, Hafner D, Goldberg K, Abbeel P (2022) Daydreamer: world models for physical robot learning. arXiv preprint [arXiv:2206.14176](#)
 282. Laskin M, Srinivas A, Abbeel P (2020) Curl: contrastive unsupervised representations for reinforcement learning. In: International conference on machine learning. PMLR, pp 5639–5650
 283. Laskin M, Lee K, Stooke A, Pinto L, Abbeel P, Srinivas A (2020) Reinforcement learning with augmented data. Adv Neural Inf Process Syst 33:19884–19895
 284. Kostrikov I, Yarats D, Fergus R (2020) Image augmentation is all you need: regularizing deep reinforcement learning from pixels. arXiv preprint [arXiv:2004.13649](#)
 285. Yarats D, Fergus R, Lazaric A, Pinto L (2021) Mastering visual continuous control: improved data-augmented reinforcement learning. arXiv preprint [arXiv:2107.09645](#)
 286. Nair S, Rajeswaran A, Kumar V, Finn C, Gupta A (2022) R3m: a universal visual representation for robot manipulation. arXiv preprint [arXiv:2203.12601](#)
 287. Parisi S, Rajeswaran A, Purushwalkam S, Gupta A (2022) The unsurprising effectiveness of pre-trained vision models for control. arXiv preprint [arXiv:2203.03580](#)
 288. Zhou C, Yan Q, Shi Y, Sun L (2022) {DoubleStar}:{Long-Range} attack towards depth estimation based obstacle avoidance in autonomous systems. In: 31st USENIX security symposium (USENIX Security 22), pp 1885–1902
 289. Zhou C, Yan Q, Kent D, Wang G, Zhang Z, Radha H (2024) Optical lens attack on deep learning based monocular depth estimation. arXiv preprint [arXiv:2409.17376](#)
 290. Zhou C, Yan Q, Liu S (2024) Transient adversarial 3d projection attacks on object detection in autonomous driving. arXiv preprint [arXiv:2409.17403](#)
 291. Jin D, Jin Z, Zhou JT, Szolovits P (2020) Is bert really robust? A strong baseline for natural language attack on text classification and entailment. In: AAAI
 292. Zang Y, Qi F, Yang C, Liu Z, Zhang M, Liu Q, Sun M (2020) Word-level textual adversarial attacking as combinatorial optimization. In: ACL
 293. Wallace E, Feng S, Kandpal N, Gardner M, Singh S (2019) Universal adversarial triggers for attacking and analyzing NLP. In: EMNLP-IJCNLP
 294. Kurita K, Michel P, Neubig G (2020) Weight poisoning attacks on pretrained models. In: ACL
 295. Schuster R, Schuster T, Meri Y, Shmatikov V (2020) Humpty dumpty: controlling word meanings via corpus poisoning. In: IEEE symposium on security and privacy
 296. Bao R, Wang J, Zhao H (2021) Defending pre-trained language models from adversarial word substitution without performance sacrifice. In: ACL/IJCNLP

297. Zhang Z, Li Y, Wang J, Liu B, Li D, Guo Y, Chen X, Liu Y (2022) Remos: reducing defect inheritance in transfer learning via relevant model slicing. In: Proceedings of the 44th international conference on software engineering, pp 1856–1868
298. Carlini N, Tramer F, Wallace E, Jagielski M, Herbert-Voss A, Lee K, Roberts A, Brown TB, Song D, Erlingsson U et al (2021) Extracting training data from large language models. In: USENIX security symposium, vol 6
299. Wang G, Zhang L, Yang Z, Li X-Y (2018) Socialite: social activity mining and friend auto-labeling. In: 2018 IEEE 37th international performance computing and communications conference (IPCCC). IEEE, pp 1–8
300. Han F, Zhang L, You X, Wang G, Li X-Y (2019) Shad: privacy-friendly shared activity detection and data sharing. In: 2019 IEEE 16th international conference on mobile ad hoc and sensor systems (MASS). IEEE, pp 109–117
301. Chen T, Zhai X, Ritter M, Lucic M, Houlisby N (2019) Self-supervised gans via auxiliary rotation loss. In: CVPR
302. Abnar S, Dehghani M, Neyshabur B, Sedghi H (2021) Exploring the limits of large scale pre-training. arXiv
303. Peters ME, Neumann M, Iyyer M, Gardner M, Clark C, Lee K, Zettlemoyer L (2018) Deep contextualized word representations. In: NAACL-HLT
304. Mikolov T, Sutskever I, Chen K, Corrado GS, Dean J (2013) Distributed representations of words and phrases and their compositionality. In: NIPS
305. Collobert R, Weston J, Bottou L, Karlen M, Kavukcuoglu K, Kuksa PP (2011) Natural language processing (almost) from scratch. *J Mach Learn Res* 12:2493–2537
306. Neelakantan A, Shankar J, Passos A, McCallum A (2014) Efficient non-parametric estimation of multiple embeddings per word in vector space. In: EMNLP. <https://doi.org/10.3115/v1/d14-1113>
307. Zhou P, Qi Z, Zheng S, Xu J, Bao H, Xu B (2016) Text classification improved by integrating bidirectional LSTM with two-dimensional max pooling. In: COLING
308. Hui DU, Xueke XU, Dayong WU, Liu Y, Zhihua YU, Cheng X (2017) A sentiment classification method based on sentiment-specific word embedding. *J Chin Inf Process* 31(3):170–176
309. Liu Y, Ma C, Zhang Y (2017) Hierarchical machine translation model based on deep recursive neural network. *Chin J Comput* 40(4):861–871
310. Liang X, Ren F, Liu Y, Pan L, Hou Y, Zhang Y, Yan LI (2018) N-reader: machine reading comprehension model based on double layers of self-attention. *J Chin Inf Process*
311. Zhichang Z, Zhenwen Z, Zhiman Z (2019) User intent classification based on indrnn-attention. *J Comput Res Dev*
312. Deng J, Dong W, Socher R, Li L-J, Li K, Fei-Fei L (2009) Imagenet: a large-scale hierarchical image database. In: 2009 IEEE conference on computer vision and pattern recognition
313. Krizhevsky A, Sutskever I, Hinton GE (2012) Imagenet classification with deep convolutional neural networks. *Adv Neural Inf Proces Syst*
314. Lin M, Chen Q, Yan S (2013) Network in network. arXiv
315. Simonyan K, Zisserman A (2014) Very deep convolutional networks for large-scale image recognition. arXiv
316. Szegedy C, Liu W, Jia Y, Sermanet P, Reed S, Anguelov D, Erhan D, Vanhoucke V, Rabinovich A (2015) Going deeper with convolutions. In: CVPR
317. He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: CVPR
318. Huang G, Liu Z, Van Der Maaten L, Weinberger KQ (2017) Densely connected convolutional networks. In: CVPR
319. Girshick R, Donahue J, Darrell T, Malik J (2014) Rich feature hierarchies for accurate object detection and semantic segmentation. In: CVPR
320. Girshick R (2015) Fast r-cnn. In: ICCV
321. Ren S, He K, Girshick R, Sun J (2015) Faster r-cnn: towards real-time object detection with region proposal networks. arXiv
322. He K, Gkioxari G, Dollár P, Girshick R (2017) Mask r-cnn. In: ICCV
323. Long J, Shelhamer E, Darrell T (2015) Fully convolutional networks for semantic segmentation. In: CVPR
324. Liu W, Anguelov D, Erhan D, Szegedy C, Reed S, Fu C-Y, Berg AC (2016) Ssd: single shot multibox detector. In: European conference on computer vision
325. Redmon J, Divvala S, Girshick R, Farhadi A (2016) You only look once: unified, real-time object detection. In: CVPR
326. Redmon J, Farhadi A (2017) Yolo9000: better, faster, stronger. In: CVPR
327. Redmon J, Farhadi A (2018) Yolo3: An incremental improvement. arXiv
328. Bochkovskiy A, Wang C-Y, Liao H-YM (2020) Yolo4: optimal speed and accuracy of object detection. arXiv
329. Chen Q, Wang Y, Yang T, Zhang X, Cheng J, Sun J (2021) You only look one-level feature. arXiv
330. Badrinarayanan V, Kendall A, Cipolla R (2017) Segnet: a deep convolutional encoder-decoder architecture for image segmentation. *IEEE Trans Pattern Anal Mach Intell* 39(12):2481–2495
331. Zhao H, Shi J, Qi X, Wang X, Jia J (2017) Pyramid scene parsing network. In: CVPR
332. Chen L-C, Papandreou G, Kokkinos I, Murphy K, Yuille AL (2014) Semantic image segmentation with deep convolutional nets and fully connected crfs. arXiv
333. Chen L-C, Papandreou G, Kokkinos I, Murphy K, Yuille AL (2017) Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE Trans Pattern Anal Mach Intell* 40(4):834–848
334. Chen L-C, Papandreou G, Schroff F, Adam H (2017) Rethinking atrous convolution for semantic image segmentation. arXiv
335. Chen L-C, Zhu Y, Papandreou G, Schroff F, Adam H (2018) Encoder-decoder with atrous separable convolution for semantic image segmentation. In: ECCV
336. Lin G, Milan A, Shen C, Reid I (2017) Refinenet: multi-path refinement networks for high-resolution semantic segmentation. In: CVPR
337. Everingham M, Eslami SA, Van Gool L, Williams CK, Winn J, Zisserman A (2015) The pascal visual object classes challenge: a retrospective. *Int J Comput Vision* 111:98–136
338. Everingham M, Van Gool L, Williams CK, Winn J, Zisserman A (2010) The pascal visual object classes (voc) challenge. *Int J Comput Vision* 88:303–338
339. Lin T-Y, Maire M, Belongie S, Hays J, Perona P, Ramanan D, Dollár P, Zitnick CL (2014) Microsoft coco: common objects in context. In: European Conference on Computer Vision
340. Rumelhart DE, Hinton GE, Williams RJ (1986) Learning representations by back-propagating errors. *Nature* 323(6088):533–536
341. Jordan MI (1997) Serial order: a parallel distributed processing approach. In: *Advances in psychology*, vol 121, pp 471–495. North-Holland
342. Elman JL (1990) Finding structure in time. *Cogn Sci* 14(2):179–211
343. Hochreiter S, Schmidhuber J (1997) Long short-term memory. *Neural Comput MIT-Press*
344. Graves A, Liwicki M, Fernández S, Bertolami R, Bunke H, Schmidhuber J (2008) A novel connectionist system for unconstrained handwriting recognition. *IEEE Trans Pattern Anal Mach Intell* 31(5):855–868
345. Vinyals O, Toshev A, Bengio S, Erhan D (2015) Show and tell: a neural image caption generator. In: CVPR

346. Sutskever I, Vinyals O, Le QV (2014) Sequence to sequence learning with neural networks. arXiv
347. Graves A (2013) Generating sequences with recurrent neural networks. arXiv
348. Sundermeyer M, Schlüter R, Ney H (2012) Lstm neural networks for language modeling. In: INTERSPEECH
349. Goodfellow IJ, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y (2014) Generative adversarial networks. arXiv
350. Isola P, Zhu J-Y, Zhou T, Efros AA (2017) Image-to-image translation with conditional adversarial networks. In: CVPR
351. Li C, Wand M (2016) Precomputed real-time texture synthesis with Markovian generative adversarial networks. In: European conference on computer vision
352. Zhu J-Y, Park T, Isola P, Efros AA (2017) Unpaired image-to-image translation using cycle-consistent adversarial networks. In: ICCV
353. Karras T, Laine S, Aila T (2019) A style-based generator architecture for generative adversarial networks. In: CVPR
354. Van Oord A, Kalchbrenner N, Kavukcuoglu K (2016) Pixel recurrent neural networks. In: International conference on machine learning
355. Kim T, Cha M, Kim H, Lee JK, Kim J (2017) Learning to discover cross-domain relations with generative adversarial networks. In: International conference on machine learning
356. Denton E, Chintala S, Szlam A, Fergus R (2015) Deep generative image models using a laplacian pyramid of adversarial networks. arXiv
357. Huang X, Li Y, Poursaeed O, Hopcroft J, Belongie S (2017) Stacked generative adversarial networks. In: CVPR
358. Hu J, Shen L, Sun G (2018) Squeeze-and-excitation networks. In: CVPR
359. Woo S, Park J, Lee J-Y, Kweon IS (2018) Cbam: convolutional block attention module. In: ECCV
360. Cao Y, Xu J, Lin S, Wei F, Hu H (2019) Gcnet: non-local networks meet squeeze-excitation networks and beyond. In: ICCV workshops
361. Huang Z, Wang X, Huang L, Huang C, Wei Y, Liu W (2019) Ccnet: Criss-cross attention for semantic segmentation. In: ICCV
362. Zhang H, Goodfellow I, Metaxas D, Odena A (2019) Self-attention generative adversarial networks. In: International conference on machine learning
363. Touvron H, Cord M, Douze M, Massa F, Sablayrolles A, Jégou H (2020) Training data-efficient image transformers & distillation through attention. arXiv
364. Carion N, Massa F, Synnaeve G, Usunier N, Kirillov A, Zagoruyko S (2020) End-to-end object detection with transformers. In: European conference on computer vision
365. Graham B, El-Nouby A, Touvron H, Stock P, Joulin A, Jégou H, Douze M (2021) Levit: a vision transformer in convnet's clothing for faster inference. arXiv
366. Zhou D, Kang B, Jin X, Yang L, Lian X, Jiang Z, Hou Q, Feng J (2021) Deepvit: towards deeper vision transformer. arXiv
367. Wang W, Xie E, Li X, Fan D-P, Song K, Liang D, Lu T, Luo P, Shao L (2021) Pyramid vision transformer: a versatile backbone for dense prediction without convolutions. arXiv
368. Guan T, Wang J, Lan S, Chandra R, Wu Z, Davis L, Manocha D (2021) M3detr: multi-representation, multi-scale, mutual-relation 3d object detection with transformers. arXiv
369. Valanarasu JMJ, Oza P, Hacıhaliloglu I, Patel VM (2021) Medical transformer: gated axial-attention for medical image segmentation. arXiv
370. Lee RCT, Chin YH, Chang SC (1976) Application of principal component analysis to multikey searching. *IEEE Trans Softw Eng* 3:185–193
371. Ye J, Janardan R, Li Q (2004) Two-dimensional linear discriminant analysis. In: *Advances in neural information processing systems vol 17* [Neural information processing systems, NIPS 2004, December 13–18, 2004, Vancouver, British Columbia, Canada], pp 1569–1576
372. Robinson S, Bennett R (1995) A typology of deviant workplace behaviors: a multidimensional scaling study. *Acad Manag J* 38:555–572
373. Samko O, Marshall AD, Rosin PL (2006) Selection of the optimal parameter value for the isomap algorithm. *Pattern Recogn. Lett.* 9:968–979
374. Roweis ST, Saul LK (2000) Nonlinear dimensionality reduction by locally linear embedding. *Science* 290(5500):2323–2326
375. Belkin M, Niyogi P (2003) Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Comput* 6:1373–1396
376. Singh AP, Gordon GJ. Relational learning via collective matrix factorization. In: *ACM SIGKDD*, pp 650–658
377. Cao S, Lu W, Xu Q (2015) Grarep: learning graph representations with global structural information. In: *CIKM*, pp 891–900
378. Ou M, Cui P, Pei J, Zhang Z, Zhu W (2016) Asymmetric transitivity preserving graph embedding. In: *ACM SIGKDD*, pp 1105–1114
379. Sugiyama M, Borgwardt KM (2015) Halting in random walk kernels. In: *NIPS*
380. Kang U, Tong H, Sun J (2012) Fast random walk graph kernel. In: *SIAM*, pp 828–838
381. Shervashidze N, Schweitzer P, Leeuwen EJ, Mehlhorn K, Borgwardt KM (2011) Weisfeiler-lehman graph kernels. *J Mach Learn Res* 12:2539–2561
382. Erhan D, Manzagol P-A, Bengio Y, Bengio S, Vincent P (2009) The difficulty of training deep architectures and the effect of unsupervised pre-training. In: *Artificial intelligence and statistics*
383. Erhan D, Courville A, Bengio Y, Vincent P (2010) Why does unsupervised pre-training help deep learning? In: *AISTATS*
384. Lee JD, Lei Q, Saunshi N, Zhuo J (2020) Predicting what you already know helps: provable self-supervised learning. arXiv
385. Tosh C, Krishnamurthy A, Hsu D (2021) Contrastive learning, multi-view redundancy, and linear models. In: *Algorithmic learning theory*
386. Arora S, Khandeparkar H, Khodak M, Plevrakis O, Saunshi N (2019) A theoretical analysis of contrastive unsupervised representation learning. arXiv
387. Anwar S, Tahir M, Li C, Mian A, Khan FS, Muzaffar AW (2020) Image colorization: a survey and dataset. arXiv
388. Ledig C, Theis L, Huszár F, Caballero J, Cunningham A, Acosta A, Aitken A, Tejani A, Totz J, Wang Z et al (2017) Photo-realistic single image super-resolution using a generative adversarial network. In: *CVPR*
389. Perarnau G, Van De Weijer J, Raducanu B, Álvarez JM (2016) Invertible conditional gans for image editing. arXiv
390. Vondrick C, Pirsaviash H, Torralba A (2016) Generating videos with scene dynamics. arXiv
391. Tulyakov S, Liu M-Y, Yang X, Kautz J (2018) Mocogan: decomposing motion and content for video generation. In: *CVPR*
392. Wang X, Gupta A (2015) Unsupervised learning of visual representations using videos. In: *ICCV*
393. Wei C, Xie L, Ren X, Xia Y, Su C, Liu J, Tian Q, Yuille AL (2019) Iterative reorganization with weak spatial constraints: solving arbitrary jigsaw puzzles for unsupervised representation learning. In: *CVPR*
394. Ahsan U, Madhok R, Essa I (2019) Video jigsaw: Unsupervised learning of spatiotemporal context for video action recognition. In: *WACV*
395. Pathak D, Girshick R, Dollár P, Darrell T, Hariharan B (2017) Learning features by watching objects move. In: *CVPR*

396. Croitoru I, Bogolin S-V, Leordeanu M (2017) Unsupervised learning from video to detect foreground objects in single images. In: ICCV
397. Sermanet P, Lynch C, Chebotar Y, Hsu J, Jang E, Schaal S, Levine S, Brain G (2018) Time-contrastive networks: self-supervised learning from video. In: ICRA
398. Korbar B, Tran D, Torresani L (2018) Cooperative learning of audio and video models from self-supervised synchronization. arXiv
399. Manning CD, Raghavan P, Schütze H (2008). Introduction to information retrieval. <https://doi.org/10.1017/CBO9780511809071>
400. Schapire RE, Singer Y (1999) Improved boosting algorithms using confidence-rated predictions. Mach Learn. <https://doi.org/10.1023/A:1007614523901>
401. Reiter E (2018) A structured review of the validity of bleu. Comput Linguist
402. Denkowski M, Lavie A (2014) Meteor universal: language specific translation evaluation for any target language. In: Proceedings of the ninth workshop on statistical machine translation
403. Lin C-Y, Hovy E (2003) Automatic evaluation of summaries using n-gram co-occurrence statistics. In: HLT-NAACL
404. Kim Y (2014) Convolutional neural networks for sentence classification. In: EMNLP. <https://doi.org/10.3115/v1/d14-1181>
405. Kalchbrenner N, Grefenstette E, Blunsom P (2014) A convolutional neural network for modelling sentences. In: ACL
406. Yang M, Zhao W, Ye J, Lei Z, Zhao Z, Zhang S (2018) Investigating capsule networks with dynamic routing for text classification. In: EMNLP
407. Yao L, Mao C, Luo Y (2019) Graph convolutional networks for text classification. In: AAAI. <https://doi.org/10.1609/aaai.v33i01.33017370>
408. Wang Y, Sun A, Han J, Liu Y, Zhu X (2018) Sentiment analysis by capsules In: WWW. <https://doi.org/10.1145/3178876.3186015>
409. Socher R, Perelygin A, Wu J, Chuang J, Manning CD, Ng AY, Potts C (2013) Recursive deep models for semantic compositionality over a sentiment treebank. In: EMNLP
410. Tai KS, Socher R, Manning CD (2015) Improved semantic representations from tree-structured long short-term memory networks. In: ACL. <https://doi.org/10.3115/v1/p15-1150>
411. Zhu X, Sobhani P, Guo H (2015) Long short-term memory over recursive structures. In: ICML
412. Cheng J, Dong L, Lapata M (2016) Long short-term memory-networks for machine reading. In: EMNLP. <https://doi.org/10.18653/v1/d16-1053>
413. Liu P, Qiu X, Chen X, Wu S, Huang X (2015) Multi-timescale long short-term memory neural network for modelling sentences and documents. In: EMNLP. <https://doi.org/10.18653/v1/d15-1280>
414. Liu P, Qiu X, Huang X (2016) Recurrent neural network for text classification with multi-task learning. In: IJCAI
415. Socher R, Pennington J, Huang EH, Ng AY, Manning CD (2011) Semi-supervised recursive autoencoders for predicting sentiment distributions. In: EMNLP
416. Shen T, Zhou T, Long G, Jiang J, Zhang C (2018) Bi-directional block self-attention for fast and memory-efficient sequence modeling. In: ICLR
417. Le QV, Mikolov T (2014) Distributed representations of sentences and documents. In: ICML
418. Iyyer M, Manjunatha V, Boyd-Graber JL, III HD (2015) Deep unordered composition rivals syntactic methods for text classification. In: ACL. <https://doi.org/10.3115/v1/p15-1162>
419. Miyato T, Dai AM, Goodfellow IJ (2017) Adversarial training methods for semi-supervised text classification. In: ICLR
420. Lai S, Xu L, Liu K, Zhao J (2015) Recurrent convolutional neural networks for text classification. In: AAAI
421. Johnson R, Zhang T (2016) Supervised and semi-supervised text categorization using LSTM for region embeddings. In: ICML
422. Bao Y, Wu M, Chang S, Barzilay R (2020) Few-shot text classification with distributional signatures. In: ICLR
423. Wu F, Jr AHS, Zhang T, Fifty C, Yu T, Weinberger KQ (2019) Simplifying graph convolutional networks. In: ICML
424. Zhang X, Zhao JJ, LeCun Y (2015) Character-level convolutional networks for text classification. In: NIPS
425. Johnson R, Zhang T (2017) Deep pyramid convolutional neural networks for text categorization. In: ACL. <https://doi.org/10.18653/v1/P17-1052>
426. Wang J, Wang Z, Zhang D, Yan J (2017) Combining knowledge with deep convolutional neural networks for short text classification. In: IJCAI. <https://doi.org/10.24963/ijcai.2017/406>
427. Huang L, Ma D, Li S, Zhang X, Wang H (2019) Text level graph neural network for text classification. In: EMNLP-IJCNLP. <https://doi.org/10.18653/v1/D19-1345>
428. Sun C, Qiu X, Xu Y, Huang X (2019) How to fine-tune BERT for text classification? In: CCL. https://doi.org/10.1007/978-3-030-32381-3_16
429. Yang Z, Yang D, Dyer C, He X, Smola AJ, Hovy EH (2016) Hierarchical attention networks for document classification. In: NAACL-HLT
430. Bowman SR, Angeli G, Potts C, Manning CD (2015) A large annotated corpus for learning natural language inference. In: EMNLP. <https://doi.org/10.18653/v1/d15-1075>
431. Wang Z, Hamza W, Florian R (2017) Bilateral multi-perspective matching for natural language sentences. In: IJCAI. <https://doi.org/10.24963/ijcai.2017/579>
432. Liu X, He P, Chen W, Gao J (2019) Multi-task deep neural networks for natural language understanding. In: ACL. <https://doi.org/10.18653/v1/p19-1441>
433. Williams A, Nangia N, Bowman SR (2018) A broad-coverage challenge corpus for sentence understanding through inference. In: NAACL-HLT. <https://doi.org/10.18653/v1/n18-1101>
434. Marelli M, Bentivogli L, Baroni M, Bernardi R, Menini S, Zamparelli R (2014) Semeval-2014 task 1: Evaluation of compositional distributional semantic models on full sentences through semantic relatedness and textual entailment. In: SemEval@COLING. <https://doi.org/10.3115/v1/s14-2001>
435. Dolan B, Quirk C, Brockett C (2004) Unsupervised construction of large paraphrase corpora: Exploiting massively parallel news sources. In: COLING
436. Fu J, Liu P, Neubig G (2020) Interpretable multi-dataset evaluation for named entity recognition. In: EMNLP. <https://doi.org/10.18653/v1/2020.emnlp-main.489>
437. Lester B, Pressel D, Hemmeter A, Choudhury SR, Bangalore S (2020) Constrained decoding for computationally efficient named entity recognition taggers. In: EMNLP. <https://doi.org/10.18653/v1/2020.findings-emnlp.166>
438. Luo Y, Zhao H, Zhan J (2020) Named entity recognition only from word embeddings. In: EMNLP. <https://doi.org/10.18653/v1/2020.emnlp-main.723>
439. Li X, Feng J, Meng Y, Han Q, Wu F, Li J (2020) A unified MRC framework for named entity recognition. In: ACL. <https://doi.org/10.18653/v1/2020.acl-main.519>
440. Zhang Y, Yang J (2018) Chinese NER using lattice LSTM. In: ACL. <https://doi.org/10.18653/v1/P18-1144>
441. Meng Y, Wu W, Wang F, Li X, Nie P, Yin F, Li M, Han Q, Sun X, Li J (2019) Glyce: Glyph-vectors for Chinese character representations. In: NeurIPS

442. Katiyar A, Cardie C (2018) Nested named entity recognition revisited. In: NAACL-HLT. <https://doi.org/10.18653/v1/n18-1079>
443. Wang B, Lu W (2018) Neural segmental hypergraphs for overlapping mention recognition. In: EMNLP. <https://doi.org/10.18653/v1/d18-1019>
444. Luan Y, Wadden D, He L, Shah A, Ostendorf M, Hajishirzi H (2019) A general framework for information extraction using dynamic span graphs. In: NAACL-HLT. <https://doi.org/10.18653/v1/n19-1308>
445. Shibuya T, Hovy EH (2020) Nested named entity recognition via second-best sequence learning and decoding. *Trans Assoc Comput Linguist* 8:605–620
446. Lin H, Lu Y, Han X, Sun L (2019) Sequence-to-nuggets: Nested entity mention detection via anchor-region networks. In: ACL. <https://doi.org/10.18653/v1/p19-1511>
447. Lai G, Xie Q, Liu H, Yang Y, Hovy EH (2017) RACE: large-scale reading comprehension dataset from examinations. In: EMNLP. <https://doi.org/10.18653/v1/d17-1082>
448. Yang Y, Yih W, Meek C (2015) Wikiqa: a challenge dataset for open-domain question answering. In: EMNLP. <https://doi.org/10.18653/v1/d15-1237>
449. Santos CN, Tan M, Xiang B, Zhou B (2016) Attentive pooling networks. *CoRR arXiv:1602.03609*
450. Lee JY, Dernoncourt F (2016) Sequential short-text classification with recurrent and convolutional neural networks. In: NAACL-HLT. <https://doi.org/10.18653/v1/n16-1062>
451. Kim S, D'Haro LF, Banchs RE, Williams JD, Henderson M (2016) The fourth dialog state tracking challenge. In: Dialogues with social robots—enablers, analyses, and evaluation, seventh international workshop on spoken dialogue systems, IWSDS 2016, Saarisekä, Finland, January 13–16, 2016. https://doi.org/10.1007/978-981-10-2585-3_36
452. Ang J, Liu Y, Shriberg E (2005) Automatic dialog act segmentation and classification in multiparty meetings. In: 2005 IEEE international conference on acoustics, speech, and signal processing, ICASSP '05, Philadelphia, Pennsylvania, USA, March 18–23, 2005. <https://doi.org/10.1109/ICASSP.2005.1415300>
453. Wan Y, Yan W, Gao J, Zhao Z, Wu J, Yu PS (2018) Improved dynamic memory network for dialogue act classification with adversarial training. In: IEEE international conference on Big Data, Big Data 2018, Seattle, WA, USA, December 10–13, 2018. <https://doi.org/10.1109/BigData.2018.8622245>
454. Raheja V, Tetreault JR (2019) Dialogue act classification with context-aware self-attention. In: Proc. NAACL, 2019. <https://doi.org/10.18653/v1/n19-1373>
455. Xu J, Gan Z, Cheng Y, Liu J (2020) Discourse-aware neural extractive text summarization. In: ACL. <https://doi.org/10.18653/v1/2020.acl-main.451>
456. Zou Y, Zhang X, Lu W, Wei F, Zhou M (2020) Pre-training for abstractive document summarization by reinstating source text. In: EMNLP. <https://doi.org/10.18653/v1/2020.emnlp-main.297>
457. Liu L, Lu Y, Yang M, Qu Q, Zhu J, Li H (2018) Generative adversarial network for abstractive text summarization. In: AAAI. <https://doi.org/10.1609/aaai.v33i01.33017362>
458. Yang M, Qu Q, Tu W, Shen Y, Zhao Z, Chen X (2019) Exploring human-like reading strategy for abstractive text summarization. In: AAAI. <https://doi.org/10.1609/aaai.v33i01.33017362>
459. Bhandari M, Gour PN, Ashfaq A, Liu P, Neubig G (2020) Re-evaluating evaluation in text summarization. In: EMNLP. <https://doi.org/10.18653/v1/2020.emnlp-main.751>
460. Dong Y, Wang S, Gan Z, Cheng Y, Cheung JCK, Liu J (2020) Multi-fact correction in abstractive text summarization. In: EMNLP. <https://doi.org/10.18653/v1/2020.emnlp-main.749>
461. Huang D, Cui L, Yang S, Bao G, Wang K, Xie J, Zhang Y (2020) What have we achieved on text summarization? In: EMNLP. <https://doi.org/10.18653/v1/2020.emnlp-main.33>
462. Kryscinski W, Paulus R, Xiong C, Socher R (2018) Improving abstraction in text summarization. In: EMNLP. <https://doi.org/10.18653/v1/d18-1207>
463. Kryscinski W, McCann B, Xiong C, Socher R (2020) Evaluating the factual consistency of abstractive text summarization. In: EMNLP. <https://doi.org/10.18653/v1/2020.emnlp-main.750>
464. Kouris P, Alexandridis G, Stafylopatis A (2019) Abstractive text summarization based on deep learning and semantic content generalization. In: ACL. <https://doi.org/10.18653/v1/p19-1501>
465. Chen K, Wang R, Utiyama M, Sumita E (2020) Content word aware neural machine translation. In: ACL. <https://doi.org/10.18653/v1/2020.acl-main.34>
466. Lin Z, Pan X, Wang M, Qiu X, Feng J, Zhou H, Li L (2020) Pre-training multilingual neural machine translation by leveraging alignment information. In: EMNLP. <https://doi.org/10.18653/v1/2020.emnlp-main.210>
467. Bugliarello E, Okazaki N (2020) Enhancing machine translation with dependency-aware self-attention. In: ACL. <https://doi.org/10.18653/v1/2020.acl-main.147>
468. Aji AF, Bogoychev N, Heafield K, Sennrich R (2020) In neural machine translation, what does transfer learning transfer? In: ACL. <https://doi.org/10.18653/v1/2020.acl-main.688>
469. Baziotis C, Haddow B, Birch A (2020) Language model prior for low-resource neural machine translation. In: EMNLP. <https://doi.org/10.18653/v1/2020.emnlp-main.615>
470. Cui Q, Huang S, Li J, Geng X, Zheng Z, Huang G, Chen J (2021) Directq: Direct pretraining for machine translation quality estimation. In: AAAI
471. Wu C, Hoi SCH, Socher R, Xiong C (2020) TOD-BERT: pre-trained natural language understanding for task-oriented dialogue. In: EMNLP. <https://doi.org/10.18653/v1/2020.emnlp-main.66>
472. Campagna G, Forciarz A, Moradshahi M, Lam MS (2020) Zero-shot transfer learning with synthesized data for multi-domain dialogue state tracking. In: ACL. <https://doi.org/10.18653/v1/2020.acl-main.12>
473. Liu Q, Yu L, Rimell L, Blunsom P (2021) Pretraining the noisy channel model for task-oriented dialogue. *CoRR arXiv:2103.10518*
474. SST Corpus. <http://nlp.stanford.edu/sentiment> (2013)
475. Pang B, Lee L (2005) Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales. In: ACL
476. Cer D, Diab M, Agirre E, Lopez-Gazpio I, Specia L (2017) SemEval-2017 task 1: semantic textual similarity-multilingual and cross-lingual focused evaluation. *arXiv*
477. Hendrickx I, Kim SN, Kozareva Z, Nakov P, Séaghdha DÓ, Padó S, Pennacchiotti M, Romano L, Szpakowicz S (2009) SemEval-2010 task 8: multi-way classification of semantic relations between pairs of nominals. In: Proc. NAACL, 2009
478. Wiebe J, Wilson T, Cardie C (2005) Annotating expressions of opinions and emotions in language. *Lang Resour Eval*. <https://doi.org/10.1007/s10579-005-7880-9>
479. MPQA Corpus. <http://www.cs.pitt.edu/mpqa/> (2005)
480. Diao Q, Qiu M, Wu C, Smola AJ, Jiang J, Wang C (2014) Jointly modeling aspects, ratings and sentiments for movie recommendation (JMARS). In: ACM SIGKDD. <https://doi.org/10.1145/2623330.2623758>
481. 20NG Corpus. <http://ana.cachopo.org/datasets-for-single-label-text-categorization> (2007)
482. AG Corpus. http://www.di.unipi.it/~gulli/AG_corpus_of_news_articles.html (2004)
483. Reuters Corpus. <https://www.cs.umb.edu/~smimarog/textmining/datasets/> (2007)
484. Reuters Corpus. <https://martin-thoma.com/nlp-reuters> (2017)

485. Lehmann J, Isele R, Jakob M, Jentzsch A, Kontokostas D, Mendes PN, Hellmann S, Morsey M, Kleef P, Auer S, Bizer C (2015) Dbpedia—a large-scale, multilingual knowledge base extracted from wikipedia. Semantic Web. <https://doi.org/10.3233/SW-140134>
486. Ohsumed Corpus (2015) <http://davis.wpi.edu/xmdv/datasets/ohsumed.html>
487. Williams A, Nangia N, Bowman SR (2017) A broad-coverage challenge corpus for sentence understanding through inference. arXiv
488. Rajpurkar P, Zhang J, Lopyrev K, Liang P (2016) Squad: 100,000+ questions for machine comprehension of text. arXiv
489. Levesque H, Davis E, Morgenstern L (2012) The winograd schema challenge. In: Thirteenth international conference on the principles of knowledge representation and reasoning
490. Dolan WB, Brockett C (2005) Automatically constructing a corpus of sentential paraphrases. In: IWP
491. Rajpurkar P, Jia R, Liang P (2018) Know what you don't know: unanswerable questions for squad. arXiv
492. Lai G, Xie Q, Liu H, Yang Y, Hovy E (2017) Race: large-scale reading comprehension dataset from examinations. arXiv
493. Jurafsky D, Shriberg E (1997) Switchboard swbd-damsl shallow-discourse-function annotation coders manual
494. Li J, Zhou P, Xiong C, Socher R, Hoi SC (2020) Prototypical contrastive learning of unsupervised representations. arXiv preprint [arXiv:2005.04966](https://arxiv.org/abs/2005.04966)
495. Donahue J, Simonyan K (2019) Large scale adversarial representation learning. Adv Neural Inf Process Syst 32
496. He K, Chen X, Xie S, Li Y, Dollár P, Girshick R (2021) Masked autoencoders are scalable vision learners. arXiv preprint [arXiv:2111.06377](https://arxiv.org/abs/2111.06377)
497. <http://yann.lecun.com/exdb/mnist/>
498. <http://ufldl.stanford.edu/housenumbers/>
499. <https://www.cs.toronto.edu/~kriz/index.html>
500. Coates A, Ng A, Lee H (2011) An analysis of single-layer networks in unsupervised feature learning. In: Proceedings of the fourteenth international conference on artificial intelligence and statistics
501. <https://cs.stanford.edu/~acoates/stl10/>
502. http://www.vision.caltech.edu/Image_Datasets/Caltech101/
503. Miller GA (1998) WordNet: an electronic lexical database
504. <https://image-net.org/>
505. <https://serre-lab.clps.brown.edu/resource/hmdb-a-large-human-motion-database/>
506. Kuehne H, Jhuang H, Garrote E, Poggio T, Serre T (2011) HMDB: a large video database for human motion recognition. In: ICCV
507. <https://www.crcv.ucf.edu/data/UCF101.php>
508. <https://www.crcv.ucf.edu/data/UCF50.php>
509. Bossard L, Guillaumin M, Van Gool L (2014) Food-101—mining discriminative components with random forests. In: European conference on computer vision
510. Berg T, Liu J, Woo Lee S, Alexander ML, Jacobs DW, Belhumeur PN (2014) Birdsnap: large-scale fine-grained visual categorization of birds. In: CVPR
511. Xiao J, Hays J, Ehinger KA, Oliva A, Torralba A (2010) Sun database: large-scale scene recognition from abbey to zoo. In: 2010 IEEE computer society conference on computer vision and pattern recognition
512. Xiao J, Ehinger KA, Hays J, Torralba A, Oliva A (2016) Sun database: exploring a large collection of scene categories. Int J Comput Vis 119:3–22
513. <http://places.csail.mit.edu/downloadData.html>
514. http://ai.stanford.edu/~jkrause/cars/car_dataset.html
515. Maji S, Kannala J, Rahtu E, Blaschko M, Vedaldi A (2013) Fine-grained visual classification of aircraft. Technical report
516. <https://sites.google.com/site/fgcomp2013/>
517. <https://www.robots.ox.ac.uk/~vgg/data/fgvc-aircraft/>
518. <https://www.robots.ox.ac.uk/~vgg/data/pets/>
519. <https://www.robots.ox.ac.uk/~vgg/data/flowers/>
520. <https://www.robots.ox.ac.uk/~vgg/data/dtd/>
521. <https://sites.google.com/view/fgvc5/competitions/inaturalist>
522. <https://www.inaturalist.org/>
523. Sun C, Shrivastava A, Singh S, Gupta A (2017) Revisiting unreasonable effectiveness of data in deep learning era. In: Proceedings of the IEEE international conference on computer vision, pp 843–852
524. <http://host.robots.ox.ac.uk/pascal/VOC/>
525. <http://host.robots.ox.ac.uk/pascal/VOC/voc2007/index.html>
526. <http://host.robots.ox.ac.uk/pascal/VOC/voc2011/index.html>
527. <http://host.robots.ox.ac.uk/pascal/VOC/voc2012/index.html>
528. Zhou B, Zhao H, Puig X, Fidler S, Barriuso A, Torralba A (2017) Scene parsing through ade20k dataset. In: CVPR
529. Zhou B, Zhao H, Puig X, Xiao T, Fidler S, Barriuso A, Torralba A (2019) Semantic understanding of scenes through the ade20k dataset. Int J Comput Vis 127:301–321
530. https://cs.nyu.edu/~silberman/datasets/nyu_depth_v2.html
531. Cordts M, Omran M, Ramos S, Scharwächter T, Enzweiler M, Benenson R, Franke U, Roth S, Schiele B (2015) The cityscapes dataset. In: CVPR workshop on the future of datasets in vision
532. Cordts M, Omran M, Ramos S, Rehfeld T, Enzweiler M, Benenson R, Franke U, Roth S, Schiele B (2016) The cityscapes dataset for semantic urban scene understanding. In: Proc. of the IEEE conference on computer vision and pattern recognition (CVPR)
533. Gupta A, Dollár P, Girshick R (2019) LVIS: A dataset for large vocabulary instance segmentation. In: CVPR
534. <https://davischallenge.org/>
535. <https://davischallenge.org/davis2017/code.html>
536. Doersch C. Data analysis project: what makes Paris look like Paris?
537. http://www.cs.toronto.edu/~nitish/unsupervised_video/
538. Srivastava N, Mansimov E, Salakhudinov R (2015) Unsupervised learning of video representations using lstms. In: International conference on machine learning
539. Thomee B, Shamma DA, Friedland G, Elizalde B, Ni K, Poland D, Borth D, Li L-J (2016) Yfcc100m: the new data in multimedia research. Commun ACM
540. <http://projects.dfki.uni-kl.de/yfcc100m/>
541. Jin W, Liu X, Zhao X, Ma Y, Shah N, Tang J (2021) Automated self-supervised learning for graphs. CoRR [arXiv:2106.05470](https://arxiv.org/abs/2106.05470)
542. Peng Z, Dong Y, Luo M, Wu X, Zheng Q (2020) Self-supervised graph representation learning via global context prediction. CoRR [arXiv:2003.01604](https://arxiv.org/abs/2003.01604)
543. Zhu Y, Xu Y, Yu F, Liu Q, Wu S, Wang L (2020) Deep graph contrastive representation learning. CoRR [arXiv:2006.04131](https://arxiv.org/abs/2006.04131)
544. Jin M, Zheng Y, Li Y, Gong C, Zhou C, Pan S (2021) Multi-scale contrastive siamese networks for self-supervised graph representation learning. CoRR [arXiv:2105.05682](https://arxiv.org/abs/2105.05682)
545. Hu Z, Fan C, Chen T, Chang K, Sun Y (2019) Pre-training graph neural networks for generic structural feature extraction. CoRR [arXiv:1905.13728](https://arxiv.org/abs/1905.13728)
546. Zhu Y, Xu Y, Yu F, Wu S, Wang L (2020) Cagmn: cluster-aware graph neural networks for unsupervised graph representation learning. arXiv preprint [arXiv:2009.01674](https://arxiv.org/abs/2009.01674)
547. Zhang H, Lin S, Liu W, Zhou P, Tang J, Liang X, Xing EP (2020) Iterative graph self-distillation. CoRR [arXiv:2010.12609](https://arxiv.org/abs/2010.12609)
548. Lin S, Zhou P, Hu Z-Y, Wang S, Zhao R, Zheng Y, Lin L, Xing E, Liang X (2021) Prototypical graph contrastive learning. IEEE trans neural networks learn syst 35(2):2747–2758
549. Subramonian A (2021) Motif-driven contrastive learning of graph representations. Proc AAAI Conf Artif Intell 35:15980–15981

550. Opolka FL, Solomon A, Cangea C, Velickovic P, Liò P, Hjelm RD (2019) Spatio-temporal deep graph infomax. CoRR

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.